Automatic Transcription of Polyphonic Music Using A Note Masking Technique

Ronan Kelly

Masters of Engineering

2010

Title:	Automatic Transcription of Polyphonic Music Using A Note Masking Technique
Author:	Ronan Kelly
Submitted for Award:	Masters of Engineering
University:	University of Limerick
Supervisor:	Dr. Jacqueline Walker

Submitted to the University of Limerick, March 2010

TABLE OF CONTENTS

Abstract.....i
Declaration.....iii
Acknowledgements.....v
List of Tables.....vii
List of Figures.....ix
List of Appendices.....xi
List of Abbreviations.....xiii
Chapter 1 Introduction.....1
Chapter 2 Music and Tuning.....5
Chapter 3 Techniques of Automatic Music Transcription.....21
Chapter 4 Background Research and Implementation of Automatic Music Transcription Techniques.....39
Chapter 5 Note Masks.....65
Chapter 6 Masking System.....77
Chapter 7 Implementation.....95
Chapter 8 Masking System Transcription Results.....113
Chapter 9 Conclusion and Recommendations for Further Work.....123
References.....129
Appendices.....135

ABSTRACT

Ronan Kelly

Automatic Transcription of Polyphonic Music Using A Note Masking Technique

Music transcription is a complex cognitive task that requires a trained musician to listen to a piece of music, write down what notes were played and the timing of the notes. The task is further complicated if the music is polyphonic, where several notes are played simultaneously, requiring the musician to listen repeatedly to the piece of music so as to work out the notes that were played and their timing. This thesis describes a polyphonic note detection system based on a simple masking technique that can accurately transcribe chords and polyphonic piano music. The system, developed in MATLAB, will take input files in .wav format. The music is segmented by using Note Average Energy (NAE) onset detection. The onsets are used to segment the music into note windows which are then analysed using the FFT. Following the extraction and compilation of the frequency peaks in each note window, an iterative masking procedure is used to detect and successively extract the notes and any associated harmonics. The masking procedure uses a database of note masks compiled from multiple note examples using both monophonic and polyphonic examples. The instrument modeled in the work described in this thesis is the Technics KN800 PCM digital keyboard. Once the .wav files have been input into the system, the system will run automatically until completion. A list of notes played and the timing of these notes is output from the system. The run time, from the input of the .wav file to completion of the analysis and output of notes played, can vary depending on variables such as the length, complexity and degree of polyphony of the musical piece entered into the system. The thesis presents the results of testing the system on isolated chords and music played at realistic tempos.

DECLARATION

Herewith I, Ronan Kelly, declare that this thesis is entirely my own work and that I have not used any other sources apart from those given. Furthermore I declare that this work has not previously been submitted for any other academic award.

_____ _____

Signed Date

ACKNOWLEDGEMENTS

I would like to take this opportunity to express my gratitude to my supervisor, Dr. Jacqueline Walker, for her invaluable assistance, guidance and encouragement throughout this research work. I would also like to thank my sisters, Ruth and Niamh, for their support during the years I spent in college and my brother, Anthony, both for his support and his expert keyboard skills. Lastly, I would like to thank my parents, Mary and Denis, for their continuous support and encouragement, without them, I would not be where I am today.

LIST OF TABLES

Table 2.1: Scale structure for Major and Natural Minor Scales
Table 2.2: Examples of Major and Minor Chords
Table 2.3: Note frequencies relative to note C, (normalised frequency of 1)
Table 2.4: Frequency values in the Just Intonation
Table 2.5: Comparing acoustic values from the Just Intonation and values obtained by going up by fifths and down by octaves, both in relation to the C value
Table 2.6: Frequencies of Acoustical notes and the corresponding frequencies in Equal Temperament
Table 2.7: Showing where the harmonic frequencies of note C2 overlap with the fundamental frequency of other notes
Table 4.1: Place Pitch Theory example with a fundamental frequency of 100 Hz
Table 4.2: Place Pitch example with 2 fundamental frequencies, 131 Hz and 247 Hz
Table 4.3: Place Pitch example demonstrating the Octave problem
Table 5.1: Additive nature of overlapping frequencies
Table 5.2: Frequency peaks used for creating a mask for note D4
Table 5.3: The amplitude values of the harmonics of note D4
Table 5.4: Amplitude values of the harmonics of note D4 as a percentage of the fundamental frequency
Table 5.5: Mask values for note D4
Table 6.1: Comparison of the harmonic values of note E3
Table 6.2: Frequency tolerance in note detection
Table 6.3: Detected amplitude values
Table 6.4: Note C4 Masking Calculations
Table 6.5: Remaining values after C4 mask values are removed
Table 6.6: Note E4 Masking Calculations
Table 6.7: Remaining values after E4 mask values are removed
Table 6.8: Note G4 Masking Calculations
Table 6.9: Remaining values after final values are removed
Table 6.10: List of final notes for this window
Table 8.1: Results when note onsets are approximately 0.5s apart
Table 8.2: The percentage of octave error
Table 8.3: Transcribing keyboard songs using the onset detection system
Table 8.4: Transcribing keyboard songs using manual onset detection
Table 8.5: Results for note detection in the octave ranges 3 and 4 and all ranges

LIST OF FIGURES

Figure 2.1: Piano keys and the associated notes
Figure 2.2: Showing G' as the perfect fifth of C
Figure 2.3: Showing G as 3/2 the frequency of C
Figure 2.4: Showing D as 9/8 the frequency of C
Figure 2.5: Frequency plot of note C4 played in isolation
Figure 4.1: A note containing approximately 25,000 samples (≈ 2.08 seconds at 12 kHz sampling rate)
Figure 4.2: A note containing approximately 10,000 samples (≈ 0.83 seconds at 12 kHz sampling rate)
Figure 4.3: Plot of the amplitude of the fundamental frequency peaks of each window, during a note
Figure 4.4: Magnitude of FFT for notes C#3 (139 HZ) & D#3 (156 HZ) played simultaneously
Figure 4.5: Magnitude of FFT for notes C#6 (1109 HZ) & D#6 (1245 HZ) played simultaneously
Figure 4.6: Plots of C#3 & D#3 in (a) Frequency Domain, (FFT magnitude plot), (b) Phase and (c) Unwrapped Phase
Figure 4.7: Plots of C#3 & D#3 (a) first difference of the unwrapped phase and (b) in the frequency domain, (FFT magnitude plot)
Figure 4.8: Power Envelope p(t) of a musical signal
Figure 4.9: Energy e(t) plot of a musical signal
Figure 4.10: Average Energy a(t) plot of a musical signal
Figure 4.11: Sample of Note Average Energy, NAE (t)
Figure 4.12: Note C4 in isolation. The fundamental of C4 and 2nd harmonic have amplitudes of very different heights
Figure 4.13: Note C4 in isolation. The fundamental of C4 and 2nd harmonic have amplitudes of comparable heights
Figure 4.14: Frequency domain plot of notes C3, E3, G3 and C4 played simultaneously
Figure 5.1: Note B2 played with varying intensity
Figure 5.2: Note C2 Mask with 12 harmonics
Figure 5.3: Note C4 Mask with 7 harmonics
Figure 5.4: Graph of harmonic amplitude values of C4 and D4 when played simultaneously

Figure 5.5: Graph of harmonic amplitude values of D4 and A4 when played simultaneously
Figure 5.6: Plot of mask for note D4
Figure 6.1: Flow chart for masking system
Figure 6.2: Frequency plot of C4, E4 and G4 played together
Figure 6.3: Frequency plot of C4, E4 and G4 with the peak at 392 Hz removed
Figure 6.4: Detected amplitude values
Figure 6.5: C4 masking values
Figure 6.6: Detected values and values to be removed by the C4 mask
Figure 6:7: Remaining values after C4 mask values are removed
Figure 6.8: E4 masking value
Figure 6.9: Detected values and values to be removed by the E4 mask
Figure 6:10: Remaining values after E4 mask values are removed
Figure 6.11: G4 masking value
Figure 6.12: Detected values and values to be removed by the G4 mask
Figure 7.1: Transcription System Structure
Figure 7.2: Amplitude peak and decline during note F4
Figure 7.3: Masking process applied to notes C#3 and C#4, and these notes being correctly detected
Figure 7.4: Amplitude values shown with the values to be removed by the C#3 Mask
Figure 7.5: Values remaining after C#3 Mask has been applied
Figure 7.6: Amplitude values shown with the values to be removed by the C#4 Mask
Figure 7.7: Values remaining after C#4 Mask has been applied
Figure 7.8: Masking process applied to notes C#3 and C#4, but C#3, C#5 and G#5 are detected
Figure 7.9: Amplitude values shown with the values to be removed by the C#3 Mask
Figure 7.10: Values remaining after C#3 Mask has been applied
Figure 7.11: Amplitude values shown with the values to be removed by the C#5 Mask
Figure 7.12: Values remaining after C#5 Mask has been applied
Figure 7.13: Amplitude values shown with the values to be removed by the G#5 Mask
Figure 8.1: Frequency plot of notes A3 & B4 played in quick succession
Figure A1.1: Onset Detection System Structure

LIST OF APPENDICES

Appendix A - Onset Detection Implementation
Appendix B - Matlab Code
Appendix C - Masks

LIST OF ABBREVIATIONS

AMT – Automatic Music Transcription
ASR – Automatic Speech Recognition
DFT – Discrete Fourier Transform
FFT – Fast Fourier Transform
HMM – Hidden Markov Model
MIDI – Musical Instrument Digital Interface
MLP – Multilayer Perceptron
Music IR – Music Information Retrieval
NAE – Note Average Energy
SNR – Signal to Noise Ratio
STFT – Short Time Fourier Transform

Chapter 1

Introduction

1.1 Automatic Music Transcription

Music transcription is a complex cognitive task requiring a trained musician to listen to a piece of music and write down the notes played. While automatic transcription of monophonic music is largely considered a solved problem and many examples exist [1], [2], automatic transcription of polyphonic music, to a high level of accuracy, remains a considerable challenge [3]. Researchers into automatic music transcription of polyphonic music have considered a range of different approaches, including neural networks [4], [5], auditory-based approaches [6], [7], probabilistic inference [8], and heuristic signal-processing based techniques [9], [10], [11].

Transcription systems described to date have had limited success. In [4] and [5] it was found that with increasing polyphony the number of spurious notes also increased. Repeat notes where a note was played in succession several times, can also affect results [5], while in [9] it was found that detecting notes with a fundamental frequency less than 200Hz was difficult to achieve accurately. Another major factor in causing errors is the detection of notes that are an octave apart. As the note an

octave above is, by definition, a doubling of frequency, it also corresponds to the second harmonic of the lower note. Octave errors are common throughout the automatic transcription field [4], [5], [7], [12], [13].

Approaches to automatic music transcription can be divided into two broad types. One approach taken is to try and reproduce the characteristics of the human hearing system [14], [15]. In these approaches, a simplified model of the human auditory system is constructed and used for initial processing of the musical signal. The second approach, which is the focus of the work presented in this thesis, is to try and transcribe exactly the notes that were played using traditional signal processing techniques [9], [10], [11], [16].

In the auditory field, *masking* occurs when a louder sound adjacent (in frequency) to a quieter sound prevents the quieter sound being heard [17]. In computer science, a *bit mask* is used to conceal bits in a word to allow the retrieval of information of interest [18]. In this research the term *masking* is relevant in both of these senses: it refers both to how the presence of overlapping harmonics from different notes in the same window leads to an apparent masking of notes by other more dominant notes and to the use of a mask for blocking out all but a particular note from a set of frequencies in the portion of music under analysis. Essentially, a masking approach is applied to polyphonic note detection. Once a note has been identified, it is removed from the signal (masked) and the system attempts to identify the next note and so on. The procedure continues until no more notes can be found in that segment of the music.

In music signal identification and transcription, one of the most problematic issues is amplitude variability: notes can be played softly as well as loudly, the amplitudes varying along a continuum. To solve this problem, the amplitudes must be analysed in proportion to the values in the rest of the window. The key insight in the masking scheme is that frequencies present in a window that overlap from different sources (notes) are approximately additive in amplitude.

The main assumptions in the work reported here are that the music is polyphonic Western Tonal music consisting of either isolated piano chords or polyphonic piano music. As the music is polyphonic, it is not known a priori how many simultaneous notes exist in each analysis window. It is further assumed that all notes have the fundamental present. The range of notes considered is C2 to B6 and the instrument used in this research is the Technics KN800 PCM Keyboard [19].

1.2 Applications of the Masking System

Music transcription is a difficult and time consuming task that can only be carried out by trained musicians. Automatic transcription would greatly speed up this process and make the output more reliable as opposed to the subjective nature of a musician listening to a musical piece over and over, trying to evaluate the notes that were played, as listeners may be very sensitive to certain types of errors but find it difficult to correctly transcribe other areas of composed music.

There are several motivations for automatic music transcription. One of the most basic reasons is for study and research. When music is being studied, particularly traditional music or improvised music, where a score does not necessarily

exist, transcription can help determine what notes were played. This may particularly apply to music from cultures which do not use Western Tonal music and specialised versions of a transcription system tuned to the particular tonal system would need to be developed. Transcription could also be used to change musical pieces into a compressed representation such as MIDI files without human intervention, which are widely used in music.

A relatively new application for automatic music transcription is Music Information Retrieval (Music IR) [20]. Instead of using a text based search engine or database searching system, the music itself would be used. For example, when searching for a piece of music where the name of the piece is unknown but a portion of the tune can be hummed or played by the user, then Music IR would be the perfect solution. Advances in music transcription research can contribute to improved techniques for Music IR.

Techniques developed through research into automatic music transcription could also be used effectively in other applications for automatic processing of music. For example, removing noise from recorded music, isolating the singer or particular instruments from a recording or allowing home users to manipulate tracks automatically.

Chapter 2

Music and Tuning

2.1 Music

When discussing music, it is generally accepted that there are two main types of music, Western Tonal and Non-Western music [21]. Here we take Middle Eastern music as one example of Non-Western music. Eastern or Oriental music has its origins in the Middle East and Western Tonal music has its origins in the Western world, Europe and the U.S. The characteristics of these two approaches to music vary greatly, although in recent times more modern Eastern music has been influenced by Western Tonal styles of music [22].

2.1.1 Western Tonal Music

In Western Tonal music there are twelve distinct elements or notes which are denoted by the letters A to G, including sharp (#) and flat ($_b$) notes. These notes are C, C# (also known as D_b), D, D# / E_b, E, F, F# / G_b, G, G# / A_b, A, A# / B_b and B. Each element is a half step (semitone) higher than the one immediately preceding it. Note that in some cases a given element may be known by two different names – one

involving a sharp and the other a flat. These 12 distinct steps compass an octave [23] which corresponds to a doubling of frequency. The frequency range of a given instrument may include several octaves. A note in a specific octave is denoted as C4, A5 where the number denotes the specific octave, the frequency designation of which may vary under different systems [24]. Note C4 and note C5 are one octave apart, meaning that note C5 is double the frequency or pitch of note C4.

Usually 8 notes, comprising a scale, are selected as the basis for a given piece of music and there are certain rules regarding which 8 notes can be used in a certain scale. Table 2.1 gives two examples of scales, Major and Natural Minor scales. For example, a typical Western song in the scale, or played in the key of, C Major will use the tones C, D, E, F, G, A, B and C', where C' is one octave higher then the first C note, as the basis for the song. The first note in a scale is known as the tonic or keynote. These scales or keys are the basic building blocks for a given song.

In Table 2.1, W represents a note change of a whole step, or tone. H represents a note change of a half step, or semitone.

Scale Type							
Major	W	W	H	W	W	W	H
Natural Minor	W	H	W	W	H	W	W

Table 2.1: Scale structure for Major and Natural Minor Scales

For example A Major consists of notes: A, B, C#, D, E, F#, G#, A', where A' is one octave above A.

Western Tonal music is polyphonic, which means “many sounds” and so it includes harmony and chords. Harmony is the overlapping of one or more notes [25], which produces a pleasing sound to the ear and is due to the particular combination of frequencies produced by the played notes, as described in Section 2.2. A chord is a combination of three or more notes played simultaneously [26]. Western Tonal music makes extensive use of chords, which are constructed, according to certain rules, from the notes in the scale that the song is played in. Examples of chords, see Table 2.2 below, would be Major and Minor. The C Major chord for example, is comprised of tones C, E and G.

Chord	Major	Minor
C	C,E,G	C,D#,G
D	D,F#,A	D,F,A
E	E,G#,B	E,G,B
A	A,C#,E	A,C,E

Table 2.2: Examples of Major and Minor Chords

Classical music from Europe relies heavily on chords although, for example, in the early 20th century Schoenberg created a form of Serialism, also known as dodecaphony, where the number of times all 12 notes are used in a piece of music is equal, so negating the need for playing music in a specific key or scale [27]. Finally, in Western Tonal music, there are usually no changes made to the original written music; western musicians do not improvise much, except in certain kinds of jazz [28].

2.1.2 Eastern Music

To a Western listener, Eastern music may sound out of key. Eastern music theory is different from Western Tonal music theory and the music itself can sound very different. Some factors which are significantly different include the use of quarter tones, the absence of chords or harmony, the distinctive rhythms used, and the use of improvisation.

The smallest inter-note relationship in Eastern music is the quarter-tone. If we were to visualise the piano in terms of Eastern music, we would still see the C and the C# piano keys, but we would now see a note halfway in between. These are called quarter tones and are not part of Western Tonal music. Instead of having 12 possible tones to use as the basis of building a scale, Eastern music has twice as many possible tones to choose from [22]. It would be impossible to play an Eastern song that uses quarter tones on many standard Western instruments such as a piano. It is possible to tune some Western instruments to play music with quarter tones, but the standard instrument tuned for normal Western symphonic or concert band music would not be capable of playing quarter tones.

There are many different styles of Eastern music, here we discuss Arabic or Middle Eastern music as an example. The music theory used in Arabic music is known as maqam theory. A maqam is like a scale in Western theory, but has some extra rules or conventions added on. The maqam is based not on a group of eight out of twelve notes, but a select group of notes from all the available notes in Arabic music (of which there is a large variety, due to the many available quarter tones) [22][28][29].

Mostly traditional Arabic music places emphasis on melody and rhythm. There are some genres of Arabic music that use polyphony but typically Arabic music is homophonic [22]. The music is sung or played in a solo, or by a single musician. Sometimes there is a group of musicians who repeat what the soloist sang or played, but often without polyphony. Part of the reason why there is little polyphony is because the soloist often improvises or makes changes in the music each time he/she plays or sings it. Traditional Arabic music is monophonic for each individual instrument and does not use chords, although harmonic counterpoint is used. When multiple instruments play, then usually one instrument will carry the primary melody, while the others will layer melody scraps or rhythm segments over it. Improvisation is highly prized in Arabic music and may have an improvised section with no accompanying rhythm before the song, or may have an improvised section within the piece. These improvised sections are known as taksims. In a taksim, a musician shows off their skills in playing their instrument, as well as their understanding of the musical theory [29].

Since the beginning of the early 20th century, Eastern composers have become more influenced by Western Tonal music and have begun to use harmony and chords in their music. The structure described above applies reasonably well to older, folkloric music, but more recent works may incorporate Western influences [29].

2.1.3 The Piano

The piano can stand as representative of Western Tonal music because it consists of 88 keys providing groups of 12 notes, which are repeated over 8

successive octaves, ranging from A0, (27.5 Hz), to C8, (4186 Hz) [30]. On a standard piano each white piano key and each black piano key represent one of the 12 possible tones that can be used in Western Tonal music. Each tone, represented by a piano key, is a half step (semitone) higher then the piano key immediately adjacent to its left. For example on a piano, the white piano key of C is a half step below the black piano key of C# / D_b. These 12 tones encompass an octave [23]. The keyboard (Technics KN800 PCM Keyboard [19]), used in this research has 61 keys, ranging from note C2 (65.41 Hz) to C7 (2093 Hz). Although there are fewer keys on the keyboard, the missing notes, compared with the piano, are in the lowest and highest octaves, which are not used as often.

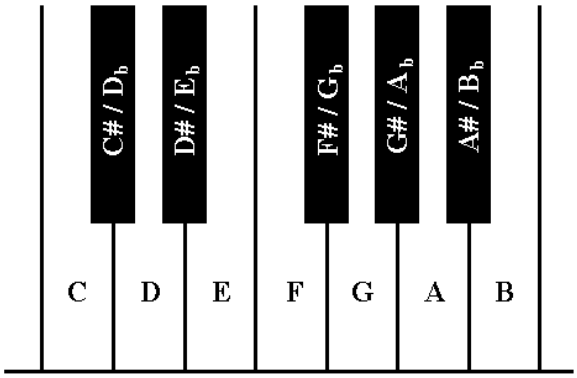


Figure 2.1: Piano keys and the associated notes

2.2 Tuning of Western Tonal Music

The sound of a certain note is defined by its pitch which is the perception the listener has of its fundamental frequency [31]. The fundamental frequency, [25], is the lowest resonant frequency of a vibrating object. For example, the note C4, (Middle C), has a fundamental frequency of 261.63 Hz, but when the note is played, other frequencies are generated as well which make up C4 and give it a full sound

that is pleasant to listen to. These frequencies are known as harmonics or partials and occur at integer multiples of the fundamental frequency. With the fundamental frequency also known as the first harmonic, the second harmonic of C4 is twice the fundamental frequency at 523.25 Hz, the third harmonic is triple the fundamental frequency at 784.89 Hz and so on.

The fundamental frequencies of the notes of Western Tonal music may be determined by the equation:

$$f_0 = 440\left(2^{n/12}\right) \qquad \text{for n = -33, -32…27} \qquad (2.1)$$

which equates to C2, C2#…C7, which is the range of notes used in this work. This formula is used to calculate the fundamental frequency of notes in Equal Temperament, which is discussed further in Section 2.2.2.

2.2.1 Pythagorean Comma

In the field of acoustics, there are two main rules, or axioms, that define how a scale is constructed. In this example, taken from [32], the scale of C Major is used, as it will avoid the use of sharps and flats for simplicity. For a given frequency, these axioms are firstly: to go up one octave, double the frequency; secondly, tripling the frequency moves to the perfect fifth in the next octave above the doubled frequency. The perfect fifth is thus also the second harmonic of the first note played, as it is a tripling of frequency. As an example, taking C4 and normalising its frequency to 1, (for ease of comparison), then C'' is the note one octave higher than C', which is one octave higher then C and the scale is constructed as follows:

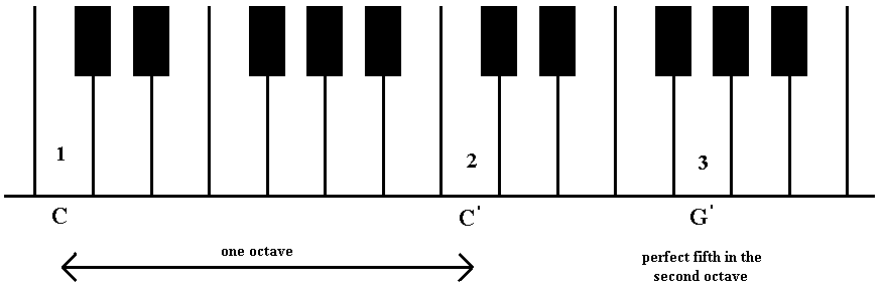


Figure 2.2: Showing G' as the perfect fifth of C

Let C have a frequency of 1, C' is one octave above and therefore is double the frequency of C. Therefore C' has a frequency of 2. Tripling the frequency of C gives the perfect fifth, which gives G', with a frequency of 3 as shown in Figure 2.2.

$$G' = C*3 = 3 \tag{2.2}$$

Halving this frequency gives G, with a frequency of $3/2$ as shown in Figure 2.3.

$$G = G'/2 = 3/2 \tag{2.3}$$

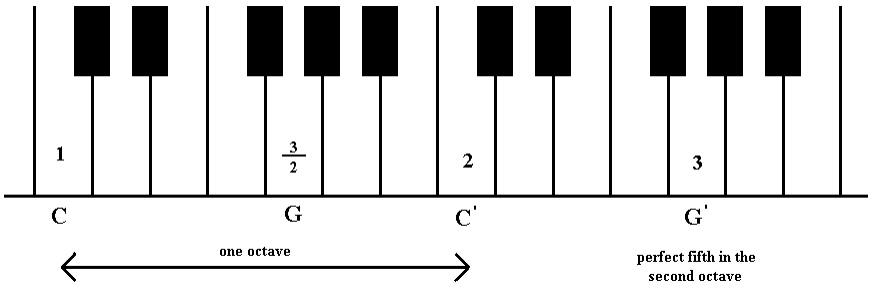


Figure 2.3: Showing G as 3/2 the frequency of C

D" is the perfect fifth of G, so to get D" the frequency of G is tripled;

$$D'' = G*3 = 3/2 * 3 = 9/2 \tag{2.4}$$

To get D' the frequency of D" is halved;

$$D' = D''/2 = 9/2 / 2 = 9/4 \tag{2.5}$$

To get D, the frequency of D' is halved;

$$D = D'/2 = 9/4 / 2 = 9/8 \tag{2.6}$$

Figure 2.4 shows how D is established.

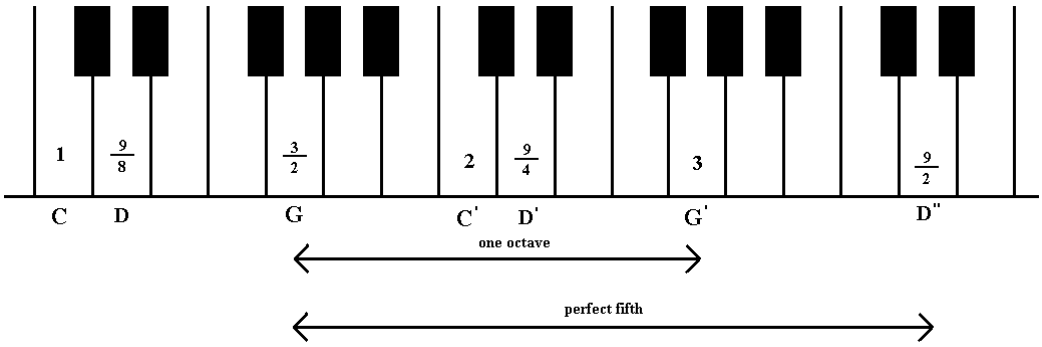


Figure 2.4: Showing D as 9/8 the frequency of C.

Using these two rules, by tripling then halving of frequencies, all notes in the scale can be established. Table 2.3 lists the subdivisions in the instance of the note C only.

Note	Frequency
C	1
D	9/8
E	81/64
F	177147/131042
G	3/2
A	27/16
B	243/128
C'	531441/262144

Table 2.3: Note frequencies relative to note C, (normalised frequency of 1)

An alternative tuning scheme is known as the Just Intonation, also known as harmonic tuning, occurs naturally in simple systems such as air columns and vibrating strings, where the string is of fixed length and is fixed at both ends, such as a piano string [33]. Table 2.4 shows the frequency values of each tone based on the Just Intonation.

Note	Frequency (Hz)
C	264
D	297
E	330
F	352
G	396
A	440
B	495
C	528

Table 2.4: Frequency values in the Just Intonation

In Table 2.5 we compare Tables 2.3 and 2.4, where the frequency values in Hertz in Table 2.4 are changed to values relative to C at 264 Hz.

Note	Just Intonation	Up by Fifths, down by octaves
C	1	1
D	9/8	9/8
E	5/4	81/64
F	4/3	177147/131042
G	3/2	3/2
A	5/3	27/16
B	15/8	243/128
C	2	531441/262144

Table 2.5: Comparing acoustic values from the Just Intonation and values obtained by going up by fifths and down by octaves, both in relation to the C value

From Table 2.5, we can see that the method of going up by fifths and down by octaves gives slightly different values in comparison with the actual acoustic values. This is known as the Pythagorean (or diatonic) Comma and is the reason that a piano cannot be tuned perfectly.

2.2.2 Equal Temperament

Equal Temperament [66], also known as Even Temperament, is the tuning method most used in Western Tonal music to compensate for the Pythagorean Comma [32]. Equal Temperament can start at any frequency as it is a relative subdivision of the

octave. Here A4 with a frequency of 440 Hz is used as a reference point. Equal Temperament divides the octave into 12 semitones, where the ratio of the frequencies of any two adjacent notes is constant as shown by Equation 2.1. Therefore the only interval that is acoustically correct is the octave [32]. The error of the Pythagorean Comma is spread evenly throughout the octave, meaning that none of the notes will be tuned to their exact acoustical frequency, with the exception of A4 at 440 Hz, the reference point. Table 2.6 shows the acoustical Just Scale frequency of the notes in the C Major scale, and the frequency values of these notes in Equal Temperament, calculated using Equation 2.1.

Note	Just Scale (Hz)	Equal Temperament (Hz)
C4	264	261.63
D4	297	293.67
E4	330	329.63
F4	352	349.23
G4	396	391.96
A4	440	440
B4	495	493.88
C5	528	523.25

Table 2.6: Frequencies of Acoustical notes and the corresponding frequencies in Equal Temperament

2.2.3 Issues in Frequency Detection due to Equal Temperament

When trying to extract note identities from polyphonic music in the frequency domain using a Fast Fourier Transform, (FFT) (see Section 4.1), it is inevitable that amongst the harmonics of a note, the fundamental frequencies of other notes will be detected. For example, if C4 is played then amongst the harmonics of that note will be detected the fundamental frequencies of notes C5 and G5 because the second

harmonic of note C4 has the same frequency value as the fundamental frequency of note C5. Moreover, the third harmonic of note C4 is very close in frequency to the fundamental frequency of note G5. As shown by the discussion in Section 2.2.1, the third harmonic of note C4 and the fundamental frequency of G5 should have the same frequency, but because of Equal Temperament, there is a small difference between the frequencies. G5 has a fundamental frequency of 783.99 Hz and the third harmonic of note C4 has a frequency of 784.89 Hz. Although it might seem that this would be a loophole which could be exploited to improve note detection, for example by using an FFT with a very high frequency resolution, it is in practice, impossible to distinguish between them.

It is also not useful to make this distinction for various reasons. With recordings, inevitably some noise will be in the piece of music and may slightly alter the detected frequency values. Piano tuners use stretched tuning which further moves the temperament and octave displacement away from an ideal tuning. This effect is know as inharmonicity [67]. Also, measured frequency of a given note can vary from its transient stage to its steady state stage [68]. Due to these reasons there has to be a tolerance allowed around the frequency of the detected notes. For example, during this research, the fundamental frequency of note C4 ‘as detected’ can range from approximately 260 Hz to 262 Hz, depending on the recording. Figure 2.5 shows note C4 played in isolation. Here the fundamental frequency of C4 can be seen as well as the harmonics of C4.

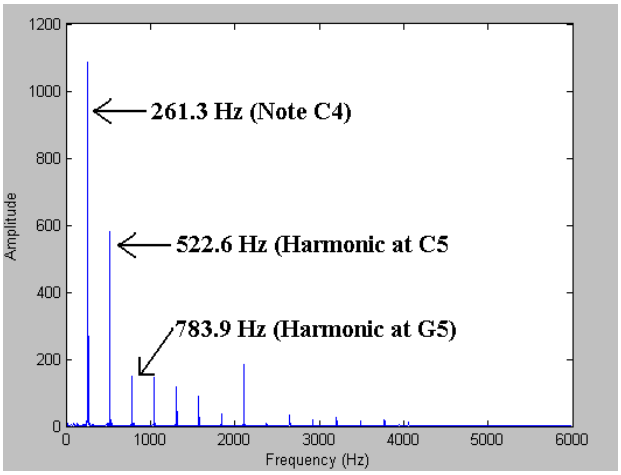


Figure 2.5: Frequency plot of note C4 played in isolation.

This overlapping of fundamental frequencies and harmonics is common and it is for this reason that some notes sound well when played together and why some notes are not usually played together as the resulting sound is not pleasant to the ear. Notes that are "in harmony", (i.e. their harmonics overlap), with each other generally sound good together, such as C4 and G4, but playing C4 and D4 together does not work as well because the harmonics do not overlap and so the notes will not sound good when played together: the sound is less coherent. Consonant chords, which are harmonious, are pleasing whereas in dissonant chords, notes sound as if they are clashing. Dissonant chords are sometimes used to create a feeling of tension in music, which may then be resolved by moving to a consonant chord.

Table 2.7 shows an example of the frequencies that make up note C2, (65.41 Hz), and shows where the frequency of some of the harmonics of C2 overlap with the fundamental frequencies of other notes. With the detection system implemented in the course of this research, the fundamental frequencies of the notes as detected can vary slightly, so those variations have to be taken into account when dealing with harmonics. A harmonic of a note may not have exactly the same frequency as the

fundamental frequency of another note, but would be seen by the detection system as the same within the tolerance allowed in the system. This is illustrated in Table 2.7 with the use of approximately corresponding notes and frequency values. As the frequency increases the harmonics of note C2 deviate increasingly from the nearby frequency values of the fundamental frequencies of the other notes. This produces less overlapping between the frequencies of the harmonics of note C2 and the fundamental frequency of other notes, so the problem becomes less acute in the high harmonics. This table also illustrates why equal temperament induced frequency differences are of little value for detection as the similarity of the fundamental frequency of overlapping notes and the harmonics of a lower note could cause the incorrect note to be detected.

Note C2

Harmonic	Frequency (Hz)	Overlapping Note	Fundamental frequency (Hz) of overlapping note
1st	65.41	~	~
2nd	130.82	C3	130.81
3rd	196.23	G3	196.00
4th	261.64	C4	261.63
5th	327.05	E4	329.63
6th	392.46	G4	392
7th	457.87	~	~
8th	523.28	C5	523.25
9th	588.69	D5	587.33
10th	654.1	~	~
11th	719.51	~	~

Table 2.7: Showing where the harmonic frequencies of note C2 overlap with the fundamental frequency of other notes

There are many different styles of music in Western Tonal Music, from Jazz to classical, rock to pop, but this research has no particular emphasis on which style of music can be transcribed, other than the music being Western Tonal. The general consensus within Western Tonal Music is to use Equal Temperament as the tuning method, and it is therefore the tuning method that is used in this research.

Due to polyphony in Western Tonal music and the Equal Temperament tuning method, establishing the correct notes that have been played is a complex procedure. The harmonics of a note should not be mistakenly detected as a fundamental frequency of another note and therefore the harmonics of each note need to be accounted for. With Equal Temperament the exact frequency of the fundamental frequency of a note cannot be established, and note identification is made more difficult if there is noise in the music. For these reasons, a detected frequency alone cannot be used to identify a note and therefore a system for detection needs to take other note attributes into account when transcribing music.

Chapter 3

Techniques of Automatic Music Transcription

3.1 Present Investigation

Music transcription is a complex cognitive task requiring a trained musician to listen to a piece of music and write down the notes played. While automatic transcription of monophonic music is largely considered a solved problem and many examples exist [1], [2], automatic transcription of polyphonic music, to a high level of accuracy, remains a considerable challenge [3]. Researchers into automatic music transcription of polyphonic music have considered a range of different approaches, including neural networks [4], [5], auditory-based approaches [6], [7], probabilistic inference [8], and heuristic signal-processing based techniques [9], [10], [11].

Approaches to automatic music transcription can be divided into two broad types. One approach taken is to try and reproduce the characteristics of the human hearing system [14], [15]. The cochlea in the human ear responds to the vibrations transmitted to it through the ear. It is known that the human ear has a logarithmic response to sound intensity and that different frequencies occurring in a note,

stimulate different parts of the basilar membrane. However, what we hear is not always determined exactly by the data in the sound and this can be seen in the phenomenon known as the missing fundamental [34] - [36]. When we hear a tone, the pitch we perceive is typically equivalent to the fundamental frequency of that tone. If for example, a sound has no energy present at the fundamental frequency and a listener heard only the harmonics present in the tone, a listener would still perceive the pitch of the tone as equivalent to the fundamental frequency [35]. It is not known exactly why this phenomenon occurs but it is surmised that the pitch that is heard by the listener is calculated using the frequency distance between the other harmonics to work back to the fundamental frequency. From the listener's point of view, although the fundamental frequency is removed, the pitch of the tone remains the same [36]. This demonstrates that the human ear and brain interpret the sound or tone and a listener 'creates' a perception of the sound or tone. The human hearing system is a complicated subject, as discussed in [37], and is perception based, whereas transcription of music tries to establish what notes were played. In the approaches by [14], [15], a simplified model of the human auditory system is constructed and used for initial processing of the musical signal. The second approach to music transcription, which is the focus of this research, is to try and transcribe the notes that were played using traditional signal processing techniques [9], [10], [11], [16]. These techniques will be discussed in further detail throughout the chapter.

3.2 Techniques of Automatic Music Transcription

3.2.1 Hidden Markov Model Approaches

In music there are, in theory, no restrictions on what notes can be played and in what order. There are of course varying types and styles of music, as explained in Section 2.1, which implicitly apply rules to musical content, but the aim of this project is to accurately transcribe polyphonic keyboard music, irrespective of what notes are played.

In ASR, (Automatic Speech Recognition), patterns can help to discover what phones [41] - [43] were played by altering the probabilities of certain combinations of phones occurring. Beginning in the 1980s statistical methods were employed in ASR, in particular Hidden Markov Models, (HMM) [38], [39]. HMMs are based on a mathematical model known as a First Order Markov Chain. A First Order Markov Chain consists of a set of q states, and describes the probability of a change of state: including the possibility of remaining in the same state, and a set of associated probabilities. The fundamental principle of the First Order Markov Chain is that the probability of moving from a given state to another depends only on the present state. In the Markov Chain, all the states are observable. For real world situations, this is not always applicable, and instead we model the situation where a certain set of observations arise from an unobservable or hidden, stochastic process which is the Markov Chain. This is called a Hidden Markov Model (HMM) [40].

Although at first ASR and Automatic Music Transcription seem to be similar, there are some fundamental differences that make the automatic transfer of successful ASR techniques to solve the problem of note extraction in music transcription

difficult. In speech no word is spoken with exactly the same features, due to factors such as inflection, emotion and emphasis. For this reason most ASR systems have a large database or a dictionary of speech samples. These samples are usually samples of phones, which are the smallest part of speech [41] - [43]. The ASR system tries to match the users spoken input to a sample found in the dictionary or database. HMMs are used to decide which phone was spoken based on probabilities. HMMs are also useful in solving the problem of homophones, which are phones that sound the same but are spelt differently. For example take the sentences “The stuff he knows” and “The stuffy nose” [44], both sound the same but have completely different meanings. In speech recognition a set of rules based on context and grammar will alter the probability of certain words following other words. These rules and probabilities also contribute to the HMM correctly recognising the speech. With music transcription some of these problems do not occur. When a given note is played, it will always have the same fundamental frequency. The use of a large database of sample notes is not necessary as, in principle, all that is needed to tell what note is played is its fundamental frequency. On the other hand, there is an almost infinite possibility of variation in the patterns of notes that can follow in each other in sequence in a melody. There is also an infinite possibility of variation in amplitudes and timbres of such notes. Thus, it would be very difficult to compile a representative database of examples.

HMMs have been used in frequency tracking systems [45], [46] and there are also other frequency tracking systems that do not use HMMs, [47], [48]. These systems are used to track and reconstruct the fundamental frequency of signals, which are embedded in background noise, for example in sonar signals. In frequency tracking systems, reconstruction of the signal is also not necessary as the SNR is

relatively large and so background noise does not affect the fundamental frequency estimation so greatly. These systems, [45] – [48], track the fundamental frequency of a signal, but for a polyphonic signal, there may be several different fundamental frequencies between an onset and an offset. HMMs are well suited to frequency tracking, as they track the most probable solution through a complete set of all possible solutions. On the other hand, to employ a HMM to establish the fundamental frequency of a monophonic music signal between an onset and an offset would be "overkill" as this can be done relatively easily.

However, HMM systems have been used successfully in certain areas of music recognition, such as rhythm detection [49]. Rhythm detection focuses on the timing of the music, extracting note duration and tempo. Rhythm extraction is analogous to continuous speech recognition and frequency tracking, as the rhythm, which may change several times, needs to be tracked through the signal. In [49], rhythm recognition of MIDI, (Musical Instrument Digital Interface), is performed using HMMs. By using MIDI music, the note values are known a priori, so the system can focus on the extraction of the rhythm. The system attempts to recognise a given sequence of fluctuating note durations to find the most likely intended note sequence. HMMs are ideally suited to continuous analysis throughout the duration of a signal. They are utilised in this rhythm detection system by combining a stochastic model of deviating note durations and a stochastic grammar model representing possible sequences of notes, which is then searched for the maximum likelihood estimate of the note sequence. This same principle is applied to time measure, (time signature), recognition and tempo estimation with the combined results leading to rhythm estimation.

HMMs have also been successfully implemented in additive sound synthesis [50]. Additive sound synthesis is the process of constructing artificial sounds by mixing simple waveforms together to create a more complex waveform. Additive synthesis is a complex task and in [50] the focus is on the peak matching procedure, which is the most difficult part of the analysis process. A combinatorial HMM system is used to track partials or overtones, by identifying time functions of parameters in the music. The HMM is constructed using a combination of several elements to create a HMM that can be accurately applied to frequency line tracking. Again this system tracks frequency partials through a signal, and so is ideally suited to HMM analysis, due to the continuous nature of both the HMM and the frequency components throughout the signal.

3.2.2 Note Onset Detection

The two main tasks in music transcription are note onset detection and deciding whether the subsequently detected frequencies represent the fundamental frequencies of a note or harmonics of a note. Using the onset detection system [51] described in Section 4.3, the onsets and the offsets for each note or chord can be identified. Therefore the musical signal is broken up into windows, according to the onsets and offsets of the notes or chords. These windows are then analysed to see what note or notes were played in a specific window of information. In [50] a relatively complex model is used to track a fundamental frequency through a signal. In this research, since the musical signal is broken up into windows, which may contain several notes in each window, tracking a single frequency through the signal is not needed. This would result in the tracker locating a specific frequency and where it occurs in the signal. There would need to be a frequency tracker for every note,

making for a complex system of frequency trackers. To add to the complexity, when a frequency is tracked, a decision on whether the frequency is a fundamental frequency or a harmonic would still need to be made, and this process would need to be carried out throughout the signal, each time the frequency was detected. Using the onset detection system, if the musical signal is broken into windows, roughly corresponding to notes, then the analysis of each window can be carried out without the complexities of the frequency tracker and so an onset detection system is used in this research [51].

Speech recognition systems make good use of HMMs but in speech recognition there is only one speaker at a time, speaking with one voice, therefore only one sound is made at a time. With music transcription, there can be several instruments, analogous to several voices, where each may be playing a number of notes, which is analogous to a speaker speaking with several voices at the one time, which of course does not occur.. This research focuses on a single instrument, the [electronic] piano, but the piano is capable of producing polyphonic music so a system that can transcribe polyphonic music is needed. HMMs could conceivably be used in monophonic music, but as discussed, may represent an over elaborate solution. Should polyphonic detection be needed, the situation would become even more complex as there may be several notes being played at the same time. Also if a note or notes are played in music, for example if a chord is played, it may be held when played and another note can then be played. This overlapping does not occur in speech as a person can only speak with one voice and this would be very difficult to model using a HMM.

It is these small but decisive factors that preclude ASR techniques from solving the problems associated with Automatic Music Transcription. Although a good starting base from which to learn about different techniques for audio transcription, different methods, more tailored towards music need to be investigated.

3.2.3 Neural Networks for Music Transcription

Artificial neural networks are designed to model biological neural networks, so as to complete complex cognitive tasks using technical equipment such as computers or electronics [52]. Neural networks have been used in speech recognition [53] because of their ability to learn from examples, generalise and robustness to noise. Due to these abilities, neural networks are suitable for the task of polyphonic music transcription and are utilised in systems such as those described in [5], [54].

In [4] a system for transcribing polyphonic piano music is discussed. Here the task is split into three stages. The first stage is to transform the audio signal into time-frequency space, based on the workings of the cochlea in the human ear. Gammatone filterbanks are used to split the signal into several frequency channels. Filter implementation is based on the models suggested by [54] and implemented by [55]. The output of each filter is processed by Meddis' model of hair cell transduction [56]. The hair cell model converts each filter output into a probabilistic representation of firing activity in the auditory nerve. Next the outputs of the filters are put into inputs of a network of adaptive oscillators, which adapt their phase and frequency in response to external input. These adaptive oscillators try to synchronise to the signal by adjusting its phase and period to that of the input signal. An estimation of the frequency of the input can be made by observing the synchronised oscillator. In this

partial tracking section, there are 88 groups of oscillators, with each group containing a base oscillator that is tuned to the fundamental frequency of one piano note. The final section is the transcription section, which consists of 88 neural networks with one network for each piano note. Outputs of the 88 neural networks represent the occurrence of piano notes in the input signal. This system was shown to give good results when used for isolated chord recognition, but when transcribing music, the system does not perform so well, as there are a lot of extra notes detected by the system, with the most common errors being octave errors, especially in the middle octaves.

In [57] time-frequency analysis techniques are combined with a neural network classifier to create a music analysis and recognition program for guitars, called Paganini. First a multi-resolution time frequency analysis is carried out on the input signal. The input signal is passed through a set of narrow band filters, with each centred on a different frequency, generating the envelopes of the outputs as separate signals, which are then logarithmically scaled, to mimic the function of the human ear. For each frequency where notes may exist, a window size is calculated to provide a specific separation between the note of interest and its closest-in-frequency neighbour. The DFT of a block of samples of this size is then evaluated at the frequency of interest. The window is then moved further along the sampled waveform and the entire procedure is repeated for the whole recording at each frequency where notes could exist.

The second stage is the neural network, which is used to differentiate between notes and their harmonics. This involves a back propagation neural network which takes an array of floating point numbers as input, one for each possible note, and

activates one output out of forty-six, (number of notes on a guitar), to indicate a match, or none if the input is a rest. The numbers fed into the neural network classifier are a time slice of the results from the time-frequency analysis stage. The output is a set of numbers of which one should be close to unity, (close to zero for a rest), for each note present in the input signal at that time. The network will then be trained to recognise a given spectral pattern as a particular note. Once the network is trained, the music analysis process runs the network in the forward direction on an input time slice with unknown note values. This results in the output of the onset time, frequency and duration of the identified notes. This system does a reasonable job of extracting some of the key features of music, but there is a wide variability in the performance of the program.

The human ear has a logarithmic response to sound intensity and both these systems try to mimic the human ear in different ways in different sections. In [4] the signal is split up into several frequency channels and is then filtered based on a model of the hair cells in the human ear. In [57] after the input signal is passed through the narrow band filters, the outputs are logarithmically scaled to mimic the hearing function of the human ear.

Both systems used neural networks in their analysis, at least one network for each note. The music signal was divided up into different streams and then the frequency is tracked through the signal. In [4] 88 neural networks are used to recognise the occurrence of a single note from the input. This again is a method that can be related to the human ear, where it is surmised that the human ear breaks the signal into different streams, which are then analysed individually by the brain [37].

3.2.4 Computational Blackboard System

In [7] a computational blackboard system is used to transcribe piano performances of four-voice Bach chorales written in the style of 18th century counterpoint. Blackboard systems were designed to bring together different forms of knowledge and integrate them into a solution for a particular problem. Three forms of knowledge were brought together to create the blackboard system in [7]; knowledge about human auditory physiology, knowledge about the physics of sound production and knowledge of the structure of tonal music, 18th century counterpoint music in particular.

A simple front end using Short term Fourier transform, (STFT), performs time frequency analysis of the sound signal and converts it into a simplified discrete representation. The short-time running energy in the acoustic signal is measured by squaring and low-pass filtering of the signal. Sharp rises in the running energy are interpreted as note onsets and are used to segment the time-frequency analysis into chunks representing individual musical chords.

The input to the blackboard system is a list of tracks received from the front end. Each track has an associated onset time, frequency, and magnitude. Blackboard systems consist of a central repository, the blackboard, knowledge sources and a scheduler. Each knowledge source, ordered by expected decreasing benefit, is made up of a precondition and an associated action. The scheduler runs the system in a series of steps. The scheduler begins with the first knowledge source and evaluates the precondition of each knowledge source in turn. When a precondition is satisfied, the associated action is performed and the scheduler moves on to the next step.

In [7], the blackboard workspace consists of a hierarchy of five levels. They are, in order of increasing abstraction: Tracks, Partials, Notes, Intervals, and Chords. The knowledge sources in the system fall under three broad areas of knowledge: garbage collection, knowledge from physics, and knowledge from musical practice. The system implemented in [7] varies from the originally designed system. The implemented system can transcribe synthesized piano performances in which two notes an octave apart are never played simultaneously and only in the range of B2 (123 Hz) to A4 (440 Hz). These limitations are due to three problems; firstly the inability to correctly distinguish between two notes which are an octave apart is due to physical ambiguity and to a lack of musical knowledge. The second problem is due to a poor assumption in the rating function for note hypotheses. Higher notes in a piano's range do not have strong upper partials and so the note hypothesis rating function, does not take this into account, and therefore note hypotheses with pitches above A5 (440 Hz) will not generally have a high enough rating to be accepted as valid. The front end imposes a third and final limitation on the system. It makes assumptions that all notes in a chord are struck simultaneously and that the sounded notes do not modulate in pitch.

While the system described in [7] has a lot of limitations, it is the flexibility of the blackboard system that makes it possible to add new knowledge sources into the system, which can add to the overall capacity of the system to transcribe a greater range of music to a greater degree of accuracy.

3.2.5 Connectionist Approach

In [5] a connectionist approach to automatic transcription of polyphonic piano music is presented. [5] makes use of a partial tracking system which is based on a combination of an auditory model and adaptive oscillator networks. A system called SONIC was developed which included the partial tracking system, time-delay neural networks and an onset detector, used for detecting repeat notes and estimating length and loudness.

The onset detection, used to improve the accuracy of the onset times of notes found by the system, uses an algorithm that splits the signal into 22 overlapping frequency bands, each covering half an octave, with a bank of gammatone filters. Channels are full-wave rectified and then processed with a filter that calculates the difference between two amplitude envelopes. The output of the filter has positive values when the signal rises, otherwise negative values are the output. The output of all 22 filters is then entered into a fully connected network of integrate and fire neurons. Outputs from this network of integrate and fire neurons are a series of impulses indicating onsets in the signal. Not all of these impulses represent onsets, so an MLP neural network is used to distinguish the onsets in the series of impulses.

The system was tested on both synthesized and real piano recordings. The system correctly found over 98.5% of all onsets and produced approximately 2% of spurious onsets. Missed notes were mostly due to notes played in fast sequences and spurious notes were generally the result of beating or other noise in the signal. For detecting repeat notes, the MLP neural network is used. Before transcription, the

tuning of the whole piano and the initial frequencies of adaptive oscillators is calculated. The SONIC system also calculates the length and loudness of each note.

The average number of correctly found notes in synthesized recordings is around 90%, with an average of 9% spurious notes. Most of the missed notes were either octave errors, repeated notes or notes in fast sequences. Again, the spurious notes were generally due to a combination of octave errors and repeat notes. Other spurious notes were due to missed or spurious onsets and high polyphony in the music. Results on real recordings were not as good due to several factors, such as reverberation and noise in real recordings and inclusion of beating and sympathetic resonance in real music. It was also seen in an analysis on a real recording of Bach's English Suits showed that SONIC also had difficulty with low pitched notes and notes in arpeggios and trills.

3.2.6 Multiple Pitch Estimation

In [10] a method for the estimation of the multiple pitches of concurrent musical sounds is discussed. In [10], sung vowels and 26 musical instruments were used to compile the data, on which multipitch estimation was performed. This study focused on the robustness of the pitch estimation with other sounds and noise present by using an iterative estimation and separation procedure. The algorithm consists of two main parts, pitch estimation and spectrum detection. For the pitch estimation, the pitch of the most prominent sound is extracted from the interference of other harmonic and noisy sounds. The spectrum of the detected sound is then estimated and subtracted from the mixture, and these steps are then repeated for the rest of the signal.

The pitch estimation algorithm calculates the independent pitch estimates at separate frequency bands, and then combines the results to get an overall estimate. This allowed for the possibility to make robust pitch detections in polyphonic signals. In tests against human subjects, only the most skilled subjects of the experimental group were able to outperform the system. The system works reliably, but the accuracy degrades with increasing polyphony and when several harmonics coincide.

3.2.7 Chord Estimation

In [9] chord estimation, by the extraction of frequencies which correspond to chords, is discussed. A constant-Q spectral analysis approach is used to extract the fundamental frequencies of 43 of the 57 possible categories of chords, in Western Polyphonic Music. The remaining chord categories have an ambiguity relating to frequency resolution. A software solution is then used for the chords which fall into this ambiguous category.

Although the Constant-Q transform is a viable approach for monophonic music, the Constant-Q spectrum is not guaranteed to contain peaks that correspond to fundamental frequencies, so a signal component at f1, which gives rise to a spectral peak at f1, provided there is no other component at f2, can be used to enhance the result. By using this method, difficulties would arise when resolving 14 of the 57 chords in Western Polyphonic Music, as it is not guaranteed that a spectral peak will be found at every fundamental frequency. The approach taken in [9] first searches for the resolvable fundamental frequencies. Then the constraints of possible combinations of fundamental frequencies, in Western Polyphonic Music, are used to look for previously un-resolvable fundamental frequencies.

Tests were carried out using valid 2, 3 and 4 note chords that a violin quartet can play, each lasting 0.5 seconds, in the frequency range of G3 (196Hz) to B8 (3,951 Hz). This software approach was used to successfully validate the performance of the approach implemented in [9] although, due to the Constant-Q implementation used in [9], fundamental frequencies below 200 Hz resulted in low resolution and an inability to resolve the fundamental frequencies in this spectral region.

3.2.8 Parametric Representation of Musical Signals

The focus of [8] is on parametric representation of musical signals, so that their high level of structure can be exploited. Bayes theorem transforms prior information in the light of available data. The Bayesian framework allows the model in [8] to be hierarchical, use many kinds of priori information and also to provide a probabilistic method of model selection.

Polyphonic pitch estimation is considered as a parameter estimation problem for a hierarchical model composed of a sum of a number of notes. Prior probability densities are used on all the parameters, which are used to represent any prior knowledge about the problem. Priors might be based on results from previous frames of data, which can then be used to represent constraints or possible parameter values. If no valuable prior information is known, the priors used can be diluted with respect to the overall likelihood. This model provides good results on monophonic music, but in the case of polyphonic music, particularly pianos, string instruments, there are high noise components which make it difficult to estimate notes particularly with increasing polyphony.

To overcome this problem, a structure is imposed, where a signal of significance has parameters which vary slowly compared to the frame length used, approximately 25ms. To increase robustness, several frames are analysed at once. In a block of successive frames, parameters are imposed to describe the variation of the frequency parameters, such as assuming that the frequency is constant over the block of frames. Using this method, notes can be clearly distinguished and significant note events tend to have duration of more than 10 frames. Around the frames 100-120, errors do occur due to the occurrence of a perfect fifth.

3.3 Overview

Transcription systems described to date have had limited success. In [4] and [5] it was found that with increasing polyphony the number of spurious notes also increased. Repeat notes where a note was played in succession several times, can also affect results [5], while in [9] it was found that detecting notes with a fundamental frequency less than 200Hz was difficult to achieve accurately. Another major factor in causing errors is the detection of notes that are an octave apart. As the note an octave above is, by definition, a doubling of frequency, it also corresponds to the second harmonic of the lower note. Octave errors are common throughout the automatic transcription field [4], [5], [7], [12], [13].

For a system to transcribe polyphonic music, it has to be able to handle several notes being played simultaneously. To do this the system needs to be able to detect the fundamental frequencies of notes that are played concurrently. The system also needs to be able to deal with the harmonics of all the fundamental frequencies

and this includes handling the octave problem. For Automatic Music Transcription a system is needed that can consistently and accurately solve these problems

Chapter 4

Implementation of Automatic Music Transcription Techniques to Monophonic and Polyphonic Music

In monophonic music there is a single line of melody. There are no chords and hence no overlapping of notes so transcription of monophonic music is a less complex task than transcription of polyphonic music. For monophonic music, the objective is to identify the fundamental frequencies or pitches of the notes and their onset and offset times. Transcription of monophonic music can be solved by frequency tracking, which is essentially a solved problem [58] [59].

This chapter describes the background research and techniques used in this research. Initially background research into monophonic music was undertaken followed by signal processing techniques, some of which are used in the transcription system. After the background research, polyphonic music is discussed along with the onset detection system. This research forms the basis of the project along with the masking system, which is described in detail in Chapter 5.

4.1 Fundamental Frequency Detection in Monophonic Music

In this section, a monophonic note detection system is described. This monophonic note detection system was implemented as the first stage in the project, to investigate the nature of notes in monophonic music and so a foundation of music knowledge could be gained with monophonic music before moving onto the more complex polyphonic music. This initial monophonic note detection implementation also helped to develop signal processing techniques.

When working with monophonic music, overlapping windows of equal size, 2048 samples, (approximately 171 ms at 12 kHz sampling rate), were used to divide up the music. A Fast Fourier Transform was performed on each of the windows and returned the frequency domain information for each window. Only one note could occur in each window, as the music is monophonic and the duration of the window is much smaller than the duration of any given note. If the window extended over a section, which included the end of one note and the beginning of another note, the note with the lower fundamental frequency would be returned by the detection system. This had a negligible effect on the process as a correct note was still being detected, but the exact timings of the offset and the next onset were missed. As the windows were small in size the offset of the first note and the onset of the second note were incorrect by only fractions of a second. In the frequency domain, the fundamental frequency detected in each window was established by identifying the most prominent peak in the frequency domain with the lowest frequency. Generally the fundamental frequency had the largest peak but this was not always the case. Harmonics were then used to verify that the fundamental frequency was correct by

checking that the frequencies of the harmonics were integer multiples of the fundamental frequency.

After calculating the fundamental frequency detected in each of the windows, next the onset and offsets of the notes were determined using a simple heuristic procedure as described below. Each window had a fundamental frequency associated with it as well as the amplitude of the fundamental frequency peak. A note in music can vary greatly in length in time, so a note, after being divided up, could contain a varying number of sampling windows, as seen in Figures 4.1 and 4.2.

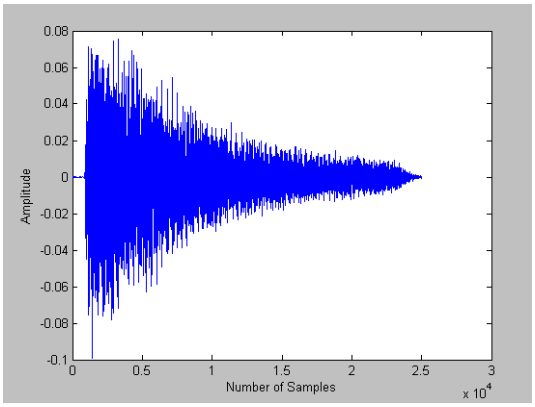


Figure 4.1: A note containing approximately 25,000 samples (≈ 2.08 seconds at 12 kHz sampling rate)

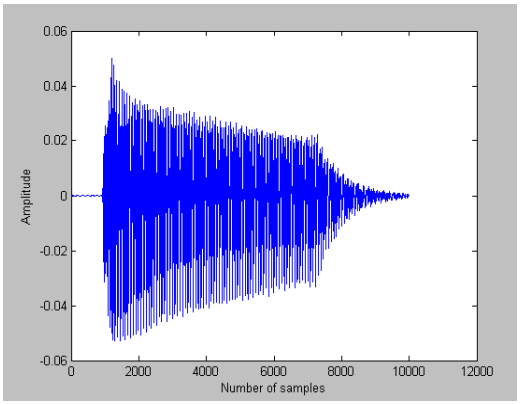


Figure 4.2: A note containing approximately 10,000 samples (≈ 0.83 seconds at 12 kHz sampling rate)

Figure 4.1 contains approximately 25,000 samples, (2.08 seconds), when broken up into windows of 2048, this results in Figure 4.1 containing approximately 12 windows. Figure 4.2 is a note shorter in duration then Figure 4.1, containing approximately 10,000 samples, (0.83 seconds). This creates approximately 5 windows, each of 2048 samples. This results in a string of fundamental frequencies, one from each of the windows. If for example Figure 4.1 was note C3, which has a

fundamental frequency of 131 Hz, and Figure 4.2 was note D3, 147 Hz, the output from the windows would be:

C3 C3 C3 C3 C3 C3 C3 C3 C3 C3 C3 C3 D3 D3 D3 D3 D3

Above is the output from the 17 windows, 12 containing C3 and 5 containing D3. In this example it can clearly be seen that C3 is played then D3. Clearly, this simple approach cannot distinguish between two identical notes played in succession. However, based on the amplitude envelope of the fundamental frequencies, the onset of a new note can be detected. When a note is played, the amplitude envelope of the fundamental frequency initially rises sharply and then declines slowly, as seen in Figure 4.3.

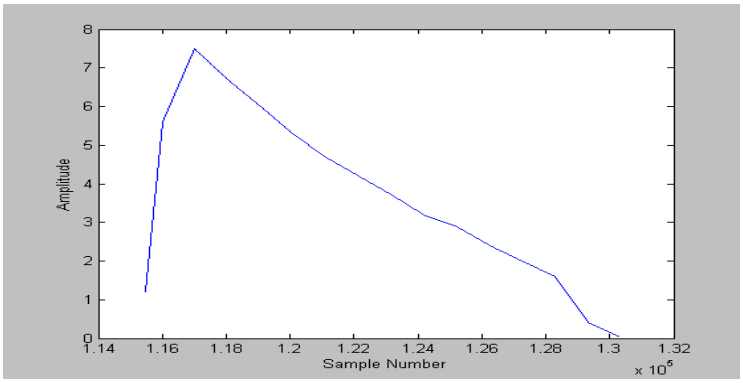


Figure 4.3: Plot of the amplitude of the fundamental frequency peaks of each window, during a note.

A new note onset is detected based on the amplitude of the fundamental frequency peak of each window and on whether the next window has detected the same fundamental frequency. This frequency tracking system, although very simple, can be used to transcribe monophonic music, although a more accurate onset detection system would improve the transcription process.

4.1.1 Signal Processing Implementation

Music files were recorded at the sampling frequency of 12000 Hz. This sampling rate allows for efficient processing of the information without compromising the quality of the music too much. These music files are windowed and the frequency domain representation is obtained using an FFT to allow identification of what frequencies are in each window and therefore what notes were played.

The DFT is given by:

For N samples, $x_0, x_1, ..., x_{N-1}$; of the signal:

$$X_n = \sum_{k=0}^{N-1} x_k e^{\frac{-j2\pi kn}{N}} \qquad n = 0,1...,N-1 \tag{4.1}$$

where the resulting coefficients X_n are complex.

The Fast Fourier Transform [60], FFT, is an efficient way of evaluating a DFT, which is much faster for large values of N, where N is a power of 2, where N is the number of samples in the sequence.

Initially, equally sized, overlapping windows of 2048 samples, (≈171 ms at 12 kHz sampling rate), were used, as described in Section 4.1. An FFT was performed on these windows to obtain the frequency domain information of each individual window. As notes in music have varying durations, a window of 2048 samples may account for a quarter of the note. If a note with a longer duration is played, 2048

samples may only account for a tenth of the note. When detecting the fundamental frequency of a window, the varying length of a note caused the fundamental frequencies in the individual windows to have slightly varying values. When a note is played, its amplitude rises sharply and then declines slowly as shown in Figure 4.3. If a note has a long duration, the frequency peaks in the windows near the offset of the note may have very low amplitudes and could be affected by noise. Generally this did not affect the system, but in some cases with notes that had long durations, of over 2 seconds, the detection of the fundamental frequency would be incorrect by as much as a few hertz. In notes in the upper octaves, from C4 upwards, where there are at least 15 Hz between different notes, e.g. C4 versus C#4, there was no problem with a fundamental frequency being incorrect by 2 or 3 Hz as the detection system could still correctly identify the note. However, if this occurred with notes in the lower octaves where there may be only a few hertz in difference between different notes, e.g. C2 versus C#2, a note could be identified incorrectly because the fundamental frequency was incorrect by 2 or 3 Hz.

At low frequencies, below 200 Hz, [C1 to G3], the FFT results in poor frequency resolution making the correct detection of notes more difficult. The frequency resolution, binwidth, for the FFT is given by:

$$\text{Binwidth} = \frac{fs}{N} \tag{4.2}$$

where fs is the sampling frequency and N is the window length of 2048 samples.

For fs = 12000 Hz, binwidth ≈ 5.86 Hz
A binwidth of 5.86 Hz, corresponds to a windows length of 171 ms.

For fs = 24000 Hz, binwidth ≈ 11.72 Hz
A binwidth of 11.72 Hz, , corresponds to a windows length of 85 ms.

For fs = 6000 Hz, binwidth ≈ 2.93 Hz
A binwidth of 2.93 Hz, corresponds to a windows length of 340 ms.

Although a sampling frequency of 6 kHz gives the best resolution, it cannot be used, as it would result in a window length of 340 ms, which could be too long when analysing notes. The 340ms windows may not contain enough information to give a clear picture of what exactly is happening in the window and so could miss important information, therefore under-sampling. A sampling rate of 24 kHz could be used, but this may be a case of over-sampling the information resulting in a lot of redundant information and also slowing the system down due to the large number of samples that would need to be analysed. A sampling rate of 12 kHz gives a good balance between a window size of 171 ms, therefore not missing important information and also not having too many samples, which are not needed in the analysis. For this reason a sampling rate of 12 kHz was used during this research. The number of samples in a window is rounded down to the nearest multiple of 1024 samples for use in the FFT calculation. Due to recent increases in computing speed and memory, performing FFTs with such large numbers of data points does not present the computational obstacle it once did.

Furthermore the frequency domain for lower frequency notes contains more data compared to that of higher frequency notes as shown in figures 4.4 and 4.5. However, this includes noise, which makes it more difficult to extract the notes and the harmonics. Due to the methods implemented in this project this poorer frequency resolution for lower frequency notes is overcome, as described in the following chapters. Once the fundamental frequency has been established, the harmonics of the

rest of the note are removed, so only the fundamental frequency of a note needs to be established and not the harmonics.

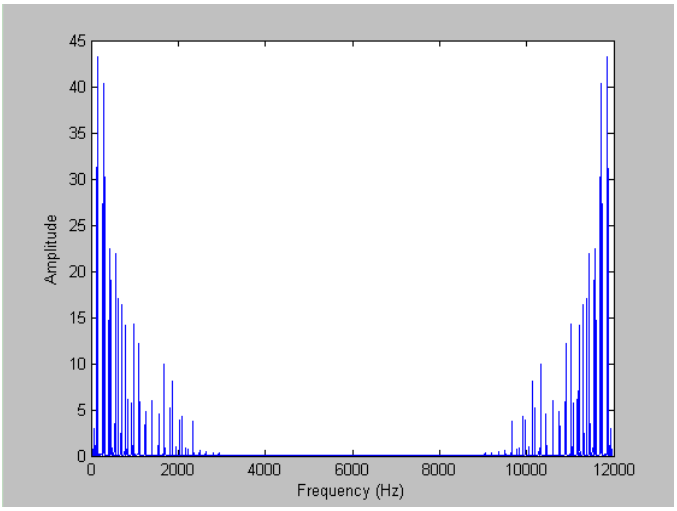


Figure 4.4: Magnitude of FFT for notes C#3 (139 HZ) & D#3 (156 HZ) played simultaneously.

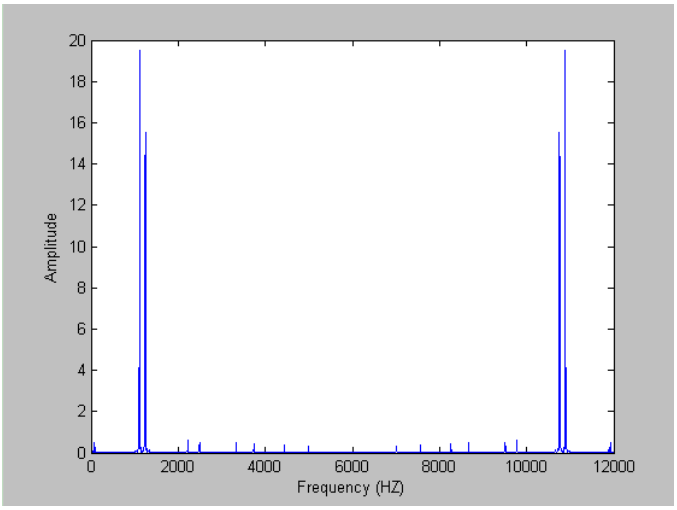


Figure 4.5: Magnitude of FFT for notes C#6 (1109 HZ) & D#6 (1245 HZ) played simultaneously.

While this implementation of using fixed size sampling windows was acceptable for monophonic music, due to the complexity of polyphonic music, this method did not work, as the fundamental frequency was often detected incorrectly. This problem was overcome with the introduction of the onset detection system [51],

see Section 4.2. The onset detection system gives the onset and offset of a note. As the position of the whole note was known, an FFT could be performed on the whole note and not just a section of the note. The resulting information from the frequency domain is more accurate at detecting the fundamental frequency of a note and therefore solved the problem of incorrectly detecting the fundamental frequency errors that occurred with fixed sized sampling windows

4.1.2 Phase Detection

To explore the behaviour of the phase component of the music signal, the FFT of the signal was first calculated. The phase is then calculated and unwrapped using standard functions in Matlab [61]. In Figure 4.6 the first plot is the magnitude of the FFT for notes C#3 & D#3. The frequency peaks can clearly be seen for both notes and also the harmonics associated with the notes. The second plot is of the phase of the signal, with the third plot showing the unwrapped phase of the signal. Small plateaus occur in the phase at the frequencies of the fundamentals, 139 Hz and 156 Hz respectively. Plateaus also occur at the harmonics of the notes. There is no discernable difference between a plateau that occurs at the fundamental frequency of a note and a plateau that occurs at a harmonic of the note. Therefore this technique does not help to distinguish between a fundamental frequency and a harmonic.

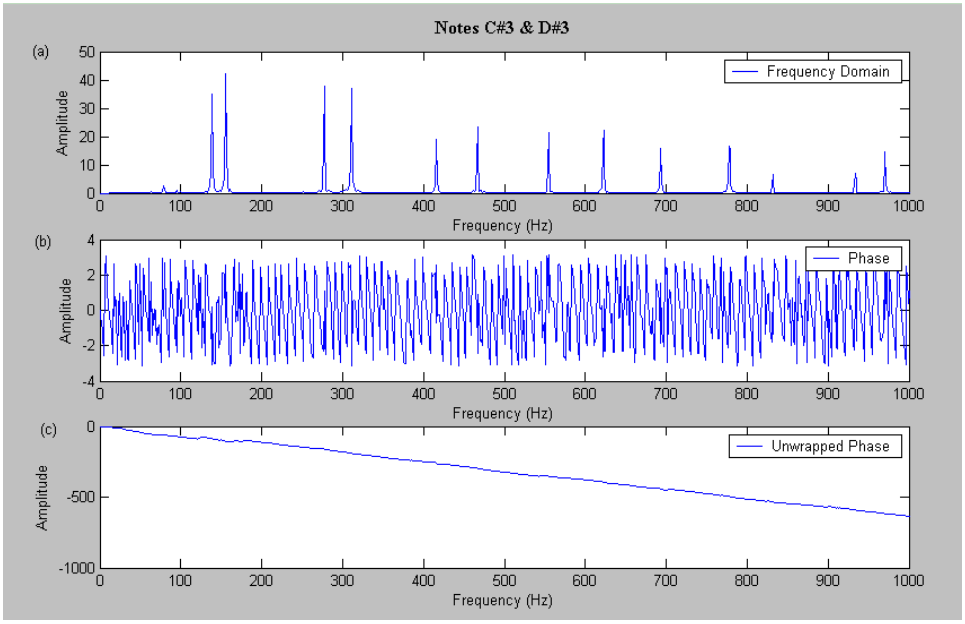


Figure 4.6: Plots of C#3 & D#3 in (a) Frequency Domain, (FFT magnitude plot), (b) Phase and (c) Unwrapped Phase.

Figure 4.7 shows the first difference of the unwrapped phase. There are clear negative peaks at the harmonic frequencies as well as at the fundamental frequencies. As with the unwrapped phase, there is no discernable difference between the fundamental frequencies and their harmonics. Also at frequencies below approximately 100 Hz, noise interference is quite prominent and makes the detection of fundamental frequency peaks difficult. Again this technique does not show any difference between a fundamental frequency and a harmonic and also cannot give a clear indication of whether the phases of overlapping frequency components are additive in nature. However the plateaus in the phase do indicate that there may be a way of detecting the difference between a fundamental frequency and a harmonic using the phase of the frequencies. This could be the subject of further work.

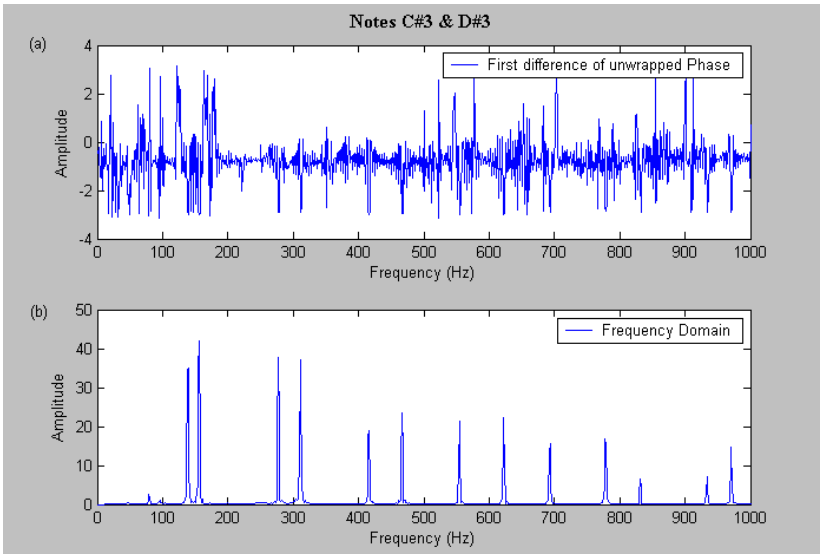


Figure 4.7: Plots of C#3 & D#3 (a) first difference of the unwrapped phase and (b) in the frequency domain, (FFT magnitude plot).

The peaks in the frequency domain, Figure 4.7 (b), that occur at the harmonics may be of use if the problem of the missing fundamental [34] occurred. The missing fundamental is the phenomenon where the fundamental frequency is missing, but can still be heard by a listener. For example, if a piano has a fundamental frequency of 100 Hz, its harmonics will be 200, 300, 400 and so on. In some cases, where say a telephone or a stereo speaker does not produce low frequencies, the fundamental frequency of 100 Hz will be missing. But to a listener, the pitch corresponding to a fundamental frequency of 100 Hz will still be heard. It is widely accepted that the brain, using the audible harmonics, calculates the missing fundamental [62]. A missing fundamental could be found by working back through the harmonics. Taking the frequency gaps between the harmonics into account, this analysis would result in the fundamental frequency.

4.2 Onset Detection System

The note onset detection system used in this project was designed to detect music onsets based on the energy level changing in a piece of music [51]. By determining the Note Average Energy (NAE), the method is insensitive to both the dynamic range of the overall energy level of the music and also to whether the song is monophonic or polyphonic, which is very important to the success of the masking system, see Section 5.2, as a piece of music may contain both monophonic and polyphonic sections.

4.2.1 Onset Detection Theory

The main approach of this onset detection system is to determine the noticeable change in the sound energy. Firstly the power envelope needs to be extracted. Assume s(t) is the sound wave signal and h(t) is the smoothing window. The instant power envelope is then the convolution of the instantaneous power, $s^2(t)$, and the smoothing window, that is;

$$p(t) = \int_0^t s^2(\tau)h(t-\tau)d\tau \tag{4.3}$$

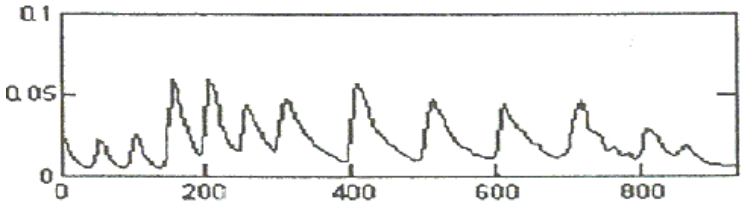


Figure 4.8: Power Envelope p(t) of a musical signal

Figure 4.8 shows the power envelope of a musical signal, the energy is calculated using Equation 4.4 and is shown in Figure 4.9. Energy is the integration of power with respect to time (t):

$$e(t) = \int_0^t p(t)dt \tag{4.4}$$

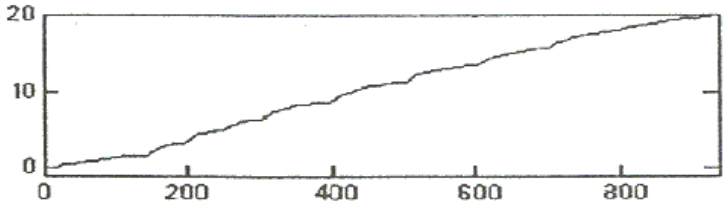


Figure 4.9: Energy e(t) plot of a musical signal

From Figure 4.9, small bends corresponding to each note can be seen in the energy plot. This bending is not a clear enough indication of note onsets because e(t) is an increasing function, so the averaged energy, a(t), is calculated using Equation 4.5:

$$a(t) = \frac{1}{t}\int_0^t p(t)dt \tag{4.5}$$

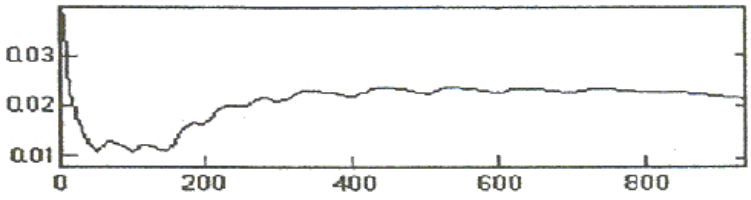


Figure 4.10: Average Energy a(t) plot of a musical signal

The average energy (Fig. 4.10) shows dips at each onset but because as t -> infinity it tends towards a constant, these dips or fluctuations eventually become too small to detect. Thus, we define the 'NAE' which is the average energy fluctuation supposing that the duration of the note is known. The NAE declines when no new note follows, therefore when a new note is played, the NAE rises as additional energy is applied

with the playing of a new note. Based on this the NAE is used to measure the noticeable energy level changes as seen below in Figure 4.11.

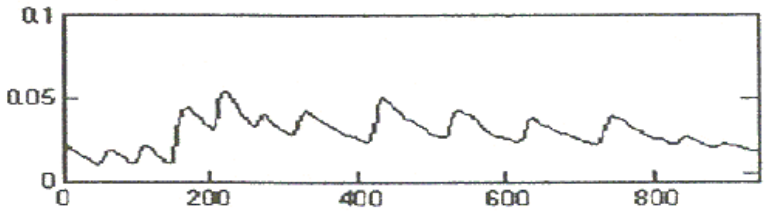


Figure 4.11: Sample of Note Average Energy, NAE (t)

Suppose the onset instant, t_n, is known for every note where n = 1, 2, … N, and N is the number of notes. NAE is defined as:

$$NAE(t) = \frac{1}{t - t_n} \int_{t_N}^{t} p(t)dt \qquad (t_n < t < t_{n+1}) \tag{4.6}$$

When $t = t_n$, $NAE(t_n) = p(t_n)$. From Figure 4.11, NAE (t) has only one increase and decrease for each note. So onsets can be determined simply by checking the local minima. Since $\{ t_n ; n = 1, 2, \ldots N \}$ is the set of onsets to be determined, NAE (t) cannot be calculated directly. This does not mean that that the position of an onset cannot be determined. Let n = 0 and $t_n = 0$, calculate NAE (t) in a range of t with a limit open to the right. If the following relation;

$$NAE(t - T) > NAE(t) < NAE(t + T) \tag{4.7}$$

is met, set n = n + 1 and $t_n = t + T$, then move on to the next detection also in the range of t with a limit open to the right. Doing this, there is no threshold problem to contend with.

4.3 Application of Automatic Music Transcription Techniques to Polyphonic Music

This section describes the initial steps taken in extending the techniques applied so far to monophonic music transcription to polyphonic music. An important aspect of polyphonic music, as already mentioned in Chapter 2.2, is the overlapping of fundamental frequencies with harmonics of different notes. If for example note C4 was played in isolation, the fundamental frequencies of notes C5 and G5 would be detected, although these notes were not played. The frequencies detected are the second and third harmonics of note C4 respectively, which happen to have the same frequency value as the fundamental frequency of notes C5 and G5. One of the main problems in music transcription is to establish whether a detected frequency peak is a fundamental frequency of a note or whether it is a harmonic of another note. This problem does not occur in monophonic music as it is known that only one note is played at a time. Polyphonic music can have several notes played together, so there could be many fundamental frequencies in a frequency domain plot of a section of polyphonic music.

If, for example, note C4 was played, in the frequency domain the frequency peak at the fundamental frequency of C4, (261 Hz), would generally be the most prominent peak and the peaks of the harmonics would tend to decrease in amplitude the higher the frequency of the harmonic. However this alone may not be enough to decide whether just one note was played and that the detected peak represents a fundamental frequency, or whether a detected frequency peak represents a harmonic. If the note was played softly, the fundamental frequency peak might be low in amplitude and with some notes, C notes for example, the second harmonic may have

a comparable amplitude to the fundamental frequency. This can make it difficult to decipher whether C4 was played and a peak at 523 Hz was the second harmonic of C4, or whether notes C4 and C5 (which has a fundamental frequency of 523 Hz), were both played together. Figure 4.12 and 4.13 both show note C4 played in isolation, but the peaks at 523 Hz vary greatly in amplitude in comparison to the peaks at 262 Hz, the fundamental frequency of C4.

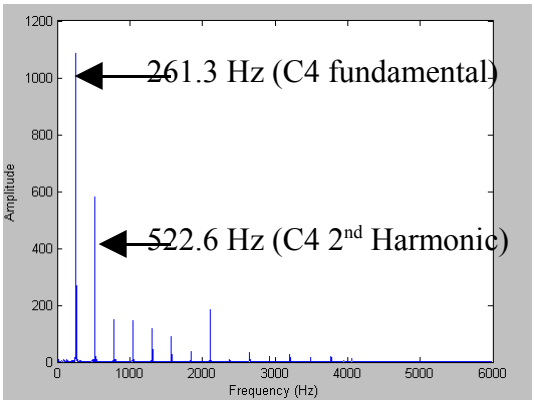


Figure 4.12: Note C4 in isolation. The fundamental of C4 and 2nd harmonic have amplitudes of very different heights.

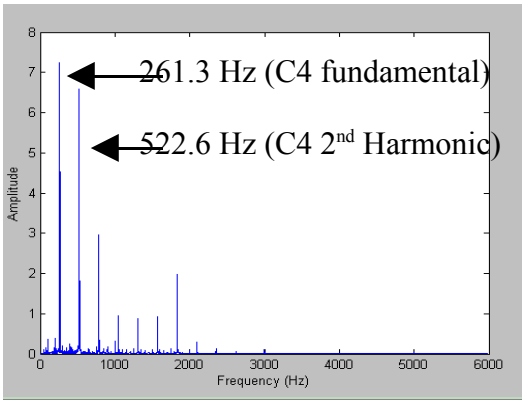


Figure 4.13: Note C4 in isolation. The fundamental of C4 and 2nd harmonic have amplitudes of comparable heights.

From Figures 4.12 and 4.13, the harmonics of note C4 can be seen to vary greatly in amplitude in comparison with the amplitude of the fundamental frequency in different examples. This is a simple case where only one note was played in isolation. If there were more notes played, the frequency plot would have more frequency peaks and extracting the notes that were played would be a much more difficult task. Figure 4.14 shows a frequency plot of notes C3, E3, G3 and C4 played simultaneously. C3, E3 and G3 make up chord C Major.

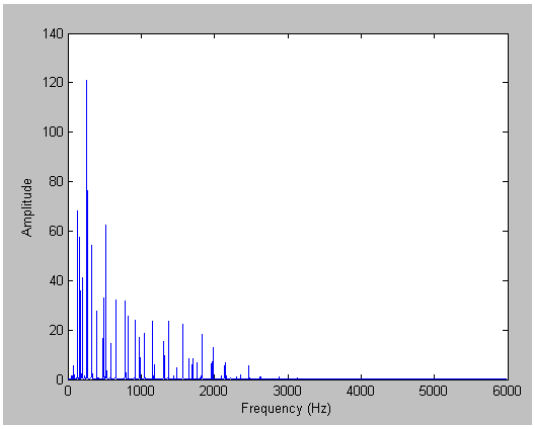


Figure 4.14: Frequency domain plot of notes

C3, E3, G3 and C4 played simultaneously.

To solve this problem several techniques were explored, as outlined in this chapter, but the most promising technique and the one reported in this work is a masking technique, where a template for every note was created. Once a note was detected in the frequency domain, the template for that note could be used to extract that note from the frequency domain and leave the remaining notes intact for the detection system to run again to try and locate any other notes that may have been played. This masking technique is described in more detail in Chapter 5.

4.3.1 The Octave Problem

A common problem in automatic music transcription is the octave problem [4], [5], [7], [12], [13]. This is where two notes, which are one octave apart, are played simultaneously. The problem occurs because the note with the higher fundamental frequency is also the second harmonic of the note with the lower fundamental frequency. Take for example C3 (131 Hz) and C4 (262 Hz), which are one octave apart. The second harmonic of C3 occurs at 262 Hz but the fundamental frequency of note C4 also occurs at 262 Hz. This causes confusion as to whether only

C3 was played and its second harmonic at 262 Hz was detected, or whether C3 and C4 were both played.

For a system to transcribe polyphonic music, it has to be able to handle several notes being played simultaneously. To do this the system needs to be able to detect the fundamental frequencies of notes that are played concurrently. The system also needs to be able to deal with the harmonics of all the fundamental frequencies and this includes handling the octave problem. For Automatic Music Transcription a system is needed that can consistently and accurately solve these problems.

4.3.2 Place Pitch Theory in Music Transcription

Experiments carried out by Hungarian scientist Georg von Békésy in the mid twentieth century demonstrated a simple model of how fundamental frequency detection may be carried out in the ear. Different frequencies occurring in a note stimulate different parts of the basilar membrane and this formed the basis for Place Pitch Theory [63]. When applied to pitch detection by J. F. Schouten, he postulated that the fundamental frequency is the highest common factor of the frequencies present in a sound. For practical use in pitch detection this may be done by dividing the frequencies present in a sound by successive integers and finding the highest common factors. A table structure can be used for this.

Table 4.1 shows an example for the fundamental frequency of 100 Hz. The frequency values for the left hand column in Place Pitch Theory are obtained from the frequency peaks from one window. The fundamental frequency of 100 Hz is listed in the first column and the expected next 9 harmonics of the note are also in the first column on the left hand side of the table.

f_i = {100, 200, 300,…1000} by the integers from 2 to 10.

$$f_i = nf_o \qquad n = 1,2,3\ldots n \tag{4.8}$$

where f_o is the fundamental frequency

n x f0	/ 2 (HZ)	/ 3 (HZ)	/ 4 (HZ)	/ 5 (HZ)	/ 6 (HZ)	/ 7 (HZ)	/ 8 (HZ)	/ 9 (HZ)	/ 10 (HZ)
1	2	3	4	5	6	7	8	9	10
100	50	33.33	25	20	16.67	14.29	12.5	11.11	10
200	100	66.67	50	40	33.33	28.57	25	22.22	20
300	150	100	75	60	50	42.86	37.5	33.33	30
400	200	133.33	100	80	66.67	57.14	50	44.44	40
500	250	166.67	125	100	83.33	71.43	62.5	55.56	50
600	300	200	150	120	100	85.71	75	66.67	60
700	350	233.33	175	140	116.67	100	87.5	77.78	70
800	400	266.67	200	160	133.33	114.29	100	88.89	80
900	450	300	225	180	150	128.57	112.5	100	90
1000	500	333.33	250	200	166.67	142.86	125	111.11	100

Table 4.1: Place Pitch Theory example with a fundamental frequency of 100 Hz

The subsequent columns display the result of dividing each frequency present in the spectrum, shown in the first column, by successive integer values, shown in the second row. The fundamental frequency of 100 Hz, circled in red, occurs in every column of the table. The second harmonic at 200 Hz, circled in blue, occurs in the first 5 columns and if the table were enlarged it would continue to occur in each column. However, Place Pitch Theory uses a square table, i.e. equal number of columns and rows, and so all the harmonics will not appear in every column as seen in Table 4.1. For Place Pitch Theory, the fundamental frequency should appear in every column. As the value of 200 Hz did not appear in every column, this suggests that it is a harmonic value and not a fundamental frequency. The 100 Hz value did appear in every column, suggesting that this value is a fundamental frequency.

4.3.3 Application of Place Pitch Theory to Polyphonic Music

Polyphonic music can contain more then one fundamental frequency and so initial experiments were carried out to see if Place Pitch Theory could be used to discover fundamental frequencies in polyphonic music.

Place Pitch Theory was not designed for use on Polyphonic music, so it is expected that frequencies will not behave as they did for monophonic music. For monophonic music, as seen in Table 4.1, the fundamental frequency of 100 Hz occurs in every column suggesting that this value is a fundamental frequency. The next frequency value in the left hand column is 200 Hz and this value does not appear in every column, suggesting that this value is a harmonic of 100 Hz. We now compare a polyphonic example in Table 4.2, which contains two fundamental frequencies, 131 Hz (Note C3) and 247 Hz (Note B3).

C3 (131Hz)
B3 (247 Hz)
262 Hz

n x f0	/ 2 (HZ)	/ 3 (HZ)	/ 4 (HZ)	/ 5 (HZ)	/ 6 (HZ)	/ 7 (HZ)	/ 8 (HZ)	/ 9 (HZ)	/ 10 (HZ)	/ 11 (HZ)	/ 12 (HZ)
131	65	44	33	26	22	19	16	15	13	12	11
247	123	82	62	49	41	35	31	27	25	22	21
262	131	87	65	52	44	37	33	29	26	24	22
393	196	131	98	79	65	56	49	44	39	36	33
494	247	165	124	99	82	71	62	55	49	45	41
523	262	174	131	105	87	75	65	58	52	48	44
654	327	218	164	131	109	93	82	73	65	59	55
741	370	247	185	148	123	106	93	82	74	67	62
785	393	262	196	157	131	112	98	87	79	71	65
916	458	305	229	183	153	131	115	102	92	83	76
988	494	329	247	198	165	141	123	110	99	90	82
1047	523	349	262	209	174	150	131	116	105	95	87

Table 4.2: Place Pitch example with 2 fundamental frequencies, 131 Hz and 247 Hz

The third value in the left hand column, 262 Hz circled in blue, is the second harmonic of the first fundamental frequency, 131 Hz circled in red. Because there is a value, 247 Hz circled in green, in between the fundamental frequency and the second harmonic, the fundamental frequency of 131 Hz will not appear in every column as seen in Table 4.2. If Table 4.2 was extended vertically, then the 131 Hz value would

appear in every column and if extended further, the 247 Hz value would also appear in every column. Place Pitch Theory is based on a square table, so if the table was extended vertically, then the table would also need to be extended horizontally, and this would again result in the 131 Hz and 247 Hz values not appearing in every column of the table, similar to that shown in Table 4.2. If a slight variation of the table was used, where the table could be extended vertically but not horizontally, the 131 Hz and 147 Hz values would eventually appear in every column, but the table would be very large and if there were more than two fundamental frequencies in the table, to include every fundamental in every column, would mean having an exceptionally large table. This would then lead to harmonics of the lower frequency notes appearing in every column, and therefore it would be difficult to judge whether a value appearing in every column was a fundamental frequency or a harmonic.

Place Pitch theory relies on the harmonics being divided by consecutive integers. This gives a square table containing the fundamental frequency in every column. If there are two fundamental frequencies, and the table is square, the two fundamental frequencies will not occur in every column because when the second fundamental frequency is divided by consecutive integers, the values will not be compatible with the values from the first fundamental frequency. This will lead to a break down in the system and cause the fundamental frequencies not to appear in every column of the table. In this example, Place Pitch Theory fails to detect either of the fundamental frequencies.

To use Place Pitch Theory with polyphonic music means that a fundamental frequency would not necessarily occur in every column of the table. The most frequently occurring value, which is also high up on the left hand side of the table, is

taken as a fundamental frequency. A decision on any subsequent value is made based on where the value appears in the table and on how frequently the value occurs in comparison to the lowest fundamental frequency found.

4.3.4 The Octave Problem in Place Pitch Theory

The Octave problem, see Section 4.3.1, can result in some of the harmonics of a note becoming prominent in the Place Pitch Theory table so that they are detected by the system as a separate note and not as a harmonic of a note. Also if two notes, which are one octave apart, are played, the note in the higher octave can look like a harmonic of the note in the lower octave. The octave problem can be seen in the example below, in Table 4.3.

C3 (131Hz)
B3 (247 Hz)
C4 (262 Hz)

n x f0	/ 2 (HZ)	/ 3 (HZ)	/ 4 (HZ)	/ 5 (HZ)	/ 6 (HZ)	/ 7 (HZ)	/ 8 (HZ)	/ 9 (HZ)	/ 10 (HZ)	/ 11 (HZ)	/ 12 (HZ)
131	65	44	33	26	22	19	16	15	13	12	11
247	123	82	62	49	41	35	31	27	25	22	21
262	131	87	65	52	44	37	33	29	26	24	22
393	196	131	98	79	65	56	49	44	39	36	33
494	247	165	124	99	82	71	62	55	49	45	41
523	262	174	131	105	87	75	65	58	52	48	44
654	327	218	164	131	109	93	82	73	65	59	55
741	370	247	185	148	123	106	93	82	74	67	62
785	393	262	196	157	131	112	98	87	79	71	65
916	458	305	229	183	153	131	115	102	92	83	76
988	494	329	247	198	165	141	123	110	99	90	82
1047	523	349	262	209	174	150	131	116	105	95	87

Table 4.3: Place Pitch example demonstrating the Octave problem

Note: The naming of the notes, scientific pitch notation, used here is that proposed to the Acoustical Society of America by Young (1939) [69].

The frequencies in the first column of the table are values obtained from an FFT analysis of two notes played simultaneously, C3 (131 Hz) and B3 (247 Hz). From the table it can clearly be seen that both C3 and B3, circled in red, appear in several columns, which would lead to their detection. Also appearing in several columns is 262 Hz, circled in blue, which is the frequency value for C4. Although C4 was not played, it is a harmonic of C3. Therefore there will, as is seen in Table 4.3, be

prominent values in the table, which are harmonics and not actual notes. In the case of Table 4.3, this would lead to the incorrect detection of C3, B3 and C4.

4.4 Note Detection in Polyphonic Music

The first sample of polyphonic music tested contained three simultaneous notes, a chord. From the resulting FFT of this sampled data, there were three dominant peaks in the frequency domain, with each peak corresponding to the fundamental frequency of one of the notes played. Based on this a function was created to detect the first six major peaks, (i.e. six notes played at a time), which could easily be increased to detect more notes if needed. Peaks were only detected if they were above a certain threshold. This threshold was based on a ratio to the largest peak in the window. If peaks were below 30% of the amplitude of the maximum peak, they were discarded. This helped to eliminate any noise peaks that might occur and distort the results.

While this method did detect the correct frequencies, the harmonics of the fundamental frequency of the notes played were also being detected. Some method of differentiation between actual notes and harmonics needed to be established. Another problem that remained was the problem encountered with the monophonic music, the problem of note onset and offset detection. Small sampling windows were again used to divide up the music sample. First tried was the method used in the monophonic music, to base the decision of note transition on the amplitude of the fundamental frequency peaks in a window and on what notes were detected in the next window. Due to the greater complexity of polyphonic music as opposed to monophonic music,

this system failed to accurately detect the transitions except on pieces where the notes were played unnaturally slowly, which clearly was not an acceptable solution.

A more effective approach was to use an onset detection system [51], see Section 4.2, to detect the onsets and offsets of the notes. Instead of analysing each individual sampling window, since now the onset and offset of a note could be established, the analysis was done on the whole note, from onset to offset. This method gives much clearer information on the overall note and reduces the problem of spurious notes being detected and causing errors in the output of the system.

Once the note was isolated using the onset detection system, the analysis began with an FFT on the whole note. From this analysis, a typical picture for each individual note could be seen occurring in the frequency domain. This picture or pattern could be seen in any example of that particular note played. From this, an extensive range of samples was taken for each individual note and models of each note were then created in the frequency domain, when the note was played in isolation and in combination with other notes. Mask creation is described in further detail in Section 5.3.

Once a certain fundamental frequency was detected, the model of the note with that particular fundamental frequency was applied to the sample music. The model of that note could then be subtracted from the frequency domain values by masking the values for the specific note detected. The system could then check to see if there were any other notes present, which would have been played at the same time, by trying to detect another fundamental frequency. If another fundamental frequency

was detected, then the system would find and apply the note model corresponding to that fundamental frequency and loop until all notes had been extracted.

Whereas there were some problems with this system in relation to the harmonics of a note sometimes not behaving as expected, results were very positive and this system of masking out detected notes was implemented and is described in further detail in Chapter 5.

Chapter 5

Note Masks

5.1 Introduction

In everyday life the human ear is exposed to a wide variety of sounds, with different frequencies and intensities. A lot of the time we are exposed to simultaneous sounds but we are able to focus specifically on certain sounds if needed. Take for example picking out and listening to a single conversation in a crowded and noisy room. The brain and human auditory system can focus on a specific conversation without perception suffering from interference from the other conversations in the room. If the intensity or loudness of the other conversations in the room increases, then this interference will decrease a listener's ability to focus on a single conversation. This is called the cocktail party effect [64]. In this research, a method is developed of concentrating on a particular note in a polyphonic section of music and focusing or "listening" to that note. Once that note has been identified, it is removed from the section and the system looks to find another note in the section of music. This continues until no more notes can be found in the section of music.

In the auditory field, masking is defined as where one louder sound masks an adjacent (in frequency) quieter sound so that the quieter sound cannot be heard [17]. In computer science, the term masking is used in the sense of a bit mask which may be used to conceal bits in a word to allow the retrieval of bits of interest [18]. In this research the term masking is used in both of these ways: it refers both to the use of a mask for blocking out all but a particular note, (fundamental frequency and its harmonics), from a set of frequencies in a window; and also to how the presence of overlapping harmonics from different notes in the same window leads to an apparent masking of frequencies by other more dominant frequencies.

5.2 Masks

In monophonic music it is straightforward to establish what notes were played in a note window of samples by simply applying an FFT to the note window. This is assuming that the monophonic music has first been separated into note windows which include one onset per window. This note window is found using the onset detection system [51], as described in Chapter 4.2, which finds the onset of notes in a piece of music. The offset is then defined to be the onset of the next note, so a note window is delimited by the onset and offset of a note. For monophonic music the lowest frequency peak in the window will thus be the fundamental frequency of the note that was present in that note window of samples. The other frequency peaks can be ignored, as they are harmonics of the fundamental frequency because, in general, only one note occurs in a note window in monophonic music..

With polyphonic music, more than one note can be present in a note window and this increases the difficulty of detecting which notes that are present. While the

lowest significant peak will probably and most commonly be the fundamental frequency of one note that is present, there may be other notes in the window, which will be more difficult to detect. There will be added complexity from the presence of the harmonics of the several fundamental frequencies present. Since it is not known a priori how many notes are present in each note window, all peaks must be analysed to see whether they are fundamental frequencies or harmonics.

Unfortunately, it is often difficult to distinguish between a fundamental frequency and a harmonic, because, for example, the second harmonic of C4 occurs at 523 Hz, which is the same as the fundamental frequency of note C5. So the only guide to establishing whether a peak is a fundamental frequency or not, is the amplitude. The phase of the frequencies was explored as an indicator, as described in Section 4.1.2, but no clear distinction could be found between the fundamental frequencies and harmonics. Phase may be of more use to discriminate between music played on different instruments, and further work might prove beneficial in fundamental frequency and harmonic detection in this case.

5.2.1 Amplitude of Harmonics in the Frequency Domain

In music signal identification and transcription, one of the most problematic issues is amplitude variability; notes can be played softly as well as loudly, the amplitudes varying along a continuum. Figure 5.1 shows two examples of note B2, (fundamental frequency and harmonics), being played with varying levels of intensity, with the fundamental frequency of the louder B2 note having an amplitude more than double that of the softer B2 fundamental frequency.

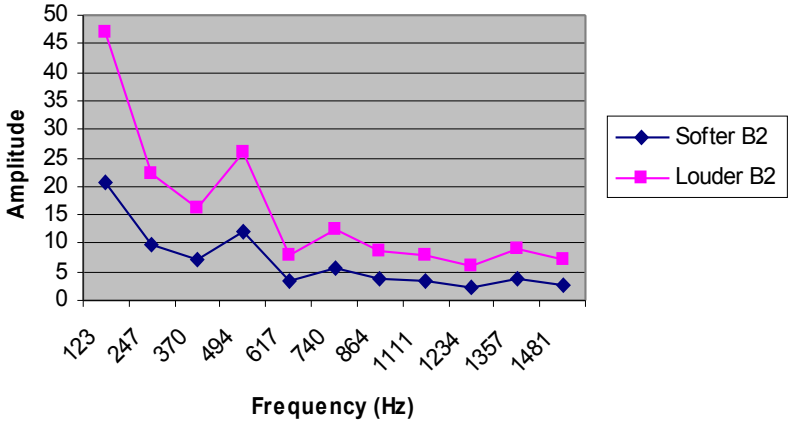


Figure 5.1: Note B2 played with varying intensity

To solve this problem, the amplitudes must be analysed in proportion to the values in the rest of the window. The key insight in the masking scheme is that frequencies present in a window that overlap from different sources (notes) are approximately additive in amplitude. If, for example, note D2 was played in isolation, its second harmonic would occur at approximately 147 Hz. 147 Hz is also the fundamental frequency of note D3. When note D2 and D3 are played together, the amplitude of the frequency peak at 147 Hz would be significantly increased due to the addition of the amplitude of the second harmonic of D2.

Table 5.1 shows the amplitude values when notes D2 and D3 are played together. Also shown is the D2 mask, and where there is no overlapping of frequencies between the two notes, the amplitudes at the D2 harmonics correspond to the D2 mask; but where the harmonics of the two notes overlap (in red) the amplitude at the overlapping frequency is much greater than the D2 mask, due to the additive affect of the D3 harmonics. The third column, labeled D2 Values, is calculated by dividing the amplitude at the corresponding frequency by the amplitude value at the

fundamental frequency of D2, 73 Hz. The fifth column shows the difference between the columns D2 Values and D2 Mask.

$$\frac{\text{Amplitude at frequency}}{\text{Amplitude of fundamental frequency}} = \text{D2 Value} \qquad (5.1)$$

D2 & D3 played simultaneously		Amplitude relative to D2 Mask Values	D2 Mask Amplitude Values	Difference
D2 Mask Frequency (Hz)	Amplitude			
73	64.737	100.00%	100.00%	0.00%
147	92.195	142.41%	66.07%	76.34%
220	24.109	37.24%	34.80%	2.44%
294	98.979	152.89%	60.61%	92.29%
367	13.282	20.52%	22.78%	-2.26%
441	36.781	56.82%	27.07%	29.75%
514	13.462	20.79%	20.28%	0.51%
587	29.449	45.49%	10.18%	35.32%
661	10.748	16.60%	18.71%	-2.11%
735	29.033	44.85%	16.04%	28.81%
808	14.708	22.72%	22.71%	0.01%
1028	27.161			
1175	13.458			
1763	11.596			

Table 5.1: Additive nature of overlapping frequencies

It is noted that some amplitudes in Table 5.1 are also decreased in this scenario and that the different harmonics affected increase by different amounts. Although every effort is made in this project to model notes and their harmonics accurately, due to the complex nature of music, notes may sometimes vary from the calculated note masks.

5.2.2 Note Masks

To create a note mask in the prototype system, 7 monophonic and 13 polyphonic examples of each note in the range C2 to B6 were recorded and analysed.

The note masks were developed using a digital keyboard (Technics KN800 PCM Keyboard [19]).

When analysing the monophonic samples, an FFT is applied to a note window of the entire duration of the note, as described in Section 4.4. The frequencies and amplitudes of the harmonics are collected. The number of harmonics collected, is dependent on the frequency range of the note. For notes in the lower octaves, such as C2, 12 harmonics are collected whereas for note C4, for example, only 7 harmonics were collected as in notes with higher fundamental frequencies, the amplitudes of the first few harmonics are much greater than the higher harmonics, whereas for notes with lower fundamental frequencies, the harmonics decline more slowly in amplitude relative to the fundamental frequency. Figures 5.2 and 5.3 show this difference between the note mask for C2 and C4.

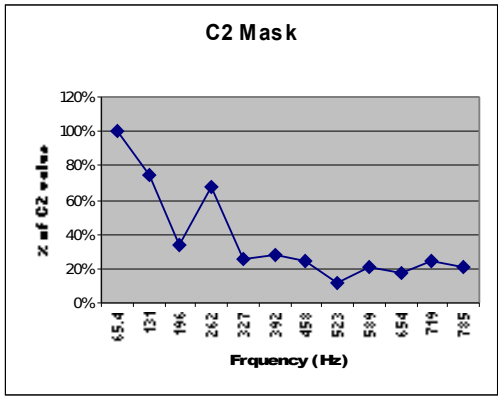


Figure 5.2: Note C2 Mask with 12 harmonics

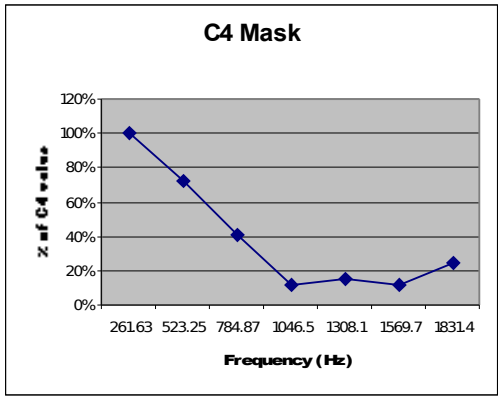


Figure 5.3: Note C4 Mask with 7 harmonics

The amplitudes of the harmonics are converted to percentages of the amplitude of the fundamental frequency. Taking all the examples of each note, average percentage amplitudes of each of the harmonics of the note in relation to the fundamental frequency are calculated.

$$\frac{\text{Sum of harmonic amplitudes (as \% of fundamental)}}{\text{Number of harmonics}} = \text{Average \% amplitude} \qquad (5.2)$$

When analysing the polyphonic samples, each note in the range is recorded with another note played at the same time. Since the theoretical frequency values of the harmonics for each note in the combination are known, the amplitude of the harmonics of each note can be identified. If the harmonics of the 2 notes coincide, then this value is discarded as the amplitude value will be the sum of a harmonic from each note and not a harmonic from a single note and there is no guaranteed formula to apportion them. However, in future, analysis of such examples could greatly increase the potential for developing more general and applicable notes masks with this approach. Again as with the monophonic samples, using Equation 5.2, the average amplitudes of the remaining harmonics in relation to the fundamental frequencies are calculated as percentages. An overall average is obtained by combining the data from the polyphonic samples with the monophonic samples.

$$\frac{\text{Average monophonic mask} + \text{Average polyphonic mask}}{2} = \text{Average Mask} \qquad (5.3)$$

5.3 Creating a Mask

To illustrate the process of note mask creation, consider that a mask is to be created for note D4. Note D4 would be played simultaneously with note C4, then with note E4 and with all other notes in that octave. Table 5.2 shows the first stage of the process for creating a mask for note D4. The frequency and amplitude of peaks detected in the frequency domain are recorded, where the harmonic values of D4 are in red, including the fundamental frequency. D4 has a fundamental frequency of approximately 294 Hz.

Notes Played	D4 & C4		D4 & E4		D4 & F4		D4 & G4		D4 & A4		D4 & B4	
Frequency (Hz) peaks detected	262	49.88	294	48.73	294	31.88	294	35.53	294	26.89	294	34.71
	294	47.90	330	52.60	349	49.97	393	35.93	440	30.86	494	24.44
	523	37.59	588	32.64	587	24.14	587	26.72	587	22.54	587	22.47
	588	33.39	881	18.21	881	14.00	881	15.54	**882**	**13.48**	881	13.85
	785	20.56	1469	7.94	2057	8.94	**1175**	**5.71**	1174	3.16	988	6.16
	881	18.32	1763	5.28			1468	5.47	1320	4.39	1469	4.90
	1047	5.90	2057	12.18			1763	4.38	1469	4.61	2057	6.41
	1175	5.38					2057	9.95	**1762**	**4.38**		
	1309	7.94							2056	8.02		
	1469	8.00										
	1570	5.89										
	1763	5.59										
	1832	13.79										
	2057	12.88										

Table 5.2: Frequency peaks used for creating a mask for note D4

From musical theory the frequencies of the harmonics of D4 and C4 are known. Where the harmonics of the two notes or the fundamental of one note and a harmonic of the other coincide, then the amplitude contributions present at the two frequencies will approximately add together. If this is the case, then that frequency peak is ignored as it is produced from two harmonics from two different notes. These overlapping values are blue in Table 5.2. A true representation of the harmonic amplitude needs to be established for a specific note, without interference from another note. At harmonics of D4, where overlapping does not occur, the amplitudes of the frequency peaks at the harmonic frequencies of D4 are recorded as a percentage of the amplitude value of the fundamental frequency of D4. Figure 5.4 graphs the harmonic values of D4 and C4, showing the harmonic values of D4 in red and the harmonic values of C4 in white. D4 and C4 have no overlapping harmonic values.

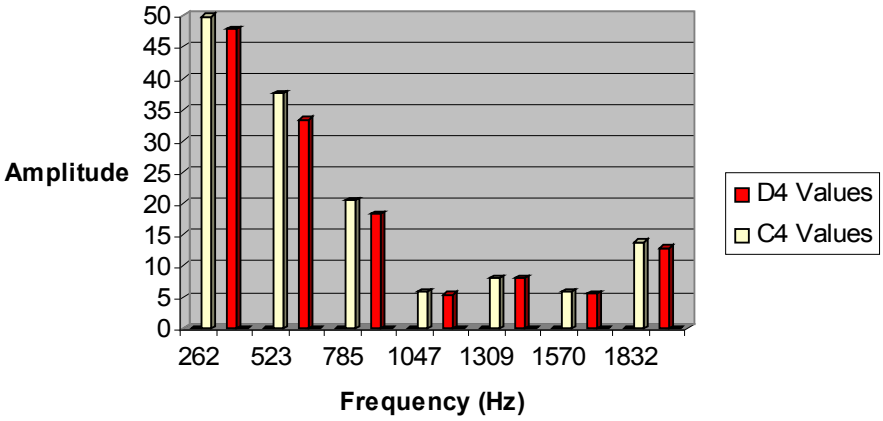


Figure 5.4: Graph of harmonic amplitude values of C4 and D4 when played simultaneously.

Figure 5.5 shows the harmonic values for notes D4 and A4, with D4 shown in red and A4 shown in white. D4 and A4 have two overlapping harmonics which are shown in blue. These amplitude values are the result of the approximate addition of the amplitudes of both notes at these frequencies.

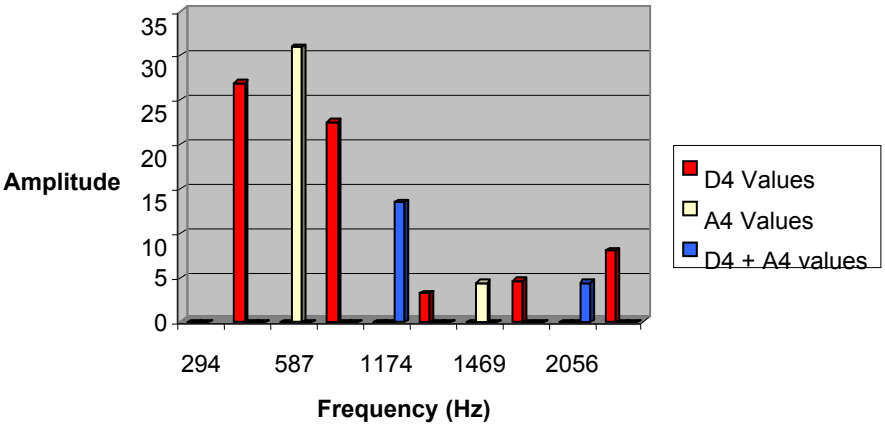


Figure 5.5: Graph of harmonic amplitude values of D4 and A4 when played simultaneously.

D4 is combined with all the notes in the octave so that a reliable mask may be developed. By using many different notes a more representative mask can be created which will include values for all the harmonics of D4. An average of the D4

harmonic values needs to be calculated using many examples to create a more accurate mask. Table 5.3 is then constructed with all the associated amplitude values of the harmonics of note D4, from Table 5.2. Any missing values in the table were either not detected or were values that overlapped with the harmonic of another note and were therefore not included.

Note D4

Frequency values (Hz)	D4 & C4	D4 & E4	D4 & F4	D4 & G4	D4 & A4	D4 & B4
294	47.90	48.73	31.88	35.53	26.89	34.71
587	33.39	32.64	24.14	26.72	22.54	22.47
881	18.32	18.21	14.00	15.54		13.85
1175	5.38				3.16	
1468	8.00	7.94		5.47	4.61	4.90
1762	5.59	5.28		4.38		
2056	12.88	12.18	8.94	9.95	8.02	6.41

Table 5.3: The amplitude values of the harmonics of note D4.

After this each amplitude value is normalized as a percentage of the fundamental frequency amplitude, as shown in Table 5.4.

As a % of D4 value

Frequency values (Hz)						
294	1	1	1	1	1	1
587	0.70	0.67	0.76	0.75	0.84	0.65
881	0.38	0.37	0.44	0.44		0.40
1175	0.11				0.12	
1468	0.17	0.16		0.15	0.17	0.14
1762	0.12	0.11		0.12		
2056	0.27	0.25	0.28	0.28	0.30	0.18

Table 5.4: Amplitude values of the harmonics of note D4
as a percentage of the fundamental frequency.

Next an average percentage value in relation to the fundamental frequency for each harmonic frequency is calculated. Table 5.5 shows the mask for note D4 and Figure 5.6 plots these values.

Frequency values (Hz)	Avg = D4 Mask	
	294	100%
	587	72.69%
	881	40.63%
	1175	11.49%
	1468	15.93%
	1762	11.61%
	2056	26.03%

Table 5.5: Mask values for note D4

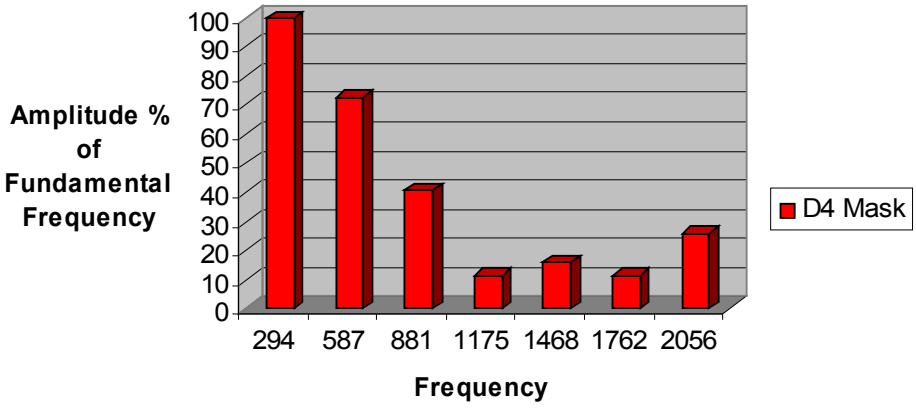


Figure 5.6: Plot of mask for note D4

Using this method to create the masks gives a broad base from which to calculate an average mask and so gives the masking system the best possible chance to accurately extract notes from a musical piece. All other masks were created using the same procedure and the full range of masks can be seen in Appendix C. The implementation of the masks and the overall system is discussed in detail in the next chapter, Chapter 6.

Chapter 6

Masking System

6.1 Mask Application

The Masking System is a polyphonic note detection system incorporating a simple masking technique that can accurately transcribe chords and polyphonic keyboard music. The Masking Application developed in Matlab [61], will take input files in wave, (.wav), format which are sampled at 12 kHz, as discussed in Chapter 4.1.1. The music is segmented by using Note Average Energy (NAE) [51] onset detection, which is described in Chapter 4.2. Onsets are used to segment the music into note windows, which are then analysed using the FFT. Following compilation of the frequency peaks in each note window, an iterative masking procedure is used to detect and extract the notes. The masking procedure uses a database of note masks which are compiled from multiple note examples using both monophonic and polyphonic examples to identify the notes in the music.

Once the wave file has been input to the system, the system will run automatically until completion and a list of notes played in the wave file is produced.

The length of time it takes for the system to complete can vary greatly, depending on variables such as the length, complexity and polyphony of the musical piece entered into the system. Figure 6.1 shows the system logic of the masking system for fundamental frequency detection in polyphonic music. The programming structure of the system is discussed in more detail in Chapter 7.

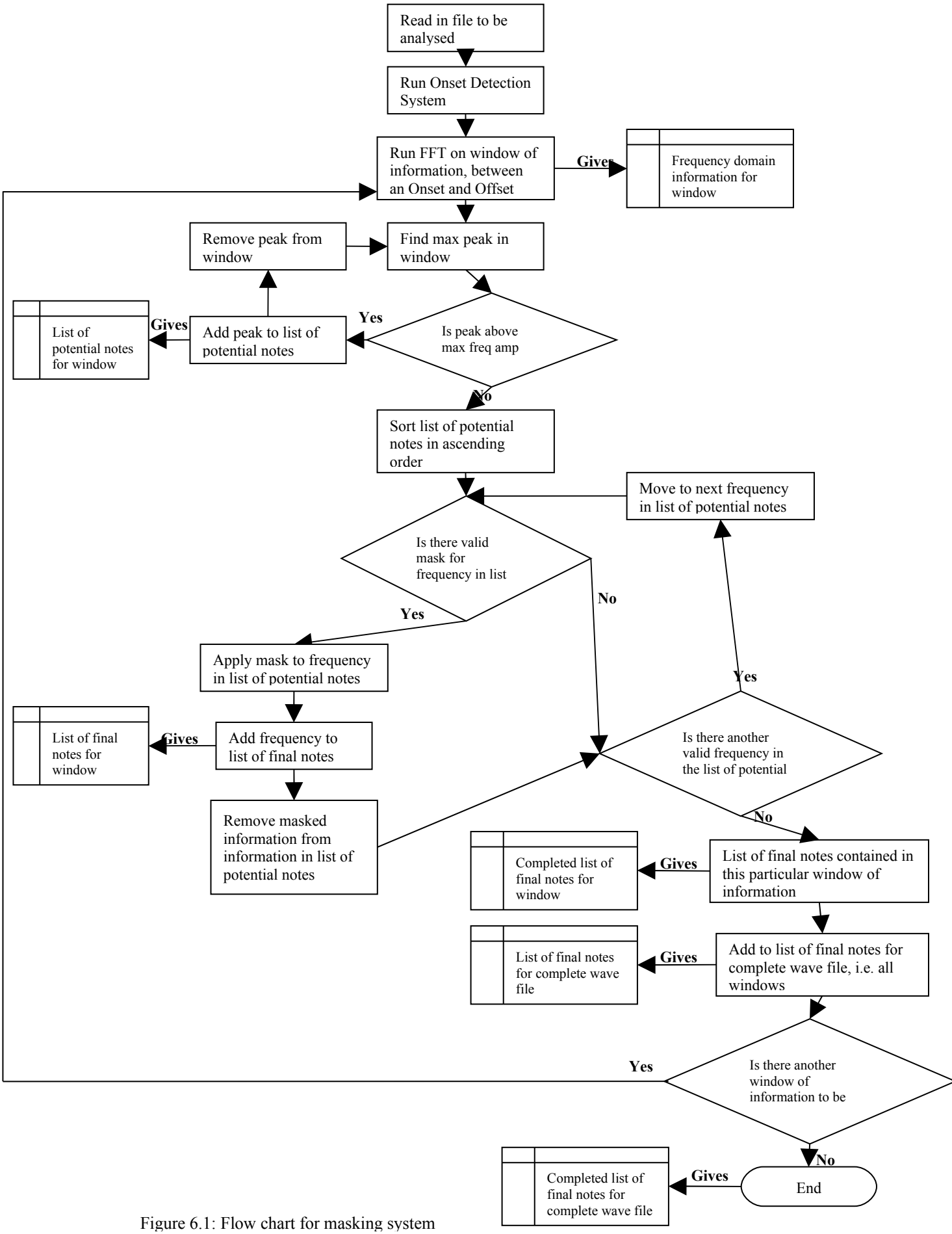


Figure 6.1: Flow chart for masking system

6.1.1 Extraction of Frequencies from the Note Window

Following the initial processing of the signal by the onset detection system into candidate note windows as described in Chapter 4.2, the first step in polyphonic note detection using a masking system is the compilation of a list of all significant frequency peaks found in each note window. This is done by a repeated procedure of peak-picking the highest frequency peak in the window and then removing it from the note window as described below.

The first step in the compilation of the list of significant frequency peaks is to find the largest amplitude frequency peak in a given note window. Amplitudes in the note window are then normalised by the amplitude of this frequency peak. The frequency value at this peak and its amplitude are used to begin the list of significant frequency peaks in the note window and the frequency peak itself is removed from the note window. Following removal of the first frequency peak, the peak with the largest amplitude in the note window is again identified (which will give the second highest frequency peak of the original note window, as the first peak has been removed). The frequency value of this second highest frequency peak and its amplitude are added to the list of significant frequency peaks in the note window and again the frequency peak is removed from the note window.

This process of peak finding and removal continues until the amplitude of the largest peak remaining in the modified note window falls below the maximum frequency amplitude threshold, which is discussed in Section 6.1.2. Following completion of this procedure, there is a resulting list of significant frequency peaks in the note window which are sorted in ascending order of frequency.

6.1.2 Maximum Frequency Amplitude Threshold

Due to the amount of information that a note window may contain, the peak picking process could continue cycling, detecting smaller and smaller frequency peaks that are of no significance to the masking system. A threshold was chosen, to limit the peak picking process so that peaks of insignificant amplitude would not be recorded. The threshold was set heuristically at 20% of the amplitude of the maximum frequency peak in the note window as, following analysis of the notes in the note database, it was found that spectral peaks below 40% of the fundamental frequency are usually high (> 5th) harmonics or noise (inharmonic), as seen for the example of note E3 in Table 6.1.

Harmonic	Frequency Detected	Amplitude Value Detected	Percentage of Fundamental Frequency
1st (Fundamental Frequency)	165 Hz	27.146	100 %
2nd	330 Hz	22.507	82.91 %
3rd	494 Hz	14.327	52.78 %
4th	659 Hz	15.597	57.46 %
5th	824 Hz	11.797	43.47 %
6th	989 Hz	4.4325	16.33 %
7th	1154 Hz	9.7568	35.94 %
8th	1318 Hz	3.9283	14.47 %
9th	1483 Hz	Not detected	N/A
10th	1648 Hz	3.826	14.09 %
11th	1979 Hz	5.4399	20.04 %
12th	2142 Hz	Not detected	N/A

Table 6.1: Comparison of the harmonic values of note E3

The threshold is set at 20%, half the empirically determined value to allow for variation in the size of the harmonics relative to the fundamental from pitch to pitch.

Setting the threshold at 20% instead of 40% does not appear to have an adverse affect on the detection system, because the masking procedure will remove small harmonics, below 40%, of a fundamental frequency before they would be "detected" as a fundamental frequency. A threshold of 20% is also well above most background noise that occurs in the signal and any detected frequency that does not match (within an accepted tolerance) an integer multiple of a fundamental frequency of a note is discarded, which should account for any noise that does go above the threshold.

6.1.3 Note Masking

In the second step of the polyphonic note detection process, the list of significant frequency peaks found in the note window has to be linked to notes. As the fundamental frequency of a note typically has the largest amplitude of all the frequency components associated with a note, the first frequency peak in the list, which will also be the lowest frequency in the list, is identified with the closest corresponding fundamental frequency of a note. A list of notes identified in the window is thus begun with the note which has a fundamental frequency corresponding to the lowest frequency peak detected.

From analysis of the notes in monophonic and polyphonic recordings, the fundamental frequency of a given note is not always present at exactly the expected frequency. The fundamental frequency has been seen to vary by 1 or 2 Hz in different recordings. This may be due to factors such as inaccuracies in the keyboard used for recording or noise in the recordings. For this reason a tolerance is introduced into the detection of frequencies. This tolerance varies depending on the note, based on analysis done on the detected frequencies of the note, but in general the tolerance for

frequency detection is +/- 1 Hz. Table 6.2 gives some examples of the frequency of a note according to Western Tonal Music theory and also the values for which the masking system will recognise a specific note. Values for the masking system are integer multiples for ease of use.

Note	Western Tonal Frequency (Hz)	Masking System Frequencies (Hz)
A2	110	109, 110 & 111
C3	130.81	129, 130 & 131
D4	293.66	292, 293 & 294
F5	698.46	697, 698 & 699
G6	1567.98	1567, 1568 & 1569

Table 6.2: Frequency tolerance in note detection

A mask, corresponding to the identified note, is then applied to the significant frequency peaks list. The mask removes all of the amplitude of the first frequency peak, from the list of significant frequency peaks. As the mask also contains a proportional representation of the harmonics of the typical note to which it corresponds, the appropriate proportion of the amplitude of the harmonics of the note are removed from the frequencies which are present in the list of significant frequency peaks. The modified list of significant frequency peaks will thus no longer contain the first note fundamental, and of the other frequencies in the list, those which are harmonics of that fundamental, will have had their amplitudes reduced by the mask.

Once the first frequency has been removed from the list of significant frequency peaks, the procedure is repeated for the next peak in the list. The amplitude of this peak may have been modified by a previously run mask. If the amplitude of the frequency peak is still above the maximum frequency amplitude threshold the frequency value of the peak is compared to note fundamental frequencies and the note corresponding to the closest fundamental frequency is selected as the next note identified in the note window. The mask corresponding to the newly identified note is applied to the list of significant frequency peaks. Again the peak at the fundamental frequency is removed and the peaks at harmonics of that fundamental frequency are reduced by the applied mask. On the other hand, if the amplitude of the frequency peak is not above the maximum frequency amplitude threshold, then that frequency is discarded, since it is likely that any remaining energy at that frequency is due to noise and does not indicate the presence of a note. This procedure continues until there are no notes remaining in the list of significant frequency peaks.

The result of repeated application of the masking procedure is a list of notes for a note window compiled during the masking procedure. This list is in the form of fundamental frequencies which have been extracted from the original frequency domain note window. This iterative masking procedure is run on every note window detected in the piece of music by the onset detection step to create a final list of notes which may then be translated into alphanumeric characters to represent the musical notes that were detected by the masking system. An example of the functioning masking system is shown below.

Figure 6.2 shows an example frequency plot of notes C4, E4 and G4 played together in a window.

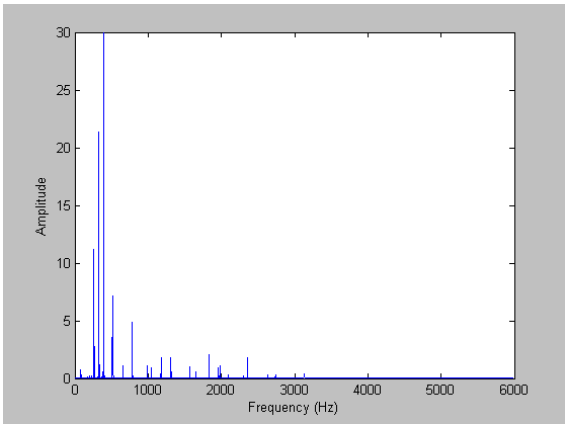


Figure 6.2: Frequency plot of C4, E4 and G4 played together.

In this note window, the maximum amplitude occurs at the frequency of 392 Hz, (G4), and its value is 29.9. This peak is removed from the window and added to the list of significant frequency peaks and a check is run on the note window for the next highest peak.

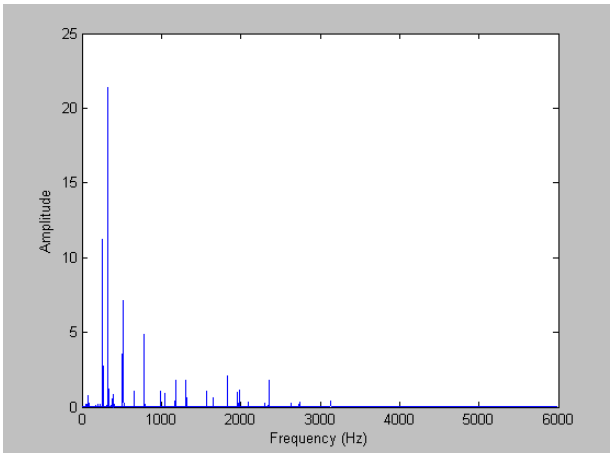


Figure 6.3: Frequency plot of C4, E4 and G4 with the peak at 392 Hz removed.

This continues until the maximum peak in a modified window falls below the maximum frequency amplitude threshold.

The result of the iterative peak-picking procedure is a list of the significant frequency peaks for a window, which is then sorted in order of frequency. The maximum peak is at 392 Hz with an amplitude of 29.918. Any values over the 20 %

maximum frequency amplitude threshold, approximately 5.98 in this example, will be included in the list below.

Frequency (Hz)	Amplitude
262	11.191
330	21.358
392	29.918
523	7.131

Table 6.3: Detected amplitude values

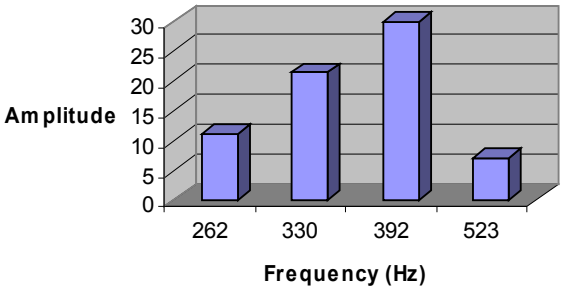
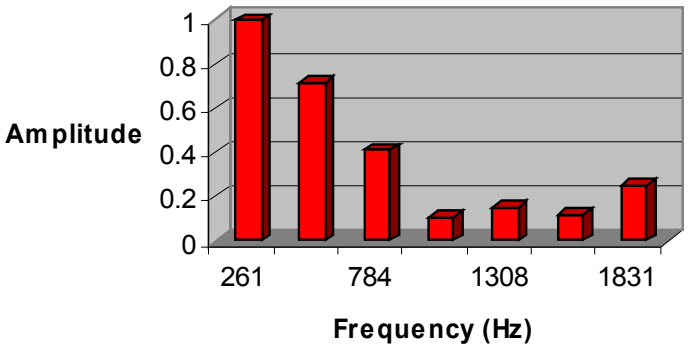


Figure 6.4: Detected amplitude values

Next a mask is applied to the list of significant frequencies beginning with the first frequency in the list. Frequency 262 Hz corresponds to note C4 and returns a mask for that value, as shown in Figure 6.5. The first frequency, 262 Hz, is added to the final list of notes played in the note window. The amplitude value of the 262 Hz frequency peak, 11.191, is multiplied by each of the masking values and the resulting values are taken away from the corresponding frequency values.



For each frequency value in the associated mask;

Masking Value x Maximum Amplitude = Value to be removed

Below, Table 6.4 shows this calculation with respect to the C4 mask.

Frequency (Hz)	Masking Value		Max Amplitude		Value to be removed
260, 261, 262	1.0	x	11.191	=	11.191
523, 524	0.72	x	11.191	=	8.058
784, 785	0.41	x	11.191	=	4.588
1046, 1047	0.11	x	11.191	=	1.231
1308, 1309	0.15	x	11.191	=	1.679
1569, 1570	0.12	x	11.191	=	1.342
1831, 1832	0.25	x	11.191	=	2.798

From Table 6.4, only two frequencies from the C4 mask, 262 Hz and 523 Hz, occur in the list of significant frequency peaks from the original note window, see Table 6.3. The amplitude of these detected values will be reduced by the value calculated in Table 6.4.

Figure 6.6 shows the frequencies and the associated amplitudes from the significant frequency peak list. The values in red are the values from the C4 mask that will be taken away from the list of detected values. Figure 6.7 shows the values left after the mask has removed the values.

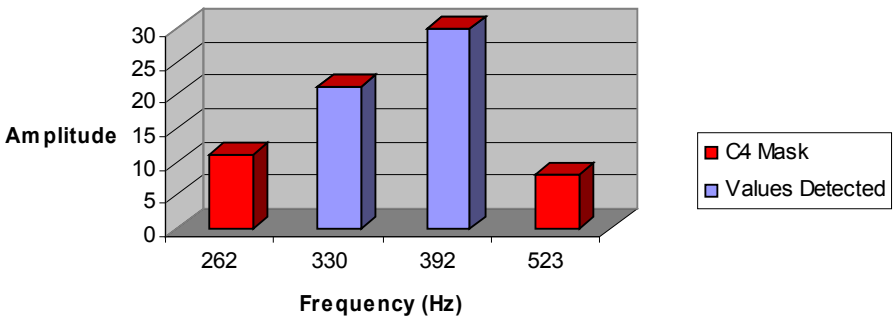


Figure 6.6: Detected values and values to be removed by the C4 mask

After the computation has been completed, the values remaining in the list of significant frequency peaks will appear as illustrated below:

Frequency (Hz)	Amplitude	
262	0	(11.191 – 11.191 = 0) Played Note
330	21.358	
392	29.918	
523	0	(7.131 – 8.058 = -0.927 -> Value thresholded at 0 as amplitude value cannot have a negative value)

Table 6.5: Remaining values after C4 mask values are removed

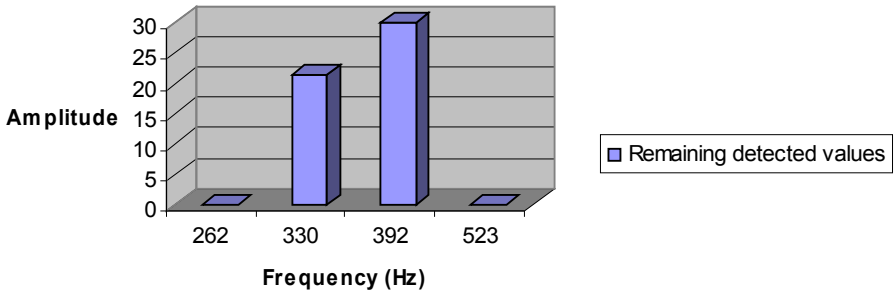


Figure 6:7: Remaining values after C4 mask values are removed

The next frequency in the list of significant frequency peaks, Table 6.5, is 330 Hz, which corresponds to note E4. The amplitude value of the 330 Hz frequency peak, 21.358, is still above the maximum frequency amplitude threshold, which is 5.98, as the C4 mask did not affect the amplitude of the 330 Hz frequency peak. After this threshold check is run and passed, 330 Hz is added to the list of final notes played in the note window. The amplitude value of the 330 Hz frequency peak, 21.358, is then multiplied by each of the masking values for E4 and the resulting value is taken away from the corresponding frequency value.

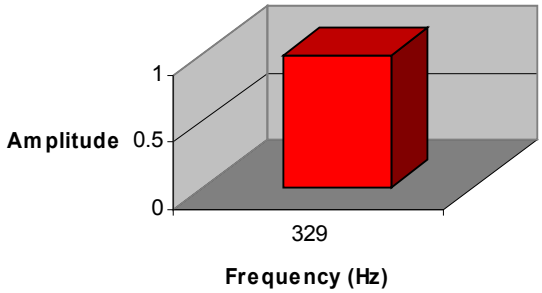


Figure 6.8: E4 masking value

Below, Table 6.6 shows this calculation with regards the E4 mask.

Frequency (Hz)	Masking Value		Max Amplitude		Value to be removed
328, 329, 330	1.0	x	21.358	=	21.358

Table 6.6 – Note E4 Masking Calculations

From Table 6.6, note E4 has only one masking value. At some of the higher frequencies, such as those greater than C6, the fundamental frequency is much larger then the associated harmonics, and so the harmonics fall below the maximum frequency amplitude threshold resulting in a mask containing only the fundamental frequency. While this is common in the higher harmonics, it also occurs in some notes in lower octaves, such as E4 and G4. A complete list of the masks can be seen in Appendix C. For note E4 the single masking value is at the fundamental frequency, between 328 and 330 Hz and so only the value of 21.358 at 330 Hz will be removed from the table of remaining frequency values, Table 6.5. Figure 6.9 shows the remaining frequencies and the associated amplitudes from the detected list. The values in red are the values from the E4 mask that will be taken away form the list of significant frequency peaks. Figure 6.10 shows the values left after the mask has removed the values.

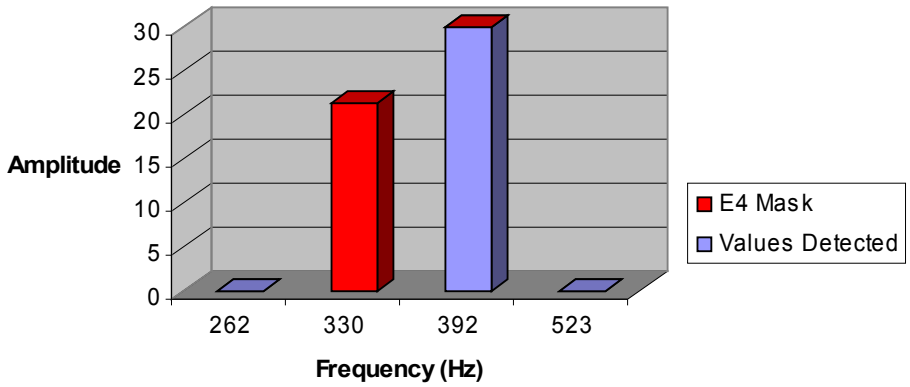


Figure 6.9: Detected values and values to be removed by the E4 mask

After the computation has been completed, the values remaining in the list of significant frequency peaks will look as below:

Frequency (Hz)	Amplitude	
330	0	(21.358 – 21.358 = 0) Played Note
392	29.918	

Table 6.7: Remaining values after E4 mask values are removed

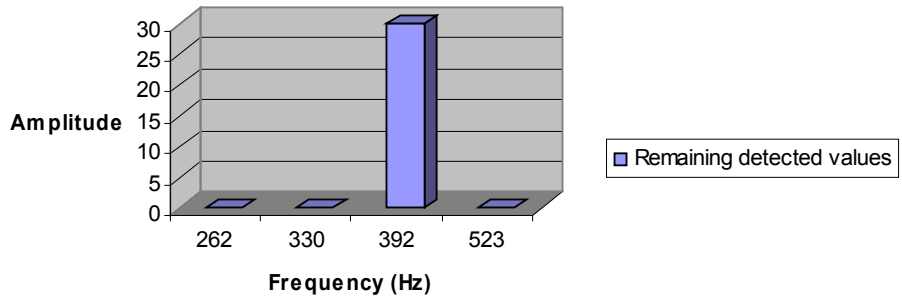


Figure 6:10: Remaining values after E4 mask values are removed

The next frequency value in the table of remaining notes, Table 6.7, is 392 Hz, which corresponds to note G4. The amplitude value of the 392 Hz frequency peak, 29.918, is still above the maximum frequency amplitude threshold, which is 5.98, as neither

the C4 nor the E4 mask reduced the amplitude of the 392 Hz frequency peak. After this threshold check is run and passed, 392 Hz is added to the list of final notes played in the note window. The amplitude value of the 392 Hz frequency peak, 29.918, is then multiplied by each of the masking values for G4 and the resulting value is taken away from the corresponding frequency value.

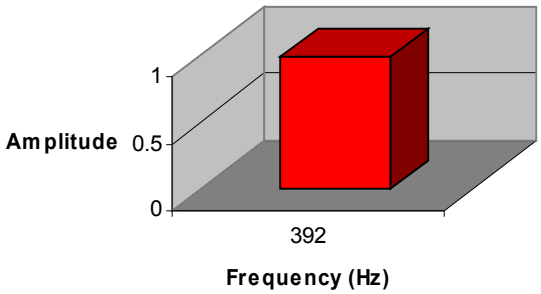


Figure 6.11: G4 masking value

Below, Table 6.8 shows this calculation with regards the G4 mask.

Frequency (Hz)	Masking Value		Max Amplitude		Value to be removed
391, 392, 393	1.0	x	29.918	=	29.918

Table 6.8 – Note G4 Masking Calculations

From Table 6.8, note G4 has only one masking value, which is at the fundamental frequency, between 391 and 393 Hz and so only the value of 29.918 at 392 Hz will be removed from the table of remaining frequency values, Table 6.7. Figure 6.9 shows the remaining frequencies and the associated amplitudes from the detected list. The values in red are the values from the G4 mask that will be taken away from the list of detected values.

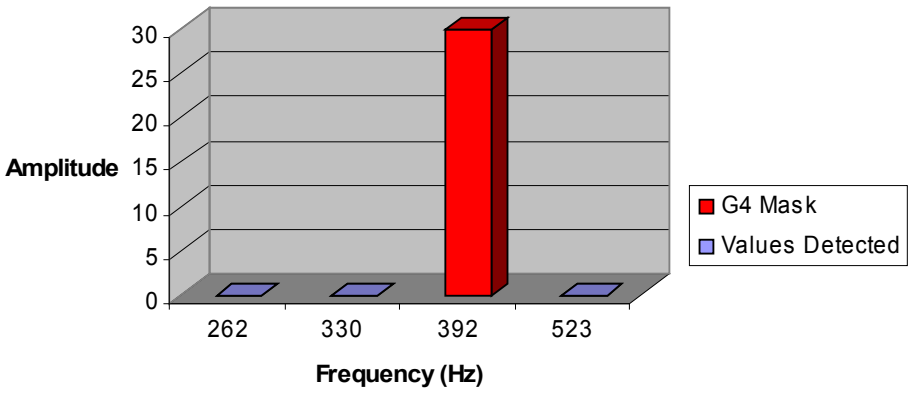


Figure 6.12: Detected values and values to be removed by the G4 mask

After the computation has been completed, there are no values remaining in the list of significant frequency peaks:

Frequency (Hz)	Amplitude	
392	0	(29.918 – 29.918 = 0) Played Note

Table 6.9: Remaining values after final values are removed

As there are no more values in the list of significant peaks, the analysis for this note window is complete. Table 6.10 shows the list of final notes played in this note window that were detected by the masking system.

Frequency (Hz)	Note
262	C4
330	E4
392	G4

Table 6.10 – List of final notes for this window

This list of final notes for this note window is added to the list of final notes for the complete wave file. The list of final notes for this note window is re-set and ready for

use again when the system moves onto the next note window that has been detected by the onset detection system. The next note window is analysed using the same method as described above, until a new completed list of final notes for the new note window is identified and is then added to the list of final notes for the complete wave file. This loop continues until the entire set of note windows have been analysed, which results in a completed list of final notes for the complete wave file and the analysis of the wave file is then complete.

Resulting from this analysis, notes C4, E4 and G4 were detected as the final notes in this window. While there was a frequency peak detected at 523 Hz, which corresponds to note C5, the masking system removed this frequency peak from the initial list of detected frequency peaks, Table 6.3, as the frequency peak was a harmonic of note C4 and not note C5 being played. While this is a fairly simple example with only one harmonic detected in the list, the basic premise for how the masking system is implemented can clearly be seen as well as the ability of the masking system to successfully deal with problems such as harmonics and the octave problem.

Chapter 7

Implementation

In this chapter the overall structure of the masking system is discussed. The masking system is written in MATLAB and is composed of two main sections: the Onset Detection section and the Masking section. A complete system structure and block diagram for the Onset Detection part is detailed in Appendix A, but the main focus of this chapter is on the Masking section of the system.

7.1 System Structure

The Onset Detection section of the system is represented below in Figure 7.1, as *TFHS_script*, which is the main function in the Onset Detection part of the system. Its child functions are not shown here but can be seen in Appendix A. The second section of the system is the masking section. Figure 7.1 shows the block diagram for the transcription system followed by a description of each of the functions and how they each relate to the system.

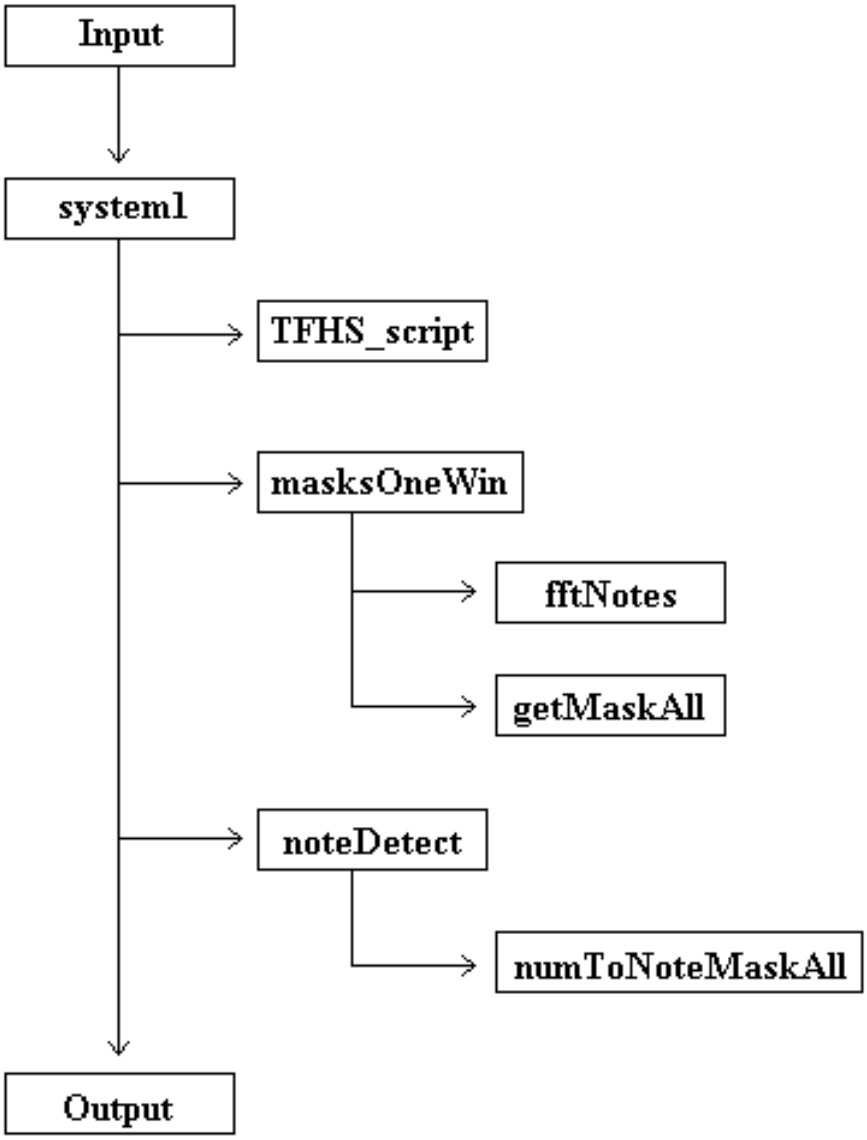


Figure 7.1: Transcription System Structure

Summary of Functions

system 1 - The main function

Input: The wave file (file) is taken as input.

Output: Returns the list of notes played in the file and also the times the notes were played at and their duration, (noteArray).

- TFHS_script - is a script as opposed to a function. A script runs from start to finish, although it may call other scripts or functions during run-time, where as a function is a section of code which has a specific input and is called by the main program to complete certain computational tasks and return the output to the main program. TFHS_script is used to detect the onsets of the notes in the music.

  Input: It takes a wave file (file) as an input.

  Outputs: Holds the sampled information from the file, (Y0), the sampling frequency, (FS), the sample numbers at which the onsets occur, (sampnums) and the sample number for the last offset of the input file, (maxsamp).

- masksOneWin: gets the frequency domain information for the windows by calling function *fftNotes* and also implements the masking system, and in doing so calls function *getMaskAll*.

  Inputs: Takes the sampled information from the file, (Y0), the sampling frequency, (FS), the onset of the note, (a), and the offset of the note, (b).

  Outputs: A list of the played frequencies, (notes), and a list of detected frequencies and their associated amplitudes, (outputColwAmps).

- fftNotes: is called by *masksOneWin*. It is used to calculate the frequency domain information for a specific window.

Inputs: The sampled information from the file, (x), the sampling frequency, (FS), the onset of the note, (a), and the offset of the note, (b). These are the same inputs as *masksOneWin*.

Outputs: Its outputs are the frequency values from the frequency domain, (f) and their associated amplitude values, (fftX).

- getMaskAll: is called by masksOneWin and returns a mask that is to be applied to the frequency domain information.

  Input: Takes in the detected frequency, (freq).

  Output: The mask as an array of numbers, (mask).

- noteDetect: implements the formatting of the information so that the output can easily be read.

  Inputs: Takes an array of numerical values which represent the notes played, (initNotes), and also takes in the sampling frequency, (FS), which is used to calculate the timing of the notes.

  Output: A formatted array containing the notes played and the onset and offset times of these notes, (noteArray). This is also the output of the main function *system1*.

- numToNoteMaskAll: is called by function *noteDetect*.

  Inputs: It takes in the frequencies of notes that have been detected as played notes, (allNotes).

  Output: An array, (finalNts), as alphanumeric characters that represent standard note values, e.g. C4, C#4.

7.2 System Implementation

7.2.1 system1

The function, *system1*, takes in a piece of music, in a wave file format, that is to be analysed and outputs a list of notes and the onsets and offsets of these notes. Whether the piece of music is monophonic or polyphonic or a mixture of both is irrelevant to the system as both are handled the same way and the output of the notes clearly shows whether sections of music are either monophonic or polyphonic.

This is the main function of the transcription system, and from here, all other functions are called. First *TFHS_script* is called, to locate the onsets and offsets in the piece of music. Next *system1* loops until the information between all onsets and offsets has been examined. During this loop it calls the function *masksOneWin* and its child functions, *fftNotes* and *getMaskAll*. After the loop exits, when the last onset and offset have been analysed, function *noteDetect* is called, which in turn calls its child function *numToNoteMaskAll*. An array is passed back from *noteDetect*, which is the final output of the system; the notes played and their onset and offset times.

7.2.2 TFHS_script

The script *TFHS_script* is responsible for the onset detection of the system. The script is run making use of 17 different functions and returns the onsets of all the notes in the form of the sample numbers where the onsets take place. The offsets of the notes are taken to be at a sample number slightly less then the value of the

subsequent onset. This script implements the onset detection system as presented in [51] and is described further in Section 4.2 and in Appendix A.

7.2.3 masksOneWin

After the onsets in the music have been located, a loop is started, where the information between each onset and offset is analysed in turn. During each iteration of this loop, the function *masksOneWin* is called. This takes in, among other values, the sample numbers between which a note or notes have occurred, i.e. between an onset and an offset, as defined in Section 3.2.2, Note Onset Detection. First the frequency domain information for this window is obtained by calling function *fftNotes*. When the frequency domain information has been returned, the negative frequencies from the frequency domain calculations are discarded. Next a list of frequencies, which are above the maximum frequency amplitude threshold of 20% of the maximum amplitude in the window, is compiled. The maximum frequency amplitude threshold is described in Section 6.1.2.

The function *getMaskAll* is then called to get the masking values associated with each of the frequencies in the list. The mask loops through the list, removing information and notes using the masking technique, as described in Chapters 5 and 6. When all frequency values in the list have been examined, *masksOneWin* returns the detected frequencies and their associated amplitude values to its parent function, s*ystem1*.

7.2.4 fftNotes

The function *fftNotes* returns the frequency domain values for the specific window that has been passed into the function. To calculate the frequency domain information, an FFT is taken using the Matlab function for the Hamming window to calculate the frequency domain information. The absolute value of the FFT is then taken and passed back to the parent function, masksOneWin.

7.2.5 getMaskAll

As the system cycles through the list of detected notes, function *getMaskAll* is called by *masksOneWin*. Provided that there is a mask associated with that frequency, *getMaskAll* will return the mask for the specific frequency. If not the frequency is discarded as noise. The mask is then applied to the frequency domain information and the detected note is extracted. This continues until all the frequencies in the list have been dealt with. The notes played in the window and the amplitudes associated with the frequencies are then returned from function *masksOneWin* to its parent function, *system1*. The masking procedure is described further in Section 5.2.

7.2.6 noteDetect

Function *system1,* cycles through all the windows repeating the same procedures. The notes, their amplitudes and the values of the onsets and offsets of the windows, which will be used for calculating the timing of the notes, are amalgamated into one array, called *allNotes*. To calculate the timing of the notes in the piece of music the sample numbers that delimit the note, between the onset and offset, are divided by the sampling frequency of the recorded music to give the approximate times of the offsets and onsets.

$$\frac{\text{onset sample value}}{\text{sampling frequency}} = \text{approx time of note beginning} \quad (7.1)$$

$$\frac{\text{offset sample value}}{\text{sampling frequency}} = \text{approx time of note ending} \quad (7.2)$$

This array is then passed into a function called *noteDetect*. A second check for noise is implemented here, checking the amplitude values of the frequencies and if they are found to be below the maximum frequency amplitude threshold, then these frequencies are eliminated.

7.2.6.1 Checking for Repeated Notes

If a note occurs in successive windows, it is possible that that note was played in the first window and then played in the second window. Also possible is that the note was played in the first window and was held when played so the note lasts for longer and may be detected in the next window. This would be a common occurrence when a chord is being played. To account for this, function *noteDetect* checks to see if the same note was played in windows subsequent to the one currently being processed and if so, then the amplitudes of the notes are compared. Typically, when a note is played, the amplitude of the note rises and peaks rapidly after onset and then declines slowly until the offset is reached, as shown in Figure 7.2.

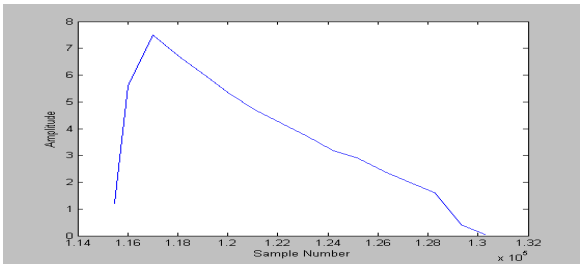


Figure 7.2: Amplitude peak and decline during note F4

Note: In Figure 7.2 the amplitude values are relatively small because the note F4 was analysed in small window increments to show how the amplitude envelope of a note behaves between the onset and offset of the note.

The test that is applied is as follows: if the value of the amplitude in the second window is above 50% of the amplitude in the first window, then that is recorded as a new note. If however the amplitude of the second window is below this 50% threshold, then it is recorded as a continuation of the note from the last window.

7.2.7 numToNoteMaskAll

Up to this point the system uses numerical values (i.e. frequency in Hz) to represent the notes. The final step is to change these numerical values into the appropriate alphanumeric characters that represent the notes played, so that the output of the system can be easily interpreted. A call to function *numToNoteMaskAll* returns these note values to function *noteDetect*.

7.3 System Limitations

This system detects notes and the timing of these notes in polyphonic keyboard music. It does not give rhythm or beat patterns of this music, so an exact transcription is not achieved. Also the system is limited to keyboard music, and has only been tested with music played on an electronic keyboard. The masking theory should be valid for other instruments, but new masks would need to be created for specific instruments. The problem of instrument recognition is another difficult problem as described in [65]. If instrument recognition and separation of each

instrument could be achieved, the masking system could then be used for the transcription of each individual instrument. This could be the subject of further work.

For this programme to work successfully, it relies on two key components to function properly, the onset detection system and the structure of the notes and their harmonics.

7.3.1 Onset Detection Limitations

The Onset Detection system, while it mostly returns the correct onset times, can produce errors, in that it may return an extra onset or it may miss an onset altogether. These problems can occur in both monophonic and polyphonic music but for simplicity in description, the examples below assume monophonic music. The resulting output would be the same for polyphonic music.

7.3.1.1 Extra Onset Detection

In the first case, where an extra onset is detected, this can sometimes be overcome. Since the system takes the note as a whole, if an extra onset is detected then the system will analyse that section. If that window is just noise where no note was played, then it will be eliminated from the list because the noise values will not be above the maximum frequency amplitude threshold and no note will be detected. If a spurious onset is detected in the middle of a note that is still being played, the note will be divided up into two different windows. The programme will detect the note in the first window and the second window. The amplitude values for the note will likely show that it is the same note being played – as discussed in section 7.2.6.1 Checking for Repeated Notes. Therefore the output will enter the note in the second

window as being continued from the first window. The times of the onset and offset will also show this, thereby eliminating the problem of a spurious onset.

7.3.1.2 Missed Onset

However should an onset be missed by the detection system, then problems will occur because although two notes were played in succession, only one window will be used for the analysis, as only one onset was detected. This could result in several possibilities depending on the make up of the music being analysed. One possibility is that all the notes will be detected together and it will seem that all the notes in the window were played at the same time. Also possible is that an incorrect frequency will be detected and an incorrect note is sent to the output, unless the frequency detected doesn't match the frequency of any note, in which case the programme will assume that no note was played there.

7.3.2 Masking Limitations

7.3.2.1 Harmonics Larger than Mask

The masking system is based on information gathered from looking at the harmonic structure and make up of notes played on the keyboard and from this a model is created of each individual note. This model is then applied to the frequency amplitude spectrum of a piece of music to detect which notes have been played. The system breaks down if a note deviates from the known model summarised by the collection of note masks developed. This can occur if for example, the second harmonic is abnormally large in comparison with the first harmonic. The mask will detect the first harmonic as the note but will not remove enough of the second

harmonic, as it is too large in amplitude. Because of this there is an extra peak left in the information and this peak could then, depending on its size, be detected as another note despite its actually being a harmonic.

7.3.2.2 Harmonics Smaller than Mask

Also the inverse is true and has been found to happen sometimes. Say, for example, that two notes were played, both the same note but one octave apart, C4 and C5. If the value of C5 is too small in amplitude, then the C4 mask could remove too much of the C5 value, as it looks as if it is a harmonic of C4. As a result, note C5 will not be detected. Depending on the circumstances, if the harmonic at C6 is large enough, because C5 was not detected, the C5 mask will never run on the information. Therefore none of the amplitude due to the harmonic at C6 will be removed, (and very little may be removed by the C4 mask, as C6 is the fourth harmonic of C4). This will in turn lead to the detection of note C6. Instead of notes C4 and C5 being detected in a window, notes C4 and C6 may be detected instead. This octave problem occurs when the same notes that are in consecutive octaves, occur in the same window. In this work (or 'With this instrument..') the problem mainly occurs in the third (C3, C#3, D3…B3), and fourth (C4, C#4, D4…B4), ranges of notes, and is more likely to occur in relation to C and D notes, including sharp, (C#), and flat, (D_b), notes. Although this may not happen in music very often and if it does occur, it will mostly be detected correctly, it can occur and give incorrect results.

Below are two examples of this problem, where notes C#3 (139 Hz) and C#4 (277 Hz) are played simultaneously. Figure 7.3 shows notes C#3 and C#4 being correctly detected and is also shown graphically in Figures 7.4 to 7.7. Figure 7.8

shows notes C#3, C#5 and G#5 being incorrectly detected and this is also shown in Figures 7.9 to 7.13.

C#3 & C#4		C#3 Mask		After Mask Applied			Values Remaining		C#4 Mask		After Mask Applied			Values Remaining		
Hz	Amplitude	Hz	Amplitude	Hz	Amplitude		Hz	Amplitude	Hz	Amplitude	Hz	Amplitude		Hz	Amplitude	
139	36.21	139	1	139	0		277	11.172	277	1	277	0		0	0	
277	41.95	277	0.85	277	11.172				554	0.72						
416	19.34	416	0.46	416	2.682	*			832	0.42	832	4.662	*			
555	9.178	555	0.51	555	-9.29	*	1387	9.397			1387	9.397	*			
693	14.29	693	0.37	693	0.897	*	1940	9.476	1940	0.17	1940	7.58	*			Finished Analysis
832	8.13			832	8.13	*										
971	12.8	971	0.38	971	-0.96	*										
1387	9.397			1387	9.397											
1644	9.504	1664	0.14	1664	4.436	*										
1940	9.476			1940	9.476											
139	C#3						277	C#4								

Notes in Red are notes that have been detected.

Notes **C#3** and **C#4** were played and detected.

* = Amplitude below maximum frequency amplitude threshold or frequency value not recognised, so note removed.

Figure 7.3: Masking process applied to notes C#3 and C#4, and these notes being correctly detected.

Figure 7.4 shows the detected amplitude values in blue and the values that will be removed by the C#3 Mask in red. Figure 7.5 shows the remaining values after the C#3 masked values have been removed.

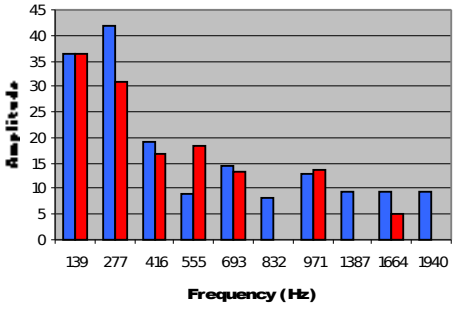


Figure 7.4: Amplitude values shown with the values to be removed by the C#3 Mask.

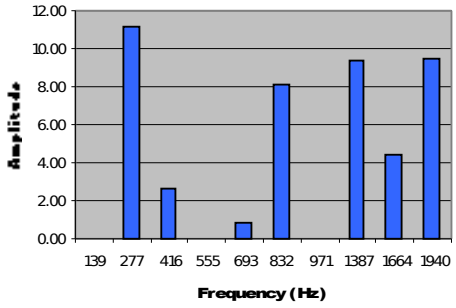


Figure 7.5: Values remaining after C#3 Mask has been applied.

Figure 7.6 shows the remaining amplitude values in blue and the values that will be removed by the C#4 Mask in red. Figure 7.7 shows the remaining values after the C#4 masked values have been removed. These remaining values are too small, below the maximum frequency amplitude threshold and are therefore discarded.

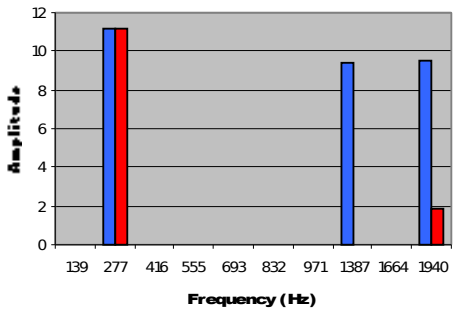


Figure 7.6: Amplitude values shown with the values to be removed by the C#4 Mask.

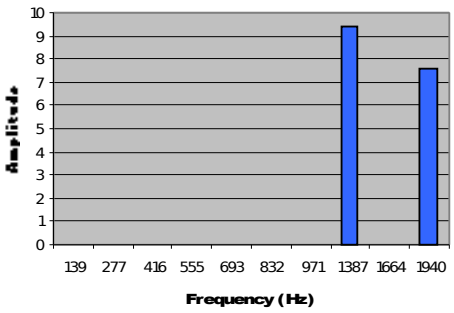


Figure 7.7: Values remaining after C#4 Mask has been applied.

In Figure 7.3, C#3 has an amplitude of 36.21 and C#4 has an amplitude of 41.95. This indicates that both C#3 and C#4 were played at the same time. The second harmonic of C#3 adds to the amplitude value of the fundamental frequency of C#4, making the amplitude value at C#4 greater than the amplitude value at C#3. The masking system extracts note C#3 first, then extracts note C#4 from the data. No more notes are detected, as there are no more valid amplitude values, so notes C#3 and C#4 are correctly detected by the system.

Figure 7.8 shows notes C#3, C#5 and G#5 being incorrectly detected.

C#3 & C#4		C#3 Mask		After Mask Applied		Values Remaining		C#5 Mask		After Mask Applied		Values Remaining		G#5 Mask		After Mask Applied		Values Remaining		
Hz	Amplitude	Hz	Amplitude	Hz	Amplitude	Hz	Amplitude	Hz	Amplitude	Hz	Amplitude	Hz	Amplitude	Hz	Amplitude	Hz	Amplitude	Hz	Amplitude	
139	36.68	139	1	139	0	555	0.23	555	1	555	0	832	12.18	832	1	832	0	0	0	
277	26.62	277	0.85	277	-4.56	*	832	12.18			832	12.18								
416	19.73	416	0.46	416	2.86	*														
555	39.94	555	0.51	555	21.23															Finished Analysis
693	14.44	693	0.37	693	0.87	*														
832	12.18			832	12.18															
971	12.94	971	0.38	971	-0.99	*														
1664	9.374	1664	0.14	1664	4.24	*														
139	C#3					555	C#4					832	C#5							

Notes in Red are notes that have been detected.

Notes C#3 and C#4 were played but notes C#3, C#5 and G#5 were detected.

* = Amplitude below maximum frequency amplitude threshold, so note removed.

Figure 7.8: Masking process applied to notes C#3 and C#4, but C#3, C#5 and G#5 are detected.

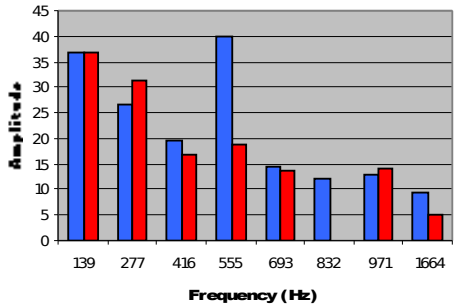


Figure 7.9: Amplitude values shown with the values to be removed by the C#3 Mask.

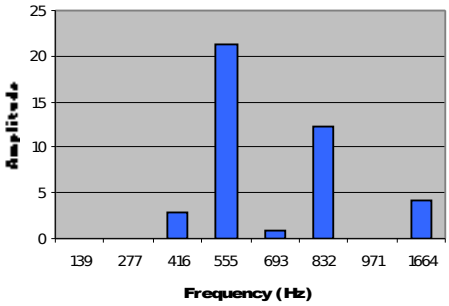


Figure 7.10: Values remaining after C#3 Mask has been applied.

Figure 7.9 shows the detected amplitude values in blue and the values that will be removed by the C#3 Mask in red. Figure 7.10 shows the remaining values after the C#3 masked values have been removed.

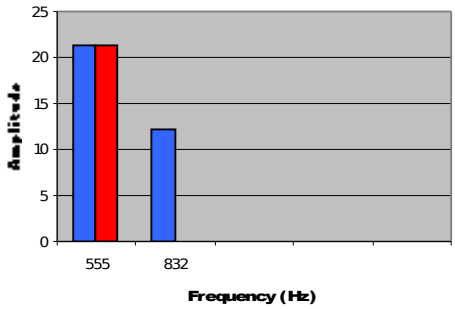


Figure 7.11: Amplitude values shown with the values to be removed by the C#5 Mask.

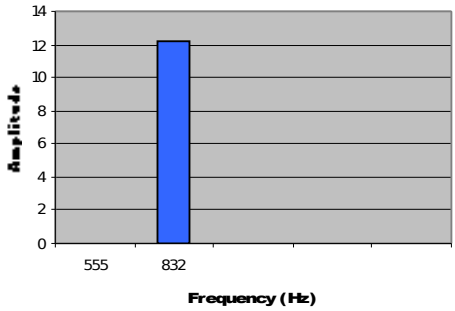


Figure 7.12: Values remaining after C#5 Mask has been applied.

Figure 7.11 shows the remaining amplitude values in blue and the values that will be removed by the C#5 Mask in red. Figure 7.12 shows the remaining values after the C#5 masked values have been removed. The amplitude value at 693 Hz is below the maximum frequency amplitude threshold and is removed.

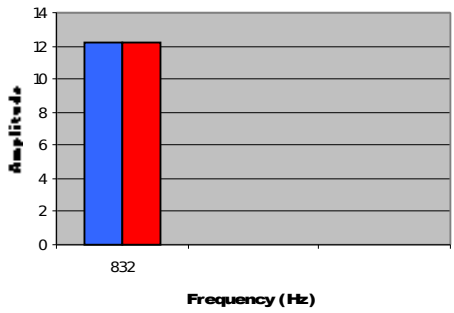


Figure 7.13: Amplitude values shown with the values to be removed by the G#5 Mask.

Figure 7.13 shows the remaining amplitude values in blue and the values that will be removed by the G#5 Mask in red. The remaining values are too small, below the maximum frequency amplitude threshold and are therefore discarded.

From Figure 7.8 note C#3, 139 Hz, has an amplitude value of 36.68 and C#4, 277 Hz, has an amplitude value of 26.62. This indicates that the value at the frequency of 277 Hz is a harmonic value and that the note C#4 was not played. Because the amplitude value at 277 Hz is lower in comparison to the amplitude value at 139 Hz, when the mask for note C#3 is run, it will completely remove the peak at 277 Hz. Because of this, note C#4 will not be detected as a note and the mask associated with C#4 will not be run. The next note, with a viable amplitude is found, which is note C#5, 555 Hz. C#5 is then taken to be a note that was played and the C#5 mask is then run. After this note G#5, 832 Hz, is detected by the system and its mask is run. After this no other note is detected. Had the value at 277 Hz been greater, the C#4 mask would have run. This would have removed the frequency peaks at 555 Hz and 832 Hz, as these are the second and third harmonics of C#4, and notes C#5 and G#5 would not have been detected. As seen in Figures 7.8 and 7.9, the amplitude value at 277 Hz was smaller in comparison to the amplitude value at 139 Hz and this lead to the incorrect detection of notes C#3, C#5 and G#5.

7.4 Conclusion

This system relies on the two main sections, firstly the Onset Detection section to reliably and correctly detect the onsets and offsets in a piece of music. The Masking section is then needed to examine the information between the onsets and offsets and to then remove any played notes from the window of information, without mistakenly extracting a harmonic of a note and labeling it as a played note. There are some functions built into each section that can account for some errors, as described above, but ultimately these two sections need to work together to accurately transcribe polyphonic piano music.

Chapter 8

Masking System Transcription Results

In this chapter, the results of testing the polyphonic note detection system in two different settings are presented. The first test involves chords played with approximately 0.5s separating each set of notes. In this setting, the note onsets are straightforward to detect and so the performance of the masking system is mostly unaffected by any limitations of the onset detection system. The second test setting involves transcribing pieces of music using the masking system as they are played at different realistic tempos. While the first test of transcription is not realistic with regards to transcribing music played on a keyboard, it does give a good indication of the potential of the masking system and the second test gives a more realistic test of actual pieces played on the keyboard modeled in this work.

8.1 Assumptions

The main assumptions in the work reported are that the music is polyphonic Western Tonal music consisting of either keyboard chords or polyphonic keyboard music. As the music is polyphonic, it is not known *a priori* how many simultaneous

notes exist in each window. It is further assumed that all notes have the fundamental present. The range of notes is from C2 to B6.

8.2 Results – Isolated Notes Played 0.5s Apart

In this section, the results for the masking system transcribing sets of notes or chords that are played 0.5s apart are discussed. By having the sets of notes 0.5s apart, errors caused by the onset detection system are minimised. Table 8.1 gives the results when the transcription system is applied to chords of varying degrees of polyphony and where the chords are played approximately a half second apart. The percentage error is calculated as:

$$\%E = \left((m + x)/n\right)100\% \tag{8.1}$$

where m is the number of missing notes, x is the number of extra notes and n is the total number of notes detected. The final row of Table 8.1, Total, gives the sum of the notes detected and the sum of the notes in error. These values are then used to calculate the overall percentage error rate of 6.77%.

	Notes Detected	**Notes in Error**	**% Error**	**Success Rate (%)**
Chords	1906	146	7.66	92.34
High Polyphony Chords (5 – 8 notes)	225	18	8.0	92.0
Chords (Triads/4 notes)	638	20	3.13	96.87
Chords sampled at diff. freq.	139	13	9.35	90.65
Total	2908	197	6.77	93.23

Table 8.1: Results when note onsets are approximately 0.5s apart.

From the results in Table 8.1 the masking system can accurately detect the notes played with an overall success rate of 93.23 %. When higher levels of polyphony are

applied, greater then 5 notes played simultaneously, the error rate increases to 8%, which is slightly above the overall average value of 6.77%. The results from Table 8.1 also show that using a higher sampling rate is not necessarily beneficial as error rates were higher with some music files with sampling rates of 24 and 48 kHz, which gave an average error rate of 9.35%.

A major cause of errors is the octave problem, which accounts for 49.24% of the errors, in common with the findings of many others, [4], [5], [7], [12], [13]. The octave problem is discussed in Section 4.3.1. Table 8.2 details the octave error that was found for the notes that were played 0.5s apart. The values for the notes detected are taken from Table 8.1. The percentage of octave error is calculated using equation (8.2). The final row in Table 8.2 gives the sum of the notes detected and the sum of the octave errors and these two values are used to calculate the overall percentage of octave error as:

$$\frac{\text{Octave Errors}}{\text{Notes Detected}} \times 100 = \%\text{ of Octave Error} \qquad (8.2)$$

	Notes Detected	**Octave Errors**	**% of Octave Error**
Chords	1906	67	3.52
High Polyphony Chords (5 – 8 notes)	225	15	6.67
Chords (Triads/4 notes)	638	5	0.78
Chords sampled at diff. freq.	139	10	7.19
Total	2908	97	3.34

Table 8.2: The percentage of octave error

From the results in Table 8.2, the octave error rate is 3.34%. From Table 8.1, the overall error rate is 6.77%, therefore the octave error is responsible for almost half of the errors seen.

8.3 Results for Musical Pieces

With a variety of musical pieces played on the keyboard and transcribed by the system, the results are not as good in comparison with the transcription of isolated notes as the error rate dramatically increases. Many errors can be attributed to the onset detection system. In some cases the onset detection system detects extra onsets and this can lead to extra notes being detected. The onset detection system can also miss an onset and this causes a lot of errors as the system tries to transcribe a large section of music that has a mix of frequencies leading to incorrect notes being detected or notes not being detected at all. Table 8.3 shows the results from keyboard tunes being transcribed at different tempos using the onset detection system. Again, error rates are calculated using Equation (8.1).

Tune	Time (s)	Beats/s	No. of Notes Detected	Total in Error	% Error
1-Desperado	61	40	77	17	22.08
1-Desperado	61	60	117	36	30.77
1-Desperado	61	80	162	43	26.54
2-Beethoven – Für Elise	61	40	131	46	35.11
2-Beethoven – Für Elise	61	60	129	74	57.36
2-Beethoven – Für Elise	52	80	119	78	65.55
3-Danny Boy	61	40	120	53	44.17
3-Danny Boy	45	60	102	45	44.12
3-Danny Boy	41	80	104	60	57.69
Total			1061	452	42.6

Table 8.3: Transcribing keyboard songs using the onset detection system

8.3.1 Manual Detection of Onsets

As the incorrect detection of onsets was partially causing the high error rates seen in Table 8.3, for transcribing musical pieces, the onset detection system was removed so that any errors found would be solely due to the masking system. Table

8.4 shows the transcription results when the onsets are manually checked using graphical methods to analyse the placing of the onsets. When the onsets are detected manually the transcription error rate decreases to 20.62%, less then half the error rate when onset detection is done automatically.

Tune	Time (s)	Beats/s	No. of Notes Detected	Total in Error	% Error
1-Desperado	61	40	84	16	19.05
1-Desperado	61	60	123	18	14.63
1-Desperado	61	80	170	25	14.71
2-Beethoven – Für Elise	61	40	142	31	21.83
2-Beethoven – Für Elise	61	60	130	35	26.92
2-Beethoven – Für Elise	52	80	122	25	20.49
3-Danny Boy	61	40	94	21	22.34
3-Danny Boy	45	60	98	23	23.47
3-Danny Boy	41	80	99	25	25.25
Total			1062	219	20.62

Table 8.4: Transcribing keyboard songs using manual onset detection

8.4 Note Deviation from Masking Models

Throughout the project one of the main issues for the masking system to resolve has been to distinguish between the frequency component of an additional note (for example, indicating the presence of a chord) and a harmonic of an already existing note. This problem, for most cases, is overcome by the masking system but in the octave ranges 3, (i.e. C3, C3#, D3…B3), and 4, (i.e. C4, C4#, D4…B4), this problem is not so easily solved. In some cases where notes in these ranges one octave apart are played, e.g. C3 and C4: after analysis the C4 fundamental frequency may appear to be a harmonic of C3, consequently the mask for C4 itself will not be applied and then note C5 may also be detected because it will then have a high

amplitude due to not being reduced by the C4 mask. This phenomenon was discussed in Section 7.3.2.2 and in that example, instead of C#3 and C#4 being correctly detected, notes C#3, C#5 and G#5 were detected.

In the pieces of music tested in this work using the digital keyboard instrument tested in this work, it was noted that this variation in the make up of the notes tends to only occur in notes in the 3rd and 4th octave ranges, and more commonly in notes C and D in those octaves. This may be a characteristic of the instrument used in these experiments or it may be due to a characteristic of this middle-ranking frequency range.

At the higher end of the scale, in the range from the 5th octave and up, (fundamental frequency approximately above 500 Hz), the frequency information becomes easier to manipulate as the amplitude information for the harmonics of the notes played becomes a lot smaller in comparison to the amplitude of the fundamental frequency. As a consequence, the masks are smaller and therefore more accurate detection of the higher frequency notes is possible.

Below, in Table 8.5, is a comparison between the error rate when testing with notes from the 3rd and 4th octave ranges and notes from all other ranges. The percentage error is calculated using Equation (8.1).

	Notes Detected	Extra Notes	Missing Notes	Total Notes in Error	% Error
Notes from octave ranges 3 & 4	219	9	15	24	10.96
Notes from all other ranges	497	6	9	15	3.02

Table 8.5: Results for note detection in the octave ranges 3 and 4 and all ranges

From the table above, Table 8.5, the error rates can be seen to vary depending on the notes played. With notes exclusively from octave ranges 3 and 4, the error is more than three times greater then when notes from all other ranges are played. This variation that can occur in octave ranges 3 and 4 may add to the overall error detected by the system.

8.5 Notes Played in Quick Succession

When analysing the data to manually detect the onsets in the music, a discovery was made about the performance of the note onset detection that affects the performance of the masking step and contributes to the larger error rate for transcription. The difficulty arises when two notes are played in quick succession. The peak of the first note causes interference with the detection of the peak of the second note. Take for example notes A3 and B4, played in quick succession. Figure 8.1 shows a frequency plot of this.

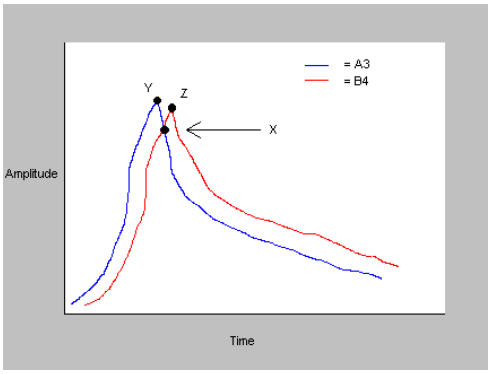


Figure 8.1: Frequency plot of notes A3 & B4 played in quick succession

The onset detection system detects note onsets only, so the offset of a note is defined to be the onset of the next note, as discussed in Section 4.4 and Section 5.2. Therefore, from Figure 8.1 the note A3 lasts from point Y to point Z. Although point

Z is clearly not the offset of the note A3, there is enough information contained in that small window between points Y and Z to correctly identify the note A3. The problem arises when trying to identify the second note, B4. When analysing the second note window, B4 will be detected but note A3 will also be detected. Because note A3 is only starting to decline as note B4 peaks, at point X, note A3 still has a large amplitude and it then appears as if note A3 was played again in the second note window. Thus, instead of correctly detecting A3 in the first window and B4 in the second window, note A3 will be detected in the first window but in the second window, notes A3 and B4 will both be detected.

If it could be established that note A3 in the second window was a continuation of the note played in the first window, then this would not be considered an error. Note A3 is only beginning to decline at the start of the second window, so the amplitude is still large in comparison with the amplitude in the first window and the system determines that note A3 was played again in the second window.

This is a simple example with two notes which have very little harmonic overlapping. When applied to frequencies which have overlapping harmonics, the problem can get quite complex with not only the notes themselves being detected in error but also the harmonic values detected in error. An error like this could also be overcome with a higher level of processing which would take the onset detection information and the polyphonic note detection information and attempt to work out a pattern or rhythm. It could, for example output note detections with a probability estimate of how likely it is to be a 'note played' – but this would require a higher level of processing possibly utilising HMM modeling of rhythm patterns [49].

8.6 Discussion

A major cause of errors is the onset detection. It produces two types of errors: it may return an extra onset or it may miss an onset altogether. In the first case, where an extra onset is detected, this can sometimes be overcome by the polyphonic note detection system. If a spurious onset is detected in the middle of a note that is still being played, an actual note will be divided up into two different note windows. The polyphonic note detection system will likely detect the note in the first window and the same note in the second window as a continuation of the note in the first window.

Most problems occur when an onset is missed and only one note window will be used, when in fact there were two or more notes in succession. Clearly, several different errors could occur depending on the characteristics of the music being analysed. One possibility is that all the notes present will be detected together and it will seem that all the notes in the window were played at the same time. It is also possible, particularly if the fundamental frequencies of the successive notes are close together and the notes are close together in time, that there will not be sufficient frequency resolution to distinguish between the two frequencies.

The masking system is based on information gathered from building a model of the harmonic structure of notes played on the keyboard. The polyphonic note detection system may fail if a note deviates significantly from the assumed model. This problem mainly occurs in the third (C3, C#3, D3…B3), and fourth (C4, C#4, D4…B4), ranges of notes, and is more likely to occur in relation to C and D notes, including sharp, (C#), and flat, (D_b), notes.

The final limitation of the polyphonic masking system as it currently exists arises when two notes are played in quick succession. The peak of the first note causes interference with the detection of the peak of the second note. When applied to frequencies which have overlapping harmonics, the problem can get quite complex and lead to notes being detected in error similar to the problems caused by the onset detection system missing notes.

The results of this system vary depending on the type of music that is being analysed. When notes and chords were played in isolation, the overall error rate was 6.64%. When tested on pieces of music played at a realistic tempo, the error rate jumped to 42.6%. The main problem for the system is when musical pieces are played at speed, the system finds it difficult to detect the onsets of each note and therefore the masking system may be analysing one data window where there should be several windows. This then leads to the problem of either missing notes or incorrectly detected notes. The accuracy of the masking system was improved by manually detecting the onsets and offsets, giving an error rate of 20.62%.

Chapter 9

Conclusion and Recommendations for Further Work

9.1 A masking system for polyphonic music transcription

In this research, a functional system for polyphonic note detection was developed and tested. The system described in this thesis, while not the complete solution to the Automatic Music Transcription problem, presents a different approach to the problem and also addresses some of the issues that have caused problems for other systems in this area.

During the research a full set of note templates for note detection for the Technics KN800 PCM keyboard in note ranges C2 to B6 was developed. The system presented uses an iterative approach to note masking for determination of the note content of the piece of music. The masking approach looks at the frequency domain information between the onset and offset of a note (termed the note window) and tries to extract each individual note from this information. It continues this process

iteratively until all notes have been extracted from the note window information. The system is able to detect notes from both monophonic as well as polyphonic music.

The system does not place restrictions on what type of music or musical style is played. There is also no limitation on the range of notes, the entire range of the keyboard modeled can be detected, although with the digital keyboard used, there are some errors in relation to C and D notes in octave ranges 3 and 4.

In the system implemented in this project the masking procedure provides a good tolerance for noise. Not only are there several checks for noise in the system, making use of a maximum frequency amplitude threshold to eliminate unwanted frequencies, but also, if a frequency is detected that does not match, within a specified tolerance, an actual note in the musical range available to Western tonal music, this frequency value is discarded. This again adds to the effectiveness of the noise tolerance and to the overall robustness of the system.

Another factor that adds to the system is the ability to detect high polyphony, which was a problem in [10], where accuracy degraded with increasing polyphony. During tests of the system the most frequently used number of notes played together was four concurrent notes, but successful tests were carried out on up to and including eight notes played simultaneously. As the system successively removes notes from the note window, having a high polyphony does not significantly affect the detection process or any of the other information in the note window, leading to the more accurate detection of music with a high degree of polyphony.

As discussed previously, the octave problem is a difficult problem in automatic music transcription [4], [5], [7], [12], [13] and while it is not completely solved by this system, the system can handle it reasonably successfully. Again due to the nature of the masking system and the fact that amplitudes at the same frequencies are approximately additive in nature, the detection of notes in consecutive octaves has been possible.

The system described in this thesis is a balanced and robust detection system. Although there are problems with musical pieces that are played with a fast tempo, mainly due to the quality of the onset detection system used as the front-end to the polyphonic transcription, the system is reasonably successful in the detection of the correct notes in general music. This work shows that encouraging results can be achieved with a system based on a note database and using high frequency resolution facilitated by today's increased computing power. While not the perfect solution to the automatic transcription problem, the system has potential and thereby advances the knowledge in this area, in pursuit of the ultimate goal of accurate Automatic Music Transcription.

This research was published in the paper "Automatic Transcription of Polyphonic Piano Music using a Note Masking Technique," and presented at the International Conference on Music Communication Science, 5-7 December 2007, Sydney, Australia.

9.2 Further Work

The key to the masking system is the set of templates built to model the harmonic structure of notes played on the keyboard. The polyphonic note detection system may fail if a note deviates significantly from the assumed template. An example of this situation is if, in a note window being analysed, the second harmonic is abnormally large in comparison with the first harmonic. The polyphonic note detection system will still detect the first harmonic of the note but will not remove enough of the atypical second harmonic, as it is too large in amplitude. Because of this there is an extra peak left in the modified note window and this peak might then, be detected as the fundamental of an additional note despite its actually being a harmonic. This problem is discussed in Section 8.4. This problem suggests that a more sophisticated masking system is required to deal with this problem when notes deviate from the mask models, in particular notes C and D in the 3 and 4 range of notes.

This polyphonic detection system can be used to distinguish polyphonic notes that are played in isolation but the system is less accurate when used on actual musical pieces played at realistic tempos as the front-end onset detection is not accurate enough and this leads to confusion for the masking system. Further research into a better onset detection system would be useful to improve the overall success rate of the transcription system.

Alternatively, for a transcription system the masking system could be developed as a frequency tracker, thereby eliminating the need for a separate onset detection system. The system would track frequencies through the course of the

music. At frequency peaks, the masking system could be used to decide whether the peak was from a fundamental frequency or from a harmonic. After this analysis all the tracked frequencies and masking system decisions would be re-combined and an overall picture of the musical tune could be compiled.

In Section 4.1.2, phase detection was discussed as a method of fundamental frequency detection. While this method was not used in the project, while looking at Figure 4.6 (c), plateaus were seen in the unwrapped phase, which signified note onsets. During analysis for Section 4.1.2, a distinction in the phase between fundamental frequencies and harmonics could not be determined, but further investigation of phase information could lead to such a distinction being made.

It is also possible that phase could be used to discriminate between music played on different instruments which was achieved in [65] using Gaussian Mixture Models. This project focused on automatic transcription of polyphonic keyboard music. The system could be adapted to be used with different instruments, and masks for the specific instruments would need to be created. If, for example, a piece of music contained both keyboard and guitar music, the phase of the music could be used to isolate each instrument in the music. Once isolated, the masking system could be used to discover what notes were played by each individual instrument. This would be a quite complex system and would require substantial further investigation.

References

[1]
Monti, G. and Sandler, M. “Monophonic transcription with autocorrelation,” Proc. COST G-6 Conf. on Digital Audio Effects (DAFX-00), Verona, Italy, 2000.

[2]
Serman, M., Griffith, N. and Serman N. “MusicTracker: a system for modelling melodic dynamics in music performance,” Proc. ICMC, Berlin, Germany, 2000.

[3]
Klapuri, A. “Automatic transcription of music,” Proc. Stockholm Music Acoustics Conference (SMAC 03), Stockholm, Sweden, 2003.

[4]
Marolt, M. “Transcription of polyphonic piano music with neural networks”. Proc. 10th Mediterranean Electrotechnical Conference, vol. 2, pp. 512-515, 2000.

[5]
Marolt, M. “A Connectionist Approach to Automatic Transcription Of Polyphonic Piano Music,” IEEE Trans. Multimedia, vol. 6, no. 3, pp. 439-449, Jun. 2004.

[6]
Tolonen, T. and Karjalainen, M. “A computationally efficient multi-pitch analysis model,” IEEE Trans. Speech Audio Process., vol. 8, no. 6, pp. 708-716, Nov. 2000.

[7]
Martin, K.D.: “A Blackboard System for Automatic Transcription of Simple Polyphonic Music”. MIT Media Lab, Technical Report #385, 1995.

[8]
Walmsley, P. J., Godsill, S. J. and Rayner, P. J. W. “Bayesian modelling of harmonic signals for polyphonic music tracking,” Cambridge Music Processing Colloquium, 30 Sep. 1999.

[9]
Hamid Nawab, S., Ayyash, S.A. and Wotiz, R. “Identification of Musical Chords using Constant-Q Spectra”. Proc. ICASSP, vol. 5, pp. 3373-3376, 2001.

[10]
Klapuri, A., Virtanen, T. and Holm. J.-M. “Robust multipitch estimation for the analysis and manipulation of polyphonic musical signals,” Proc. COST-G6 Conf. on Digital Audio Effects, DAFx-00, Verona, Italy, 2000.

[11]
Klapuri. A. “Number Theoretical Means of Resolving a Mixture of Several Harmonic Sounds,” Proc. EUSIPCO, Rhodes, Greece, 1998.

[12]
Monti, G., M. Sandler: “Automatic Polyphonic Piano Note extraction using Fuzzy Logic in a Blackboard System”. In Proceedings of the 5th International Conference on Digital Audio Effects, Sept. 26 – 28, 2002, Hamburg, Germany.

[13]
Lao, W., E.T. Tan, A.H. Kam: “Computationally Inexpensive and Effective Scheme for Automatic Transcription of Polyphonic Music”. In Proceedings for IEEE International Conference on Multimedia and Expo(ICME), 2004.

[14]
Cooke, M., Modelling auditory processing and organisation, 1993: Cambridge University Press, Cambridge.

[15]
M. Slaney, “Lyon’s Cochlear Model”, Apple Computer Technical Report #13, Apple Computer, Inc., Cupertino, CA 1988.

[16]
Privošnik, M., M. Marolt: “A System for Automatic Transcription of Music Based on Multiple-Agents Architecture”. In Proceedings of MELECON ’98, Tel Aviv, Israel, 1998.

[17]
Minidisc.org, Masking and Perceptual Coding [online], available: http://www.minidisc.org/MaskingPaper.html [accessed 17 Sept 2009].

[18]
Humpage, W. D. and Nguyen, T.T., *Elements of System-Level Software Engineering*, 1987: Dept. of Electrical and Electronic Engineering, Perth, Western Australia.

[19]
Panasonic, Matsushita Electric Industrial Co., Ltd. Technics KN800 PCM Keyboard: http://www.panasonic.com [accessed 17 Sept 2009].

[20]
Music IR (2007) Music-IR.org [online], available: http://www.music-ir.org/ [accessed 17 Sept 2009].

[21]
Schmidt-Jones, C. (2007) What Kind of Music is That? [online], available: http://cnx.org/content/m11421/1.8/ [accessed 17 Sept 2009].

[22]
The Colombia Encyclopedia, Sixth Edition (2008), Arabian music [online], available: http://www.encyclopedia.com/doc/1E1-Arabianm.html [accessed 22nd Feb 2010]

[23]
Encyclopedia Britannica (2009) Octave [online], available: http://www.britannica.com/EBchecked/topic/424821/octave [accessed 17 Sept 2009].

[24]
Encyclopedia Britannica (2009) Scale [online], available: http://www.britannica.com/EBchecked/topic/526314/scale [accessed 17 Sept 2009].

[25]
Georgia State University, HyperPhysics (2005) Fundamental and Harmonics [online], available: http://hyperphysics.phy-astr.gsu.edu/hbase/waves/funhar.html [accessed 17 Sept 2009].

[26]
Encyclopedia Britannica (2009) Chord [online], available: http://www.britannica.com/EBchecked/topic/114441/chord [accessed 18 Sept 2009].

[27]
The Free Dictionary by Farlex, Twelve Tone System [online], available: http://encyclopedia.farlex.com/twelve-tone+system [accessed 17 Sept 2009].

[28]
San Francisco Unified School District (2007) Islamic and Middle Eastern Music and Dance [online], available: http://www.sfusd.k12.ca.us/schwww/sch618/Music/Islam_Music_&_Dance.html [accessed 17 Sept 2009].

[29]
Saltanah Middle Eastern Music (2009) Middle Eastern Music [online], available: http://www.saltanah.com/index.php/about-main.html [accessed 17 Sept 2009].

[30]
Georgia State University, HyperPhysics (2005) History of the Piano [online], available: http://hyperphysics.phy-astr.gsu.edu/hbase/music/pianof.html#c4 [accessed 17 Sept 2009].

[31]
Meixner, K., The Basics of Reading Music [online], available: http://www.notationmachine.com/how_to_read_sheetmusic/readingmusic.htm [accessed 17 Sept 2009].

[32]
Dunne, E. and McConnell, M. “Pianos and Continued Fractions”, Mathematics Magazine, vol. 72, no. 2 (1999), pp. 104-115.

[33]
Michigan Technological University, Department of Physics (1998) Physics of Music – Notes [online], available: http://www.phy.mtu.edu/~suits/scales.html [accessed 17 Sept 2009].

[34]
Matsuoka, T., K. Ito:"Perception of Missing Fundamental and Consideration on it Characteristics". In Proceedings of the 25th Annual Conference of the IEEE EMBS, Cancun, Mexico, September 17-21, 2003

[35]
National Taiwan University (2006) The Case of the Missing Fundamental [online], available:http://ccms.ntu.edu.tw/~karchung/Phonetics%20II%20page%20thirteen.htm [accessed 17 Sept 2009].

[36]
J.D. Larsen and K. Fritsch: "A Valid Demonstration of the Missing Fundamental Illsuion", Teaching of Psychology, v25 n1 p29-20 1998

[37]
Bregman, A.S.: "Auditory Scene Analysis: The Perceptual Organisation of Sound", ISBN: 0-262-02297-4, MIT Press, Cambridge, MA, 1990.

[38]
J. Ferguson, Ed., *Hidden Markov Models for Speech*. Princeton, NJ: IDA, 1980.

[39]
B. H. Juang, "Maximum likelihood estimation for mixture multivariate stochastic observations of Markov chains," *AT&T Tech. J.*, vol. 64, 1985.

[40]
L. R. Rabiner, "A tutorial on hidden Markov models and selected applications in speech recognition," in *Proc. IEEE*, vol. 77, Feb. 1989, pp. 257–286.

[41]
Bristow, G.: "Electronic Speech Recognition: techniques, technology and applications", ISBN: 0-070-07913-7, Collins, London, 1986

[42]
Keller, E.: "Fundamentals of speech synthesis and speech recognition: basic concepts, state of the art, and future challenges", ISBN: 0-471-94449-1, Wiley, Chichester, England, 1994

[43]
Encyclopedia Britannica (2009) Linguistics [online], available: http://www.britannica.com/EBchecked/topic/342418/linguistics [accessed 17 Sept 2009].

[44]
Allen, J.: "Applications of automatic speech recognition to natural language and conversational speech". In R. Stuckless (Ed). Frank W. Lovejoy Symposium on

Applications of Automatic Speech Recognition with Deaf and Hard of Hearing People. Rochester, NY: Rochester Institute of Technology, 1997

[45]
B. Doval and X. Rodet: “Fundamental Frequency Estimation and Tracking using Maximum Likelihood Harmonic Matching and HMMs”, *Proceedings of ICASSP*, 1993.

[46]
R. L. Streit and R. F. Barret: "*Frequency line tracking using hidden Markov models*," IEEE Trans. Signal Processing, vol. 38, pp. 586-598, Apr. 1990.

[47]
S. Bittanti and S. M. Savaresi: “On the parametrization and design of an extended Kalman filter frequency tracker,” *IEEE Trans. Automatic Contr.*, vol. 45, pp. 1718–1724, Sept. 2000.

[48]
M. Padmanabhan and **K.** Martin: “A second-order hyperstable adaptive filter for frequency estimation,” *IEEE Trans. Circuits Syst. IZ,* vol. 40, pp. 398403, June 1993.

[49]
H. Takeda, N. Saito, T. Otsuki, M. Nakai, H. Shimodaira, and S. Sagayama: “Hidden Markov Model for Automatic Transcription of MIDI Signals”. In *2002 International Workshop on Multimedia Signal Processing*, Dec 2002.

[50]
P. Depalle, G. García and X. Rodet: "Tracking of partials for additive sound synthesis using hidden Markov models", In Proceedings of ICASSP-1993 IEEE International Conference on Acoustics, Speech and Signal Processing , 1993

[51]
Liu, R., N. Griffith, J. Walker, P. Murphy. 2003. "Time domain note average energy based music onset detection". In Proceedings of the Stockholm Music Acoustics Conference, Aug. 6-9, 2003, (SMAC'03) Stockholm, pp. 553-556.

[52]
Encyclopedia Britannica (2009) Neural network [online], available: http://www.britannica.com/EBchecked/topic/410549/neural-network [accessed 17 Sept 2009].

[53]
Yuk, D.S., C.W. Che, J. Flanagan: “Robust Speech Recognition using Maximum Likelihood Neural Networks and Continuous Density Hidden Markov Models”. In IEEE Workshop on Automatic Speech Recognition and Understanding, 1997.

[54]
Patterson, R.D., K. Robinson, J. Holdsworth, D. McKeown, C. Zhang and M.H. Allerhand: “Complex sounds and auditory images”, In Auditory Physiology and Perception, (Eds.) Y. Cazals, L. Demany, K. Horner, Pergamon, Exford, 1992.

[55]
Slaney, M.: “An Efficient Implementation of the Patterson-Holdsworth Auditory Filter Bank”, Apple Computer Technical Report #35, 1993.

[56]
Meddis, R.: “Simulation of mechanical to neural transduction in the auditory receptor”, In the Journal of the Acoustical Society of America, vol. 79, no. 3, pp. 702 – 711, March 1986.

[57]
Franklin, D.R., J. Chicharo: “Paganini – A music analysis and recognition program”. In Proceeding of the Fifth International Symposium on Signal Processing and its Applications, ISSPA ’99, Brisbane, Australia, August 1999.

[58]
Brown, Judith C. Musical fundamental frequency tracking using a pattern recognition method. In *Journal of the Acoustical Society of America*, 92(3):1394-1402, 1992.

[59]
Marchand, Sylvain, “An Efficient Pitch-Tracking Algorithm Using a Combination of Fourier Transforms”. In *Proceedings of the COST G-6 Conference on Digital Audio Effects (DAFX-01)*, Limerick, Ireland, December 2001.

[60]
J. W. Cooley and J. W. Tukey, “An algorithm for machine calculation of complex Fourier series”, Math. Comp., 19 (1965), 297-301.

[61]
The MathWorks, Inc. (2009) Matlab [online], available: http://www.mathworks.com/products/matlab/ [accessed 17 Sept 2009].

[62]
Encyclopedia Britannica (2009) Sound [online], available: http://www.britannica.com/EBchecked/topic/555255/sound [accessed 17 Sept 2009].

[63]
Moore, B. C. J. An Introduction to the Psychology of Hearing, 2003: Academic Press.

[64]
Arons, B “A Review of the Cocktail Party Effect”, Journal of the American I/O Society, 1992.

[65]
Slim, E., R. Gaël, D. Bertrand: “Instrument Recognition in Polyphonic Music”. In Proceedings of ICASSP, Philadelphia, March 2005.

[66]
Encyclopedia Britannica (2009) Equal Temperament [online], available: http://www.britannica.com/EBchecked/topic/190596/equal-temperament [accessed 23 Nov 2009].

[67]
Post Musical Instruments [online], available:
http://www.postpiano.com/support/updates/tech/Tuning.htm [accessed 24 Feb 2010]

[68]
Röbel, A: “Onset Detection in Polyphonic Signals by means of Transient Peak Classification”, Music Information Retrieval Evaluation eXchange (MIREX), 2007

[69]
Young, R.W.: ”Terminology for Logarithmic Frequency Units” In the Journal of the Acoustical Society of America, vol. 11, Issue 1, pp. 134-139, July 1939.

Appendix A

Onset Detection Implementation

Figure A1.1 shows the programme structure of the Onset Detection system and is followed by a description of the Onset Detection functions.

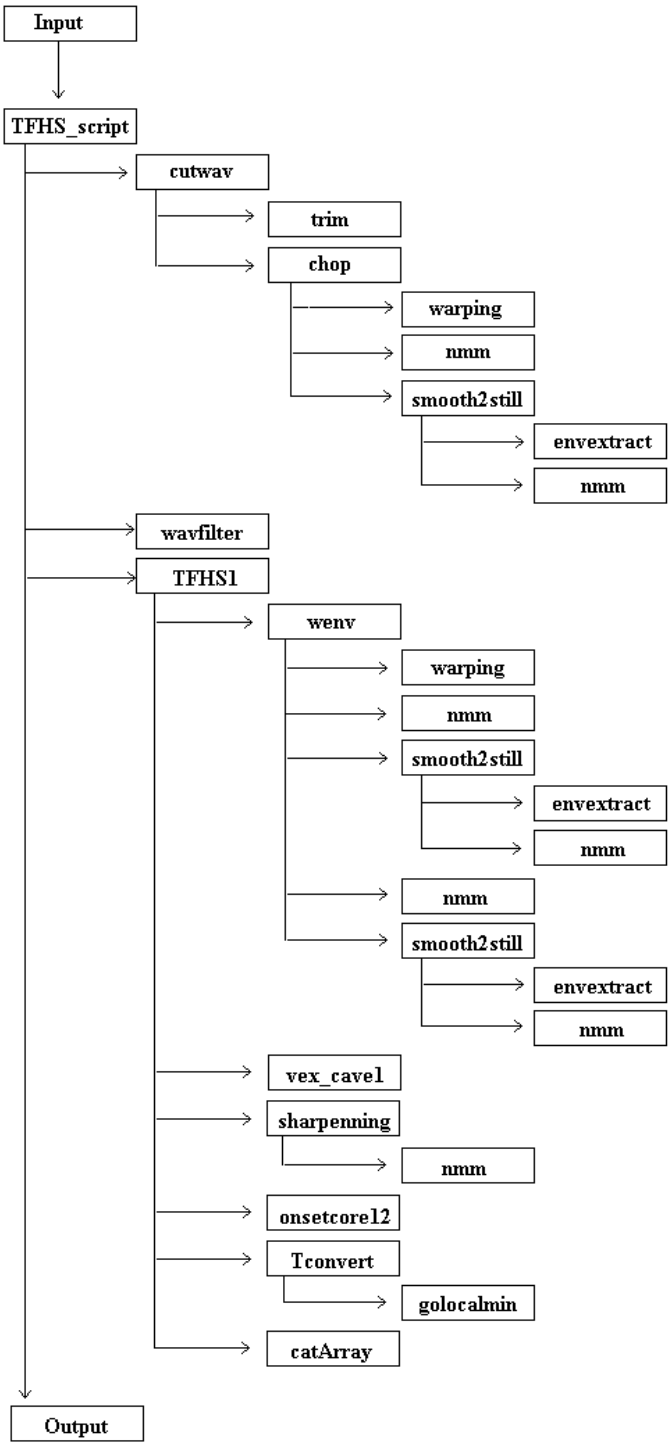


Figure A1.1: Onset Detection System Structure

Functions

TFHS_script: is the main function in the programme. It takes as input a file, (file) and returns the output of the system, the sampled information from the input file, (Y0), the sampling frequency, (FS), the sample numbers at which onsets occur, (sampnums) and the sample number for the last offset of the input file, (maxsamp).

cutwav: uses the wavread function to input the file, and searches for suitable cut-off points near the desired segment length. It calls function *trim* and *chop*.

trim: removes samples at the beginning and end of the file until sample values are at least at a desired cut off ratio of the maximum sample value.

chop: searches for suitable cut-points close to the desired segment length. These cut-points should occur at local minima. It calls functions *warping*, *nmm* and *smooth2still*.

warping: warps the input curve by shrinking a frame of x seconds to be just sampled with the value of the maximum of that frame. Joint frames overlap.

nmm: counts the number of bounces and their indexes in curve power.

smooth2still: smooths repeatedly the input curve until the number of bounces of the input keeps unchanged. It calls functions *envextract* and *nmm*.

envextract: extracts the energy envelope of sound power by the convolution with a Hanning window of x seconds.

wavfilter: filters the stored wave file samples into the number of bands specified by the variable bands.

TFHS1: frame-wise detects onsets in the samples taken from the wave file. It calls functions *wenv, vex_cave1, sharpening, onsetcore12, Tconvert* and *catArray*.

wenv: takes samples of wave file as input and returns the warped and smoothed envelope corresponding to this data. It calls functions *warping, nmm* and *smooth2still*.

vex_cave1: returns 1 when a specific value is a convex curve, -1 for concave curve or 0 for (nearly) direct line.

sharpening: sets all the declines of a variable to be an exponential decline. It calls the function *nmm*.

onsetcore12: is the onset detection method used for versions 1 and 2. Onset detection based on cumulative energy curve.

Tconvert: checks at which positions in the envelope onsets were detected, makes sure these positions are located at local minima and converts the sample number at which these minima occur to a time in seconds. It calls the function *golocalmin*.

golocalmin: searches the local minimum point of a variable, backward/forward according to the relation between that variable at j and at j+1.

catArray: concatenates two arrays of different sizes along the specified dimension padding, the smaller dimensions with zeros if both input arrays are not empty. If the first variable is empty it may return that value or the cat function can be used to produce an array of a specific dimension.

Appendix B

Matlab Code

system1

```
function [noteArray] = system1(file)

filename = file;
TFHS_script

initNotes = [0,0,0,0];
sampnums = [sampnums,maxsamp];
% Adding the last value to sampnums as is not included

% in the sampnums array
for i = 1:(length(sampnums)-1)

    a = round(sampnums(i));
    temp = sampnums(i+1);
    b = (floor(((temp - a)/1024)) * 1024) + a;

    [notes,outputColwAmps] = masksOneWin(Y0,FS,a,b);

    ntsAndAmp = [0,0,0,0];

    for i = 1:length(notes)

        [x,y] = find(outputColwAmps == notes(i));
% Gets the amplitudes form outputColwAmps and adds them
to
        if i == 1
% notes, which is put in a new array called ntsAndAmp
            ntsAndAmp = [notes(i),
outputColwAmps(x,2),a,b];        % Also has the sampnums
which can be used later for the timing
```

```
        else
% information of the notes
            ntsAndAmp = [ntsAndAmp; notes(i),
outputColwAmps(x,2),a,b];
        end

    end

    [x,y] = size(ntsAndAmp);

    if y == 4
        initNotes = [initNotes;ntsAndAmp];
        initNotes = [initNotes;10001,10001,10001,10001];
    end

end

ntsPld = initNotes;

[noteArray] = noteDetect(initNotes,FS);
```

masksOneWin

```
function [notes,outputColwAmps] = masksOneWin(x,fs,a,b)
% Used when function is in a loop for a complete file,
saves
                                                        %
calling wavread several times

                                  % Gets the frequency
domain information for the wave file between certain
[f,fftX] = fftNotes(x,fs,a,b);              % limits,
defined by values a and b

len = length(f);                            % Gets rid of
the second half of the frequency domain information as
f = f(1:round(len/2));                                % it
is only a duplicate of the first half and would interfere
with the
fftX = fftX(1:round(len/2));                          %
calculation of frequencies

[x,y] = max(fftX);
freq = f(y);
```

```
maxAmp = fftX(y);                              % Gets the
maximum amplitude in the frequency domain and its
frequency
count = 1;
flag = 1;
notes = 0;
colwAmps = [round(freq),maxAmp];

while flag == 1
 try
    fftX((y-8):(y+8))=0;
 catch
    fftX(y)=0;
 end
    [x,y] = max(fftX);
    freq = f(y);                               % Gives a
list of frequencies that are greater
    amp = fftX(y);                             % the 0.1 of
the max Amp.

    if (maxAmp * 0.2) < amp
        colwAmps = [colwAmps;round(freq),amp];      %
floor() freq rather then round because the slight
variation
        count = count +1;                              % in
the frequency would give 2 different values for the same
        flag = 1;                                      %
frequency, e.g. 261 & 262 often occur, both are C4.
    else
        flag = 0;
    end

end

colwAmps = sortrows(colwAmps);                 % Sorts
matrix into ascending order starting with the lowest
frequency
outputColwAmps = colwAmps;

ntsFlag = 1;

while colwAmps(1,1) ~= 0                           % Loop
that will continue until either colwAmps equals 0, all
frequencies
                                                   % have
been resolved, or unitl no mask is returned for a
frequency
    flag = 1;                                      % and the
loop is broken
```

```
a = 1;

    [szRow,szCol] = size(colwAmps);

    while flag == 1

        [tmpRow,tmpCol] = size(colwAmps);
        if a > tmpRow
            strtFreq = 0;
            break;
        end

        if colwAmps(a,2) > (maxAmp * 0.2)       % Gets
the first value that's greater then 0.2 of the max amp
because
            strtFreq = colwAmps(a,1);           %
sometimes, at approx 78 Hz, a peak is detected, generally
            flag = 0;                           % noise,
but would be slightly greater than 0.1 of max amp so
would
        else                                    % make it
into the list of frequencies
            a = a+1;
            flag = 1;
        end
    end


    [mask] = getMaskAll(strtFreq);                % Gets
the mask related to a certain frequency (note).
    [initRow,initCol] = find(colwAmps == strtFreq);

    if mask == 0                                % If no
mask is returned, then break from loop.
        break;
    end

    if ntsFlag == 1                             % If the
first time around, then changes notes to equal strtFreq
        notes = strtFreq;                       % else
adds strtFreq to the end notes
        ntsFlag = 0;
    else
        notes = [notes;strtFreq];
    end


    [lenCol,lenRow] = size(mask);
```

```
    maskedVals = [strtFreq,(colwAmps(1,2) -
(colwAmps(1,2) * mask(1,2)))];  % Calculation for the
first value of the mask then takes

% the value away from colwAmps to leave answer in
maskedVals
    for i=3:lenCol                              % Starts
at 4 because first 3 frequency values represent the same
note
        [x,y] = find(colwAmps == mask(i,1));    % Finds
values from maskedVals in colwAmps and subtracts the
values where
        if length(x) ~= 0                       % the
frequencies match
            maskedVals = [maskedVals; mask(i,1),
(colwAmps(x,2) - (colwAmps(initRow,2) * mask(i,2)))];
        end
    end

    [a,b] = size(maskedVals);

    for i=1:a
        [x,y] = find(colwAmps == maskedVals(i,1));
% Changes colwAmps by replacing the matching values from
        if length(x) ~= 0
% maskedVals with those in colwAmps, so colwAmps now has
            colwAmps(x,2) = maskedVals(i,2);
% the final values after the mask calculations have been
        end
% applied.
    end

    [a,b] = size(colwAmps);
    rmningNts = [0,0];
    flgRmnNts = 1;

    for i=1:a
% rmningNts are the values frm colwAmps that are
        if colwAmps(i,2) > (maxAmp * 0.2)
% greater then the maxAmp * 0.2, so as to get rid of
            if flgRmnNts == 1
% any negative and small values that still remain in
colwAmps
                rmningNts =
[colwAmps(i,1),colwAmps(i,2)];
                flgRmnNts = 0;
% flgRmnNts is so that the first time around rmningNts
            else
% is changed to a specific value and after that, the
```

```
                rmningNts =
[rmningNts;colwAmps(i,1),colwAmps(i,2)];    % values are
added to rmningNts
            end
        end
    end


    colwAmps = rmningNts;
% Changes colwAmps to equal rmningNts so values now in

% colwAmps will be used for the next time around the loop
end
```

fftNotes

```
function [f,fftX] = fftNotes(X,fsX,a,b)

% X = Information returned from wavread command
% fsX = sampling frequency of the wave file
% a and b are the limits between which the function is to
ba carried out, on a note

N=(b+1)-a;
w=window(@hamming,N);

X=X(a:b);
x=X.*w;

fftX=fft(x);
fftX=abs(fftX);

f=(0:N-1)'.*((fsX)/N);

fftX(1:10)=0;
```

getMaskAll

```
function [mask] = getMask(freq)

mask = 0;

if (freq >=64 & freq <=124)
```

```
    switch (freq)
        case 64     % C2
            mask = [64,1; 65,1; 66,1;    130,0.74;
131,0.74;    196,0.34; 197,0.34;    261,0.68; 262,0.68;
327,0.26; 328,0.26;    392,0.28; 393,0.28;    457,0.24;
458,0.24;    523,0.12; 524,0.12;    588,0.21; 589,0.21;
654,0.18; 655,0.18;    719,0.25; 720,0.25;    784,0.21;
785,0.21];
        case 65     % C2
            mask = [64,1; 65,1; 66,1;    130,0.74;
131,0.74;    196,0.34; 197,0.34;    261,0.68; 262,0.68;
327,0.26; 328,0.26;    392,0.28; 393,0.28;    457,0.24;
458,0.24;    523,0.12; 524,0.12;    588,0.21; 589,0.21;
654,0.18; 655,0.18;    719,0.25; 720,0.25;    784,0.21;
785,0.21];
        case 66     % C2
            mask = [64,1; 65,1; 66,1;    130,0.74;
131,0.74;    196,0.34; 197,0.34;    261,0.68; 262,0.68;
327,0.26; 328,0.26;    392,0.28; 393,0.28;    457,0.24;
458,0.24;    523,0.12; 524,0.12;    588,0.21; 589,0.21;
654,0.18; 655,0.18;    719,0.25; 720,0.25;    784,0.21;
785,0.21];

        case 68     % C2#
            mask = [68,1; 69,1; 70,1;    138,0.71;
139,0.71;    207,0.31; 208,0.31;    277,0.6; 278,0.6;
346,0.23; 347  0.23;    415,0.25; 416,0.25;    485,0.21;
486,0.21;    762,0.24; 763,0.24];
        case 69     % C2#
            mask = [68,1; 69,1; 70,1;    138,0.71;
139,0.71;    207,0.31; 208,0.31;    277,0.6; 278,0.6;
346,0.23; 347  0.23;    415,0.25; 416,0.25;    485,0.21;
486,0.21;    762,0.24; 763,0.24];
        case 70     % C2#
            mask = [68,1; 69,1; 70,1;    138,0.71;
139,0.71;    207,0.31; 208,0.31;    277,0.6; 278,0.6;
346,0.23; 347  0.23;    415,0.25; 416,0.25;    485,0.21;
486,0.21;    762,0.24; 763,0.24];

        case 72     % D2
            mask = [72,1; 73,1; 74,1;    146,0.66;
147,0.66;    220,0.35; 221,0.35;    293,0.61; 294,0.61;
367,0.23; 368,0.23;    440,0.27; 441,0.27;    513,0.2;
514,0.2;    587,0.1; 588,0.1;    660,0.19; 661,0.19;
734,0.16; 735,0.16;    807,0.23; 808,0.23;    880,0.17;
881,0.17];
        case 73     % D2
            mask = [72,1; 73,1; 74,1;    146,0.66;
147,0.66;    220,0.35; 221,0.35;    293,0.61; 294,0.61;
```

```
367,0.23; 368,0.23;    440,0.27; 441,0.27;    513,0.2;
514,0.2;    587,0.1; 588,0.1;    660,0.19; 661,0.19;
734,0.16; 735,0.16;    807,0.23; 808,0.23;    880,0.17;
881,0.17];
        case 74     % D2
            mask = [72,1; 73,1; 74,1;    146,0.66;
147,0.66;    220,0.35; 221,0.35;    293,0.61; 294,0.61;
367,0.23; 368,0.23;    440,0.27; 441,0.27;    513,0.2;
514,0.2;    587,0.1; 588,0.1;    660,0.19; 661,0.19;
734,0.16; 735,0.16;    807,0.23; 808,0.23;    880,0.17;
881,0.17];

        case 77    % D2#
            mask = [77,1; 78,1;    79,1;      155,0.65;
156,0.65; 233,0.34; 234,0.34;    311,0.58; 312,0.58;
389,0.21; 390,0.21;  466,0.26; 467,0.26];
        case 78     % D2#
            mask = [77,1; 78,1;    79,1;      155,0.65;
156,0.65; 233,0.34; 234,0.34;    311,0.58; 312,0.58;
389,0.21; 390,0.21;  466,0.26; 467,0.26];
        case 79     % D2#
            mask = [77,1; 78,1;    79,1;      155,0.65;
156,0.65; 233,0.34; 234,0.34;    311,0.58; 312,0.58;
389,0.21; 390,0.21;  466,0.26; 467,0.26];

        case 80     % E2
            mask = [80,1; 81,1; 82,1;    164,0.64;
165,0.64;    247,0.35; 248,0.35;    329,0.6; 330,0.6;
412,0.21; 413,0.21;    494,0.26; 495,0.26;    576,0.2;
577,0.2;    741,0.18; 742,0.18;    824,0.14; 825,0.14;
906,0.22; 907,0.22;    988,0.16; 989,0.16];
        case 81     % E2
            mask = [81,1; 82,1; 83,1;    164,0.64;
165,0.64;    247,0.35; 248,0.35;    329,0.6; 330,0.6;
412,0.21; 413,0.21;    494,0.26; 495,0.26;    576,0.2;
577,0.2;    741,0.18; 742,0.18;    824,0.14; 825,0.14;
906,0.22; 907,0.22;    988,0.16; 989,0.16];
        case 82     % E2
            mask = [81,1; 82,1; 83,1;    164,0.64;
165,0.64;    247,0.35; 248,0.35;    329,0.6; 330,0.6;
412,0.21; 413,0.21;    494,0.26; 495,0.26;    576,0.2;
577,0.2;    741,0.18; 742,0.18;    824,0.14; 825,0.14;
906,0.22; 907,0.22;    988,0.16; 989,0.16];
        case 83     % E2
            mask = [81,1; 82,1; 83,1;    164,0.64;
165,0.64;    247,0.35; 248,0.35;    329,0.6; 330,0.6;
412,0.21; 413,0.21;    494,0.26; 495,0.26;    576,0.2;
577,0.2;    741,0.18; 742,0.18;    824,0.14; 825,0.14;
906,0.22; 907,0.22;    988,0.16; 989,0.16];
```

```
        case 86     % F2
            mask = [86,1; 87,1; 88,1;    174,0.49;
175,0.49;    261,0.34; 262,0.34;    349,0.52; 350,0.52;
436,0.17; 437,0.17;    523,0.23; 524,0.23;    611,0.17;
612,0.17;    785,0.16; 786,0.16;    873,0.13; 874,0.13;
960,0.19; 961,0.19;    1047,0.15; 1048,0.15];
        case 87     % F2
            mask = [86,1; 87,1; 88,1;    174,0.49;
175,0.49;    261,0.34; 262,0.34;    349,0.52; 350,0.52;
436,0.17; 437,0.17;    523,0.23; 524,0.23;    611,0.17;
612,0.17;    785,0.16; 786,0.16;    873,0.13; 874,0.13;
960,0.19; 961,0.19;    1047,0.15; 1048,0.15];
        case 88     % F2
            mask = [86,1; 87,1; 88,1;    174,0.49;
175,0.49;    261,0.34; 262,0.34;    349,0.52; 350,0.52;
436,0.17; 437,0.17;    523,0.23; 524,0.23;    611,0.17;
612,0.17;    785,0.16; 786,0.16;    873,0.13; 874,0.13;
960,0.19; 961,0.19;    1047,0.15; 1048,0.15];

        case 92     % F2#
            mask = [92,  1; 93,1; 94,1; 184,0.57;
185,0.57;     277,0.31; 278,0.31;   369,0.54; 370,0.54;
     554,0.23; 555,0.23];
        case 93     % F2#
            mask = [92,  1; 93,1; 94,1; 184,0.57;
185,0.57;     277,0.31; 278,0.31;   369,0.54; 370,0.54;
     554,0.23; 555,0.23];
        case 94     % F2#
            mask = [92,  1; 93,1; 94,1; 184,0.57;
185,0.57;     277,0.31; 278,0.31;   369,0.54; 370,0.54;
     554,0.23; 555,0.23];

        case 97       % G2
            mask = [97,1; 98,1; 99,1;    196,0.47;
197,0.47;    294,0.3; 295,0.3;    392,0.47; 393,0.47;
490,0.16; 491,0.16;    588,0.23; 589,0.23;    686,0.17;
687,0.17;    882,0.14; 883,0.14;    980,0.12; 981,0.12;
1078,0.17; 1079,0.17;    1176,0.14; 1177,0.14];
        case 98       % G2
            mask = [97,1; 98,1; 99,1;    196,0.47;
197,0.47;    294,0.3; 295,0.3;    392,0.47; 393,0.47;
490,0.16; 491,0.16;    588,0.23; 589,0.23;    686,0.17;
687,0.17;    882,0.14; 883,0.14;    980,0.12; 981,0.12;
1078,0.17; 1079,0.17;    1176,0.14; 1177,0.14];
        case 99       % G2
            mask = [97,1; 98,1; 99,1;    196,0.47;
197,0.47;    294,0.3; 295,0.3;    392,0.47; 393,0.47;
490,0.16; 491,0.16;    588,0.23; 589,0.23;    686,0.17;
```

```
687,0.17;    882,0.14; 883,0.14;    980,0.12; 981,0.12;
1078,0.17; 1079,0.17;    1176,0.14; 1177,0.14];

        case 102       % G2#
            mask = [102,1; 103,1;104,1;  207,0.51;
208,0.51; 311,0.31; 312,0.31;  415,0.48; 416,0.48;
     623,0.23; 624,0.23];
        case 103       % G2#
            mask = [102,1; 103,1;104,1;  207,0.51;
208,0.51; 311,0.31; 312,0.31;  415,0.48; 416,0.48;
     623,0.23; 624,0.23];
        case 104       % G2#
            mask = [102,1; 103,1;104,1;  207,0.51;
208,0.51; 311,0.31; 312,0.31;  415,0.48; 416,0.48;
     623,0.23; 624,0.23];

        case 109      % A2
            mask = [109,1; 110,1; 111,1;    220,0.43;
221,0.43;    330,0.31; 331,0.31;    440,0.49; 441,0.49;
550,0.17; 551,0.17;    659,0.23; 660,0.23; 661,0.23;
770,0.17; 771,0.17;    990,0.15; 991,0.15;    1100,0.12;
1101,0.12;    1210,0.18; 1211,0.18;    1320,0.15;
1321,0.15];
        case 110      % A2
            mask = [109,1; 110,1; 111,1;    220,0.43;
221,0.43;    330,0.31; 331,0.31;    440,0.49; 441,0.49;
550,0.17; 551,0.17;    659,0.23; 660,0.23; 661,0.23;
770,0.17; 771,0.17;    990,0.15; 991,0.15;    1100,0.12;
1101,0.12;    1210,0.18; 1211,0.18;    1320,0.15;
1321,0.15];
        case 111      % A2
            mask = [109,1; 110,1; 111,1;    220,0.43;
221,0.43;    330,0.31; 331,0.31;    440,0.49; 441,0.49;
550,0.17; 551,0.17;    659,0.23; 660,0.23; 661,0.23;
770,0.17; 771,0.17;    990,0.15; 991,0.15;    1100,0.12;
1101,0.12;    1210,0.18; 1211,0.18;    1320,0.15;
1321,0.15];

        case 115      % A2#
            mask = [115,1; 116,1; 117,1;     233,0.53;
234,0.53;    349,0.3;    350,0.3;    466,0.51;
     467,0.51;  699,0.22; 700,0.22];
        case 116      % A2#
            mask = [115,1; 116,1; 117,1;     233,0.53;
234,0.53;    349,0.3;    350,0.3;    466,0.51;
     467,0.51;  699,0.22; 700,0.22];
        case 117      % A2#
```

```
            mask = [115,1; 116,1; 117,1;     233,0.53;
234,0.53;    349,0.3;    350,0.3;    466,0.51;
     467,0.51;  699,0.22; 700,0.22];

        case 122      % B2
            mask = [122,1; 123,1; 124,1;    246,0.46;
247,0.46;    370,0.35; 371,0.35;    493,0.54; 494,0.54;
617,0.16; 618,0.16;    740,0.25; 741,0.25;    863,0.17;
864,0.17;    1110,0.15; 1111,0.15;    1234,0.12;
1235,0.12;    1357,0.18; 1358,0.18;    1480,0.14;
1481,0.14];
        case 123      % B2
            mask = [122,1; 123,1; 124,1;    246,0.46;
247,0.46;    370,0.35; 371,0.35;    493,0.54; 494,0.54;
617,0.16; 618,0.16;    740,0.25; 741,0.25;    863,0.17;
864,0.17;    1110,0.15; 1111,0.15;    1234,0.12;
1235,0.12;    1357,0.18; 1358,0.18;    1480,0.14;
1481,0.14];
        case 124      % B2
            mask = [122,1; 123,1; 124,1;    246,0.46;
247,0.46;    370,0.35; 371,0.35;    493,0.54; 494,0.54;
617,0.16; 618,0.16;    740,0.25; 741,0.25;    863,0.17;
864,0.17;    1110,0.15; 1111,0.15;    1234,0.12;
1235,0.12;    1357,0.18; 1358,0.18;    1480,0.14;
1481,0.14];

        otherwise
            mask = 0;
    end
end


if (freq >=129 & freq <=248)
    switch(freq)
        case 129       % C3
            mask = [129,1; 130,1; 131,1;    261,0.94;
262,0.94;    392,0.55; 393,0.55;    523,0.53; 524,0.53;
654,0.42; 655,0.42;    784,0.16; 785,0.16;    915,0.39;
916,0.39;    1046,0.14; 1047,0.14;    1308,0.14;
     1309,0.14;    1569,0.21; 1570,0.21;    1700,0.12;
1701,0.12;    1962,0.11; 1963,0.11];
        case 130       % C3
            mask = [129,1; 130,1; 131,1;    261,0.94;
262,0.94;    392,0.55; 393,0.55;    523,0.53; 524,0.53;
654,0.42; 655,0.42;    784,0.16; 785,0.16;    915,0.39;
916,0.39;    1046,0.14; 1047,0.14;    1308,0.14;
     1309,0.14;    1569,0.21; 1570,0.21;    1700,0.12;
1701,0.12;    1962,0.11; 1963,0.11];
        case 131       % C3
```

```
            mask = [129,1; 130,1; 131,1;    261,0.94;
262,0.94;    392,0.55; 393,0.55;    523,0.53; 524,0.53;
654,0.42; 655,0.42;    784,0.16; 785,0.16;    915,0.39;
916,0.39;    1046,0.14; 1047,0.14;    1308,0.14;
     1309,0.14;    1569,0.21; 1570,0.21;    1700,0.12;
1701,0.12;    1962,0.11; 1963,0.11];

        case 138       % C3#
            mask = [138, 1;   139, 1;   140, 1;   277,
     0.85;     278, 0.85;     415, 0.46;     416, 0.46;
     554, 0.51;     555, 0.51;     693, 0.37;     694,
     0.37;     970, 0.38;     971, 0.38;     1663,
     0.14;     1664,     0.14];
        case 139       % C3#
            mask = [138, 1;   139, 1;   140, 1;   277,
     0.85;     278, 0.85;     415, 0.46;     416, 0.46;
     554, 0.51;     555, 0.51;     693, 0.37;     694,
     0.37;     970, 0.38;     971, 0.38;     1663,
     0.14;     1664,     0.14];
        case 140       % C3#
            mask = [138, 1;   139, 1;   140, 1;   277,
     0.85;     278, 0.85;     415, 0.46;     416, 0.46;
     554, 0.51;     555, 0.51;     693, 0.37;     694,
     0.37;     970, 0.38;     971, 0.38;     1663,
     0.14;     1664,     0.14];

        case 145       % D3
            mask = [145,1; 146,    1;   147, 1;   293,
     0.95;     294, 0.95;     440, 0.53;     441, 0.53;
     587, 0.53;     588, 0.53;     734, 0.42;     735,
     0.42;     880, 0.16;     881, 0.16;     1027,
     0.4; 1028,     0.4; 1174,     0.15;     1175,
     0.15;     1468,     0.14;     1469,     0.14;
     1761,     0.22;     1762,     0.22;     1908,
     0.1; 1909,     0.1; 2202,     0.1; 2203,     0.1];
        case 146       % D3
            mask = [145,1; 146,    1;   147, 1;   293,
     0.95;     294, 0.95;     440, 0.53;     441, 0.53;
     587, 0.53;     588, 0.53;     734, 0.42;     735,
     0.42;     880, 0.16;     881, 0.16;     1027,
     0.4; 1028,     0.4; 1174,     0.15;     1175,
     0.15;     1468,     0.14;     1469,     0.14;
     1761,     0.22;     1762,     0.22;     1908,
     0.1; 1909,     0.1; 2202,     0.1; 2203,     0.1];
        case 147       % D3
            mask = [145,1; 146,    1;   147, 1;   293,
     0.95;     294, 0.95;     440, 0.53;     441, 0.53;
     587, 0.53;     588, 0.53;     734, 0.42;     735,
     0.42;     880, 0.16;     881, 0.16;     1027,
```

```
0.4; 1028,     0.4; 1174,     0.15;     1175,
0.15;     1468,     0.14;     1469,     0.14;
1761,     0.22;     1762,     0.22;     1908,
0.1; 1909,     0.1; 2202,     0.1; 2203,     0.1];

   case 154       % D3#
       mask = [154, 1;   155, 1;   156, 1;   311,
0.92;     312, 0.92;     466, 0.54;     467, 0.54;
622, 0.55;     623, 0.55;     778, 0.42;     779,
0.42;     1089,     0.37;     1090,     0.37;
1866,     0.22;     1867,     0.22];
   case 155       % D3#
       mask = [154, 1;   155, 1;   156, 1;   311,
0.92;     312, 0.92;     466, 0.54;     467, 0.54;
622, 0.55;     623, 0.55;     778, 0.42;     779,
0.42;     1089,     0.37;     1090,     0.37;
1866,     0.22;     1867,     0.22];
   case 156       % D3#
       mask = [154, 1;   155, 1;   156, 1;   311,
0.92;     312, 0.92;     466, 0.54;     467, 0.54;
622, 0.55;     623, 0.55;     778, 0.42;     779,
0.42;     1089,     0.37;     1090,     0.37;
1866,     0.22;     1867,     0.22];

   case 163       % E3
       mask = [163,1; 164,    1;   165, 1;   329,
0.84;     330, 0.84;     494, 0.52;     495, 0.52;
659, 0.52;     660, 0.52;     824, 0.4; 825, 0.4;
988, 0.15;     989, 0.15;     1153,     0.38;
1154,     0.38;     1318,     0.15;     1319,
0.15;     1648,     0.13;     1649,     0.13;
1977,     0.19;     1978,     0.19;     2142,
0.11;     2143,     0.11];
   case 164       % E3
       mask = [163,1; 164,    1;   165, 1;   329,
0.84;     330, 0.84;     494, 0.52;     495, 0.52;
659, 0.52;     660, 0.52;     824, 0.4; 825, 0.4;
988, 0.15;     989, 0.15;     1153,     0.38;
1154,     0.38;     1318,     0.15;     1319,
0.15;     1648,     0.13;     1649,     0.13;
1977,     0.19;     1978,     0.19;     2142,
0.11;     2143,     0.11];
   case 165       % E3
       mask = [163,1; 164,    1;   165, 1;   329,
0.84;     330, 0.84;     494, 0.52;     495, 0.52;
659, 0.52;     660, 0.52;     824, 0.4; 825, 0.4;
988, 0.15;     989, 0.15;     1153,     0.38;
1154,     0.38;     1318,     0.15;     1319,
0.15;     1648,     0.13;     1649,     0.13;
```

```
1977,     0.19;     1978,     0.19;     2142,
0.11;     2143,     0.11];

   case 173       % F3
       mask = [173,1; 174,    1;   175, 1;   349,
0.82;     350, 0.82;     523, 0.53;     524, 0.53;
698, 0.52;     699, 0.52;     873, 0.39;     874,
0.39;     1047,     0.15;     1048,     0.15;
1222,     0.38;     1223,     0.38;     1396,
0.15;     1397,     0.15;     1746,     0.14;
1747,     0.14;     2095,     0.19;     2096,
0.19;     2269,     0.11;     2270,     0.11];
   case 174       % F3
       mask = [173,1; 174,    1;   175, 1;   349,
0.82;     350, 0.82;     523, 0.53;     524, 0.53;
698, 0.52;     699, 0.52;     873, 0.39;     874,
0.39;     1047,     0.15;     1048,     0.15;
1222,     0.38;     1223,     0.38;     1396,
0.15;     1397,     0.15;     1746,     0.14;
1747,     0.14;     2095,     0.19;     2096,
0.19;     2269,     0.11;     2270,     0.11];
   case 175       % F3
       mask = [173,1; 174,    1;   175, 1;   349,
0.82;     350, 0.82;     523, 0.53;     524, 0.53;
698, 0.52;     699, 0.52;     873, 0.39;     874,
0.39;     1047,     0.15;     1048,     0.15;
1222,     0.38;     1223,     0.38;     1396,
0.15;     1397,     0.15;     1746,     0.14;
1747,     0.14;     2095,     0.19;     2096,
0.19;     2269,     0.11;     2270,     0.11];

   case 184       % F3#
       mask = [184, 1;   185, 1;   186, 1;   370,
0.21;     371, 0.21;     555, 0.32;     556, 0.32;
740, 0.28;     741, 0.28;     925, 0.36;     926,
0.36;     1295,     0.5; 1296,     0.5];
   case 185       % F3#
       mask = [184, 1;   185, 1;   186, 1;   370,
0.21;     371, 0.21;     555, 0.32;     556, 0.32;
740, 0.28;     741, 0.28;     925, 0.36;     926,
0.36;     1295,     0.5; 1296,     0.5];
   case 186       % F3#
       mask = [184, 1;   185, 1;   186, 1;   370,
0.21;     371, 0.21;     555, 0.32;     556, 0.32;
740, 0.28;     741, 0.28;     925, 0.36;     926,
0.36;     1295,     0.5; 1296,     0.5];

   case 195       % G3
```

```
       mask = [195,1; 196,    1;   197, 1;   392,
0.21;     393, 0.21;     588, 0.34;     589, 0.34;
784, 0.3; 785, 0.3; 980, 0.38;     981, 0.38;
1372,     0.53;     1373,     0.53;     1764,
0.16;     1765,     0.16];
   case 196       % G3
       mask = [195,1; 196,    1;   197, 1;   392,
0.21;     393, 0.21;     588, 0.34;     589, 0.34;
784, 0.3; 785, 0.3; 980, 0.38;     981, 0.38;
1372,     0.53;     1373,     0.53;     1764,
0.16;     1765,     0.16];
   case 197       % G3
       mask = [195,1; 196,    1;   197, 1;   392,
0.21;     393, 0.21;     588, 0.34;     589, 0.34;
784, 0.3; 785, 0.3; 980, 0.38;     981, 0.38;
1372,     0.53;     1373,     0.53;     1764,
0.16;     1765,     0.16];

   case 206       % G3#
       mask = [206, 1;   207, 1;   208, 1;   622,
0.32;     623, 0.32;     624, 0.32;     830, 0.27;
831, 0.27;     1038,     0.37;     1039,     0.37;
1454,     0.48;     1455,     0.48];
   case 207       % G3#
       mask = [206, 1;   207, 1;   208, 1;   622,
0.32;     623, 0.32;     624, 0.32;     830, 0.27;
831, 0.27;     1038,     0.37;     1039,     0.37;
1454,     0.48;     1455,     0.48];
   case 208       % G3#
       mask = [206, 1;   207, 1;   208, 1;   622,
0.32;     623, 0.32;     624, 0.32;     830, 0.27;
831, 0.27;     1038,     0.37;     1039,     0.37;
1454,     0.48;     1455,     0.48];

   case 219       % A3
       mask = [219,1; 220,    1;   221, 1;   440,
0.3; 441, 0.3; 660, 0.4; 661, 0.4; 880, 0.34;
881, 0.34;     1100,     0.43;     1101,     0.43;
1540,     0.61;     1541,     0.61;     1980,
0.19;     1981,     0.19];
   case 220       % A3
       mask = [219,1; 220,    1;   221, 1;   440,
0.3; 441, 0.3; 660, 0.4; 661, 0.4; 880, 0.34;
881, 0.34;     1100,     0.43;     1101,     0.43;
1540,     0.61;     1541,     0.61;     1980,
0.19;     1981,     0.19];
   case 221       % A3
       mask = [219,1; 220,    1;   221, 1;   440,
0.3; 441, 0.3; 660, 0.4; 661, 0.4; 880, 0.34;
```

```
881, 0.34;     1100,     0.43;     1101,     0.43;
1540,     0.61;     1541,     0.61;     1980,
0.19;     1981,     0.19];

   case 232       % A3#
       mask = [232, 1;   233, 1;   234, 1;   466,
0.21;     467, 0.21;     699, 0.32;     700, 0.32;
932,    0.28;    933,    0.28;     934, 0.28;
1165,     0.39;     1166,     0.39;     1632,
0.52;     1633,     0.52];
   case 233       % A3#
       mask = [232, 1;   233, 1;   234, 1;   466,
0.21;     467, 0.21;     699, 0.32;     700, 0.32;
932,    0.28;    933,    0.28;     934, 0.28;
1165,     0.39;     1166,     0.39;     1632,
0.52;     1633,     0.52];
   case 234       % A3#
       mask = [232, 1;   233, 1;   234, 1;   466,
0.21;     467, 0.21;     699, 0.32;     700, 0.32;
932,    0.28;    933,    0.28;     934, 0.28;
1165,     0.39;     1166,     0.39;     1632,
0.52;     1633,     0.52];

   case 245       % B3
       mask = [245,1; 246,    1;   247, 1;   493,
0.28;     494, 0.28;     740, 0.36;     741, 0.36;
987, 0.34;     988, 0.34;     1234,     0.43;
1235,     0.43;     1728,     0.63;     1729,
0.63;     2222,     0.16;     2223,     0.16];
   case 246       % B3
       mask = [245,1; 246,    1;   247, 1;   493,
0.28;     494, 0.28;     740, 0.36;     741, 0.36;
987, 0.34;     988, 0.34;     1234,     0.43;
1235,     0.43;     1728,     0.63;     1729,
0.63;     2222,     0.16;     2223,     0.16];
   case 247       % B3
       mask = [245,1; 246,    1;   247, 1;   493,
0.28;     494, 0.28;     740, 0.36;     741, 0.36;
987, 0.34;     988, 0.34;     1234,     0.43;
1235,     0.43;     1728,     0.63;     1729,
0.63;     2222,     0.16;     2223,     0.16];
   case 248       % B3
       mask = [246,1; 247,    1;   248, 1;   493,
0.28;     494, 0.28;     740, 0.36;     741, 0.36;
987, 0.34;     988, 0.34;     1234,     0.43;
1235,     0.43;     1728,     0.63;     1729,
0.63;     2222,     0.16;     2223,     0.16];

   otherwise
```

```
            mask = 0;
    end
end


if (freq >=260 & freq <=494)
    switch(freq)
        case 260       % C4
            mask = [260, 1;   261, 1;   262, 1;   523,
     0.72;     524, 0.72;     784, 0.41;     785, 0.41;
     1046,     0.11;     1047,     0.11;     1308,
     0.15;     1309,     0.15;     1569,     0.12;
     1570,     0.12;     1831,     0.25;     1832,
     0.25];
        case 261       % C4
            mask = [260, 1;   261, 1;   262, 1;   523,
     0.72;     524, 0.72;     784, 0.41;     785, 0.41;
     1046,     0.11;     1047,     0.11;     1308,
     0.15;     1309,     0.15;     1569,     0.12;
     1570,     0.12;     1831,     0.25;     1832,
     0.25];
        case 262       % C4
            mask = [260, 1;   261, 1;   262, 1;   523,
     0.72;     524, 0.72;     784, 0.41;     785, 0.41;
     1046,     0.11;     1047,     0.11;     1308,
     0.15;     1309,     0.15;     1569,     0.12;
     1570,     0.12;     1831,     0.25;     1832,
     0.25];

        case 276       % C4#
            mask = [276, 1;   277, 1;   278, 1;   554,
     0.72;     555, 0.72;     831, 0.42;     832, 0.42;
     1940,     0.17;     1941,     0.17];
        case 277       % C4#
            mask = [276, 1;   277, 1;   278, 1;   554,
     0.72;     555, 0.72;     831, 0.42;     832, 0.42;
     1940,     0.17;     1941,     0.17];
        case 278       % C4#
            mask = [276, 1;   277, 1;   278, 1;   554,
     0.72;     555, 0.72;     831, 0.42;     832, 0.42;
     1940,     0.17;     1941,     0.17];

        case 292       % D4
            mask = [292, 1;   293, 1;   294, 1;   587,
     0.73;     588, 0.73;     881, 0.41;     882, 0.41;
     1174,     0.11;     1175,     0.11;     1468,
     0.16;     1469,     0.16;     1762,     0.12;
     1763,     0.12;     2055,     0.26;     2056,
     0.26;     2057,     0.26];
```

```
   case 293       % D4
       mask = [292, 1;   293, 1;   294, 1;   587,
0.73;     588, 0.73;     881, 0.41;     882, 0.41;
1174,     0.11;     1175,     0.11;     1468,
0.16;     1469,     0.16;     1762,     0.12;
1763,     0.12;     2055,     0.26;     2056,
0.26;     2057,     0.26];
   case 294       % D4
       mask = [292, 1;   293, 1;   294, 1;   587,
0.73;     588, 0.73;     881, 0.41;     882, 0.41;
1174,     0.11;     1175,     0.11;     1468,
0.16;     1469,     0.16;     1762,     0.12;
1763,     0.12;     2055,     0.26;     2056,
0.26;     2057,     0.26];

   case 310       % D4#
       mask = [310, 1;   311, 1;   312, 1;   622,
0.77;     623, 0.77;     933, 0.41;     934, 0.41;
2178,     0.28;     2179,     0.28];
   case 311       % D4#
       mask = [310, 1;   311, 1;   312, 1;   622,
0.77;     623, 0.77;     933, 0.41;     934, 0.41;
2178,     0.28;     2179,     0.28];
   case 312       % D4#
       mask = [310, 1;   311, 1;   312, 1;   622,
0.77;     623, 0.77;     933, 0.41;     934, 0.41;
2178,     0.28;     2179,     0.28];

   case 328       % E4
       mask = [328, 1;   329, 1;   330, 1];
   case 329       % E4
       mask = [328, 1;   329, 1;   330, 1];
   case 330       % E4
       mask = [328, 1;   329, 1;   330, 1];

   case 348       % F4
       mask = [348, 1;   349, 1;   350, 1];
   case 349       % F4
       mask = [348, 1;   349, 1;   350, 1];
   case 350       % F4
       mask = [348, 1;   349, 1;   350, 1];

   case 369       % F4#
       mask = [369, 1;   370, 1;   371, 1];
   case 370       % F4#
       mask = [369, 1;   370, 1;   371, 1];
   case 371       % F4#
       mask = [369, 1;   370, 1;   371, 1];
```

```
   case 391       % G4
       mask = [391, 1;   392, 1;   393, 1];
   case 392       % G4
       mask = [391, 1;   392, 1;   393, 1];
   case 393       % G4
       mask = [391, 1;   392, 1;   393, 1];

   case 414       % G4#
       mask = [414, 1;   415, 1;   416, 1;  830,
0.21;     831, 0.21];
   case 415       % G4#
       mask = [414, 1;   415, 1;   416, 1;  830,
0.21;     831, 0.21];
   case 416       % G4#
       mask = [414, 1;   415, 1;   416, 1;  830,
0.21;     831, 0.21];

   case 439       % A4
       mask = [439, 1;   440, 1;   441, 1;   880,
0.2; 881, 0.2; 1320,     0.14;     1321,     0.14;
1760,     0.1; 1761,     0.1];
   case 440       % A4
       mask = [439, 1;   440, 1;   441, 1;   880,
0.2; 881, 0.2; 1320,     0.14;     1321,     0.14;
1760,     0.1; 1761,     0.1];
   case 441       % A4
       mask = [439, 1;   440, 1;   441, 1;   880,
0.2; 881, 0.2; 1320,     0.14;     1321,     0.14;
1760,     0.1; 1761,     0.1];

   case 465       % A4#
       mask = [465, 1;   466, 1;   467, 1;  933,
0.26;     934, 0.26];
   case 466       % A4#
       mask = [465, 1;   466, 1;   467, 1;  933,
0.26;     934, 0.26];
   case 467       % A4#
       mask = [465, 1;   466, 1;   467, 1;  933,
0.26;     934, 0.26];

   case 492       % B4
       mask = [492, 1;   493, 1;   494, 1;   987,
0.21;     988, 0.21;     1481,     0.13;     1482,
0.13];
   case 493       % B4
       mask = [492, 1;   493, 1;   494, 1;   987,
0.21;     988, 0.21;     1481,     0.13;     1482,
0.13];
   case 494       % B4
```

```
            mask = [492, 1;   493, 1;   494, 1;   987,
     0.21;     988, 0.21;     1481,     0.13;     1482,
     0.13];

        otherwise
            mask = 0;
    end
end


if (freq >=522 & freq <=988)
    switch(freq)
        case 522       % C5
            mask = [522, 1;   523, 1;   524, 1;   1046,
0.11;   1047,  0.11;     1048,     0.11];
        case 523       % C5
            mask = [522, 1;   523, 1;   524, 1;   1046,
0.11;   1047,  0.11;     1048,     0.11];
        case 524       % C5
            mask = [522, 1;   523, 1;   524, 1;   1046,
0.11;   1047,  0.11;     1048,     0.11];

        case 554       % C5#
            mask = [554, 1;   555, 1;   556, 1];
        case 555       % C5#
            mask = [554, 1;   555, 1;   556, 1];
        case 556       % C5#
            mask = [554, 1;   555, 1;   556, 1];

        case 586       % D5
            mask = [586, 1;   587, 1;   588, 1;   1174,
     0.11;     1175,     0.11];
        case 587       % D5
            mask = [586, 1;   587, 1;   588, 1;   1174,
     0.11;     1175,     0.11];
        case 588       % D5
            mask = [586, 1;   587, 1;   588, 1;   1174,
     0.11;     1175,     0.11];

        case 621       % D5#
            mask = [621, 1;   622, 1;   623, 1];
        case 622       % D5#
            mask = [621, 1;   622, 1;   623, 1];
        case 623       % D5#
            mask = [621, 1;   622, 1;   623, 1];

        case 658       % E5
            mask = [658, 1;   659, 1;   660, 1;   1318,
     0.11;     1319,     0.11];
```

```
   case 659       % E5
       mask = [658, 1;   659, 1;   660, 1;   1318,
0.11;     1319,     0.11];
   case 660       % E5
       mask = [658, 1;   659, 1;   660, 1;   1318,
0.11;     1319,     0.11];

   case 697       % F5
       mask = [697, 1; 698, 1; 699, 1];
   case 698       % F5
       mask = [697, 1; 698, 1; 699, 1];
   case 699       % F5
       mask = [697, 1; 698, 1; 699, 1];

   case 739       % F5#
       mask = [739, 1; 740, 1; 741, 1];
   case 740       % F5#
       mask = [739, 1; 740, 1; 741, 1];
   case 741       % F5#
       mask = [739, 1; 740, 1; 741, 1];

   case 783       % G5
       mask = [783, 1; 784, 1; 785, 1];
   case 784       % G5
       mask = [783, 1; 784, 1; 785, 1];
   case 785       % G5
       mask = [783, 1; 784, 1; 785, 1];

   case 830       % G5#
       mask = [830, 1; 831, 1; 832, 1];
   case 831       % G5#
       mask = [830, 1; 831, 1; 832, 1];
   case 832       % G5#
       mask = [830, 1; 831, 1; 832, 1];

   case 879       % A5
       mask = [879, 1; 880, 1; 881, 1];
   case 880       % A5
       mask = [879, 1; 880, 1; 881, 1];
   case 881       % A5
       mask = [879, 1; 880, 1; 881, 1];

   case 931       % A5#
       mask = [931, 1; 932, 1; 933, 1];
   case 932       % A5#
       mask = [931, 1; 932, 1; 933, 1];
   case 933       % A5#
       mask = [931, 1; 932, 1; 933, 1];
```

```
        case 986       % B5
            mask = [986, 1; 987, 1; 988, 1];
        case 987       % B5
            mask = [986, 1; 987, 1; 988, 1];
        case 988       % B5
            mask = [986, 1; 987, 1; 988, 1];

        otherwise
            mask = 0;
    end
end


if (freq >=1045 & freq <=1976)
    switch(freq)
        case 1045       % C6
            mask = [1045, 1;    1046, 1;    1047, 1];
        case 1046       % C6
            mask = [1045, 1;    1046, 1;    1047, 1];
        case 1047       % C6
            mask = [1045, 1;    1046, 1;    1047, 1];
        case 1048       % C6
            mask = [1046, 1;    1047, 1;    1048, 1];

        case 1108       % C6#
            mask = [1108, 1;    1109, 1;    1110, 1];
        case 1109       % C6#
            mask = [1108, 1;    1109, 1;    1110, 1];
        case 1110       % C6#
            mask = [1108, 1;    1109, 1;    1110, 1];

        case 1173       % D6
            mask = [1173, 1;    1174, 1;    1175,1];
        case 1174       % D6
            mask = [1173, 1;    1174, 1;    1175,1];
        case 1175       % D6
            mask = [1173, 1;    1174, 1;    1175,1];
        case 1176       % D6
            mask = [1174, 1;    1175, 1;    1176,1];

        case 1244       % D6#
            mask = [1244, 1;    1245, 1;    1246,1];
        case 1245       % D6#
            mask = [1244, 1;    1245, 1;    1246,1];
        case 1246       % D6#
            mask = [1244, 1;    1245, 1;    1246,1];

        case 1317       % E6
            mask = [1317, 1;    1318, 1;    1319, 1];
```

```
case 1318       % E6
    mask = [1317, 1;    1318, 1;    1319, 1];
case 1319       % E6
    mask = [1317, 1;    1318, 1;    1319, 1];

case 1396       % F6
    mask = [1396, 1;    1397, 1;    1398, 1];
case 1397       % F6
    mask = [1396, 1;    1397, 1;    1398, 1];
case 1398       % F6
    mask = [1396, 1;    1397, 1;    1398, 1];

case 1479       % F6#
    mask = [1479, 1;    1480, 1;    1481, 1];
case 1480       % F6#
    mask = [1479, 1;    1480, 1;    1481, 1];
case 1481       % F6#
    mask = [1479, 1;    1480, 1;    1481, 1];

case 1567       % G6
    mask = [1567, 1;    1568, 1;    1569, 1];
case 1568       % G6
    mask = [1567, 1;    1568, 1;    1569, 1];
case 1569       % G6
    mask = [1567, 1;    1568, 1;    1569, 1];

case 1660       % G6#
    mask = [1660, 1;    1661, 1;    1662, 1];
case 1661       % G6#
    mask = [1660, 1;    1661, 1;    1662, 1];
case 1662       % G6#
    mask = [1660, 1;    1661, 1;    1662, 1];

case 1759       % A6
    mask = [1759, 1;    1760, 1;    1761, 1];
case 1760       % A6
    mask = [1759, 1;    1760, 1;    1761, 1];
case 1761       % A6
    mask = [1759, 1;    1760, 1;    1761, 1];

case 1864       % A6#
    mask = [1864, 1;    1865, 1;    1866, 1];
case 1865       % A6#
    mask = [1864, 1;    1865, 1;    1866, 1];
case 1866       % A6#
    mask = [1864, 1;    1865, 1;    1866, 1];

case 1974       % B6
    mask = [1974, 1;    1975, 1;    1976, 1];
```

```
        case 1975       % B6
            mask = [1974, 1;    1975, 1;    1976, 1];
        case 1976       % B6
            mask = [1974, 1;    1975, 1;    1976, 1];

        otherwise
            mask = 0;
    end
end


if (freq >=2092 & freq <=2094)
    switch(freq)
        case 2092       % C7
            mask = [2092, 1;    2093, 1;    2094, 1];
        case 2093       % C7
            mask = [2092, 1;    2093, 1;    2094, 1];
        case 2094       % C7
            mask = [2092, 1;    2093, 1;    2094, 1];

        otherwise
            mask = 0;
    end

end
```

noteDetect

```
function [noteArray] = noteDetect(initNotes,FS)

thresholdArray = [0,0,0,0];

for i=1:length(initNotes)


                  if    initNotes    (i,1)    >    1044
% Using 10 as a threshold value,
                                        thres    =    4;
% anything below this is not included,
                                                  else
% unless frequency is above 1044, in the range of 6
                                       thres    =    10;
% as amplitudes tend to decrease in magnitude
                                                   end
% i some of the higher frequency notes.
```

```
    if initNotes(i,2) > thres
                                        thresholdArray    =
[thresholdArray;initNotes(i,1),initNotes(i,2),initNotes(i
,3),initNotes(i,4)];
    end

end

noteArray = thresholdArray;

totalLoopVal = find(thresholdArray == 10001);
loopVal = ((length(totalLoopVal))/4)-1;

count = 2;
regressCount = 2;

completeTempArray = [-1,-1,-1,-1];
tmpArraySizes = 0;

for i = 1:loopVal

    tempArray1 = [-1,-1,-1,-1];
    tempArray2 = [-1,-1,-1,-1];


    count = regressCount;

    if count >= length(thresholdArray)
        break;
    end

    while thresholdArray(count,1) == 10001
        count = count +1;
    end


    while thresholdArray(count,1) ~= 10001
                                            tempArray1    =
[tempArray1;thresholdArray(count,1),thresholdArray(count,
2),thresholdArray(count,3),thresholdArray(count,4)];
        count = count+1;
        regressCount = count + 1;
    end

    count = count +1;

    if count < length(thresholdArray)
        while thresholdArray(count,1) ~= 10001
```

```
                                                tempArray2    =
[tempArray2;thresholdArray(count,1),thresholdArray(count,
2),thresholdArray(count,3),thresholdArray(count,4)];
            count = count+1;
        end
    end

    [x1,y1] = size(tempArray1);
    [x2,y2] = size(tempArray2);
    tmpArraySizes = [tmpArraySizes;x1;x2];

    for k = 2:x1

        val = tempArray1(k,1);
        ampVal = tempArray1(k,2);

        for j = 2:x2
                if (val == tempArray2(j,1) || (val-1) ==
tempArray2(j,1)   ||   (val+1)   ==   tempArray2(j,1))   &
(tempArray2(j,2) < (ampVal * 0.5))
                                        tempArray1(k,4)  =  0;
% Threshold, value has to be greater than
                                        tempArray2(j,3)  =  0;
% 50% of the previous same note to be considered
                                                          end
% as a new note. If below threshold, then is a
                                                          end
% continueing note. Also some notes will vary slightly

% in frequency so have val +/- 1 to account for this.
    end


    if i == loopVal
                                        completeTempArray    =
[completeTempArray;tempArray1;tempArray2];
    elseif i ~= loopVal
                                        completeTempArray    =
[completeTempArray;tempArray1];
    end

end

noteArray = completeTempArray;

[row,col] = size(completeTempArray);

tmpNoteArray = completeTempArray';
[x,y] = find (tmpNoteArray == 0);
```

```
[he,wi] = size(tmpNoteArray);

pos = (he * (y-1)) + x;
freqPos = pos -3;
tmpVal = 1;

for m = 1:length(x)

    for n = pos(1)+1:he*wi
        if tmpNoteArray(n) == tmpNoteArray(freqPos(1)) ||
tmpNoteArray(n)    ==    tmpNoteArray(freqPos(1))-1    ||
tmpNoteArray(n) == tmpNoteArray(freqPos(1))+1
            tmpNoteArray(n+2) = 0;
            freqPos = freqPos(2:length(freqPos));
            pos = pos(2:length(pos));
            break;
        end
    end

end

completeTempArray = tmpNoteArray';

tempNote = [0];

for n = 1:row
    if n == 1
        tempNote = [completeTempArray(n)];
    else
        tempNote = [tempNote;completeTempArray(n)];
    end
end

[finalNts] = numToNoteMaskAll(tempNote);

tempTimes = [0,0];

for n = 1:row
    if n == 1
                                                tempTimes     =
[completeTempArray(n,3),completeTempArray(n,4),];
    else
                                                tempTimes     =
[tempTimes;completeTempArray(n,3),completeTempArray(n,4)]
;
    end
end

times = {'Start','Finish';'Time (sec)','Time (sec)'};
```

```
for i = 1:row-2

    temp = {'a','a'};
    times = [times;temp];

end

for p = 1:2
    for n = 3:row

        if tempTimes(n,p) == -1
            times(n,p) = {'------'};
        elseif tempTimes(n,p) == 0
            times(n,p) = {'contd.'};
        else
            tmpVal = num2str(tempTimes(n,p)/FS);
            val = cellstr(tmpVal);
            times(n,p) = (val);
        end
    end
end

noteArray = [finalNts times];
```

numToNoteMaksAll

```
function [finalNts] = numToNoteMask(allNotes)

len = length(allNotes);
finalNts = {'------'};

for i=1:len

      if allNotes(i) == -1
          temp = {'------'};
          if i == 1
              finalNts = [temp];
          else
            finalNts = [finalNts;temp];
          end
      end


    if (allNotes(i) >= 64 & allNotes(i) <= 124)  % ||
allNotes(i) == 0

        switch(allNotes(i))
            case 64
```

```
    temp = {'C2'};
case 65
    temp = {'C2'};
case 66
    temp = {'C2'};

case 68
    temp = {'C#2'};
case 69
    temp = {'C#2'};
case 70
    temp = {'C#2'};

case 72
    temp = {'D2'};
case 73
    temp = {'D2'};
case 74
    temp = {'D2'};

case 77
    temp = {'D#2'};
case 78
    temp = {'D#2'};
case 79
    temp = {'D#2'};

case 81
    temp = {'E2'};
case 82
    temp = {'E2'};
case 83
    temp = {'E2'};

case 86
    temp = {'F2'};
case 87
    temp = {'F2'};
case 88
    temp = {'F2'};

case 92
    temp = {'F#2'};
case 93
    temp = {'F#2'};
case 94
    temp = {'F#2'};

case 97
```

```
                temp = {'G2'};
            case 98
                temp = {'G2'};
            case 99
                temp = {'G2'};

            case 102
                temp = {'G#2'};
            case 103
                temp = {'G#2'};
            case 104
                temp = {'G#2'};

            case 109
                temp = {'A2'};
            case 110
                temp = {'A2'};
            case 111
                temp = {'A2'};

            case 115
                temp = {'A#2'};
            case 116
                temp = {'A#2'};
            case 117
                temp = {'A#2'};

            case 122
                temp = {'B2'};
            case 123
                temp = {'B2'};
            case 124
                temp = {'B2'};
        end

        finalNts = [finalNts;temp];
    end


    if (allNotes(i) >= 129 & allNotes(i) <= 248)  % ||
allNotes(i) == 0

        switch(allNotes(i))
            case 129
                temp = {'C3'};
            case 130
                temp = {'C3'};
            case 131
                temp = {'C3'};
```

```
case 138
    temp = {'C#3'};
case 139
    temp = {'C#3'};
case 140
    temp = {'C#3'};

case 145
    temp = {'D3'};
case 146
    temp = {'D3'};
case 147
    temp = {'D3'};

case 154
    temp = {'D#3'};
case 155
    temp = {'D#3'};
case 156
    temp = {'D#3'};

case 163
    temp = {'E3'};
case 164
    temp = {'E3'};
case 165
    temp = {'E3'};

case 173
    temp = {'F3'};
case 174
    temp = {'F3'};
case 175
    temp = {'F3'};

case 184
    temp = {'F#3'};
case 185
    temp = {'F#3'};
case 186
    temp = {'F#3'};

case 195
    temp = {'G3'};
case 196
    temp = {'G3'};
case 197
    temp = {'G3'};
```

```
            case 206
                temp = {'G#3'};
            case 207
                temp = {'G#3'};
            case 208
                temp = {'G#3'};

            case 219
                temp = {'A3'};
            case 220
                temp = {'A3'};
            case 221
                temp = {'A3'};

            case 232
                temp = {'A#3'};
            case 233
                temp = {'A#3'};
            case 234
                temp = {'A#3'};

            case 245
                temp = {'B3'};
            case 246
                temp = {'B3'};
            case 247
                temp = {'B3'};
            case 248
                temp = {'B3'};
        end

        finalNts = [finalNts;temp];
    end


    if (allNotes(i) >= 260 & allNotes(i) <= 494)  % ||
allNotes(i) == 0

        switch(allNotes(i))
            case 260
                temp = {'C4'};
            case 261
                temp = {'C4'};
            case 262
                temp = {'C4'};

            case 276
                temp = {'C#4'};
            case 277
```

```
    temp = {'C#4'};
case 278
    temp = {'C#4'};

case 292
    temp = {'D4'};
case 293
    temp = {'D4'};
case 294
    temp = {'D4'};

case 310
    temp = {'D#4'};
case 311
    temp = {'D#4'};
case 312
    temp = {'D#4'};

case 328
    temp = {'E4'};
case 329
    temp = {'E4'};
case 330
    temp = {'E4'};

case 348
    temp = {'F4'};
case 349
    temp = {'F4'};
case 350
    temp = {'F4'};

case 369
    temp = {'F#4'};
case 370
    temp = {'F#4'};
case 371
    temp = {'F#4'};

case 391
    temp = {'G4'};
case 392
    temp = {'G4'};
case 393
    temp = {'G4'};

case 414
    temp = {'G#4'};
case 415
```

```
            temp = {'G#4'};
        case 416
            temp = {'G#4'};

        case 439
            temp = {'A4'};
        case 440
            temp = {'A4'};
        case 441
            temp = {'A4'};

        case 465
            temp = {'A#4'};
        case 466
            temp = {'A#4'};
        case 467
            temp = {'A#4'};

        case 492
            temp = {'B4'};
        case 493
            temp = {'B4'};
        case 494
            temp = {'B4'};
    end

    finalNts = [finalNts;temp];
end


    if (allNotes(i) >= 522 & allNotes(i) <= 988)  % ||
allNotes(i) == 0

    switch(allNotes(i))
        case 522
            temp = {'C5'};
        case 523
            temp = {'C5'};
        case 524
            temp = {'C5'};

        case 554
            temp = {'C#5'};
        case 555
            temp = {'C#5'};
        case 556
            temp = {'C#5'};

        case 586
```

```
    temp = {'D5'};
case 587
    temp = {'D5'};
case 588
    temp = {'D5'};

case 621
    temp = {'D#5'};
case 622
    temp = {'D#5'};
case 623
    temp = {'D#5'};

case 658
    temp = {'E5'};
case 659
    temp = {'E5'};
case 660
    temp = {'E5'};

case 697
    temp = {'F5'};
case 698
    temp = {'F5'};
case 699
    temp = {'F5'};

case 739
    temp = {'F#5'};
case 740
    temp = {'F#5'};
case 741
    temp = {'F#5'};

case 783
    temp = {'G5'};
case 784
    temp = {'G5'};
case 785
    temp = {'G5'};

case 830
    temp = {'G#5'};
case 831
    temp = {'G#5'};
case 832
    temp = {'G#5'};

case 879
```

```
            temp = {'A5'};
        case 880
            temp = {'A5'};
        case 881
            temp = {'A5'};

        case 931
            temp = {'A#5'};
        case 932
            temp = {'A#5'};
        case 933
            temp = {'A#5'};

        case 986
            temp = {'B5'};
        case 987
            temp = {'B5'};
        case 988
            temp = {'B5'};
    end

    finalNts = [finalNts;temp];
end


    if (allNotes(i) >= 1045 & allNotes(i) <= 1976)  % ||
allNotes(i) == 0

    switch(allNotes(i))
        case 1045
            temp = {'C6'};
        case 1046
            temp = {'C6'};
        case 1047
            temp = {'C6'};
        case 1048
            temp = {'C6'};

        case 1108
            temp = {'C#6'};
        case 1109
            temp = {'C#6'};
        case 1110
            temp = {'C#6'};

        case 1173
            temp = {'D6'};
        case 1174
            temp = {'D6'};
```

```
case 1175
    temp = {'D6'};
case 1176
    temp = {'D6'};

case 1244
    temp = {'D#6'};
case 1245
    temp = {'D#6'};
case 1246
    temp = {'D#6'};

case 1317
    temp = {'E6'};
case 1318
    temp = {'E6'};
case 1319
    temp = {'E6'};

case 1396
    temp = {'F6'};
case 1397
    temp = {'F6'};
case 1398
    temp = {'F6'};

case 1479
    temp = {'F#6'};
case 1480
    temp = {'F#6'};
case 1481
    temp = {'F#6'};

case 1567
    temp = {'G6'};
case 1568
    temp = {'G6'};
case 1569
    temp = {'G6'};

case 1660
    temp = {'G#6'};
case 1661
    temp = {'G#6'};
case 1662
    temp = {'G#6'};

case 1759
    temp = {'A6'};
```

```
            case 1760
                temp = {'A6'};
            case 1761
                temp = {'A6'};

            case 1864
                temp = {'A#6'};
            case 1865
                temp = {'A#6'};
            case 1866
                temp = {'A#6'};

            case 1974
                temp = {'B6'};
            case 1975
                temp = {'B6'};
            case 1976
                temp = {'B6'};
        end

        finalNts = [finalNts;temp];
    end


     if (allNotes(i) >= 2092 & allNotes(i) <= 2094)  % ||
allNotes(i) == 0

        switch(allNotes(i))
            case 2092
                temp = {'C7'};
            case 2093
                temp = {'C7'};
            case 2094
                temp = {'C7'};
        end

        finalNts = [finalNts;temp];
    end

end
```

Appendix C

Masks

The masks used in this project are listed below.

C2

Frequency (Hz)	Masking Value
64, 65, 66	1
130, 131	0.74
196, 197	0.34
261, 262	0.68
327, 328	0.26
392, 393	0.28
457, 458	0.24
523, 524	0.12
588, 589	0.21
654, 655	0.18
719, 720	0.25
784, 785	0.21

C#2

Frequency (Hz)	Masking Value
68, 69 ,70	1
138, 139	0.71
207, 208	0.31
277, 278	0.6
346, 347	0.23
415, 416	0.25
485, 486	0.21
762, 763	0.24

D2

Frequency (Hz)	Masking Value
72, 73, 74	1
146, 147	0.66
220, 221	0.35
293, 294	0.61
367, 368	0.23
440, 441	0.27
513, 514	0.2
587, 588	0.1
660, 661	0.19
734, 735	0.16
807, 808	0.23
880, 881	0.17

D#2

Frequency (Hz)	Masking Value
77, 78, 79	1
155, 156	0.65
233, 234	0.34
311, 312	0.58
389, 390	0.21
466, 467	0.26

E2

Frequency (Hz)	Masking Value
80, 81, 82	1
164, 165	0.64
247, 248	0.35
329, 330	0.6
412, 413	0.21
494, 495	0.26
576, 577	0.2
741, 742	0.18
824, 825	0.14
906, 907	0.22
988, 989	0.16

F2

Frequency (Hz)	Masking Value
86, 87, 88	1
174, 175	0.49
261, 262	0.34
349, 350	0.52
436, 437	0.17
523, 524	0.23
611, 612	0.17
785, 786	0.16
873, 874	0.13
960, 961	0.19
1047, 1048	0.15

F#2

Frequency (Hz)	Masking Value
92, 93, 94	1
184, 185	0.57
277, 278	0.31
369, 370	0.54
554, 555	0.23

G2

Frequency (Hz)	Masking Value
97, 98, 99	1
196, 197	0.47
294, 295	0.3
392, 393	0.47
490, 491	0.16
588, 589	0.23
686, 687	0.17
882, 883	0.14
980, 981	0.12
1078, 1079	0.17
1176, 1177	0.14

G#2

Frequency (Hz)	Masking Value
102, 103, 104	1
207, 208	0.51
311, 312	0.31
415, 416	0.48
632, 624	0.23

A2

Frequency (Hz)	Masking Value
109, 110, 111	1
220, 221	0.43
330, 331	0.31
440, 441	0.49
550, 551	0.17
659, 660, 661	0.23
770, 771	0.17
990, 991	0.15
1100, 1101	0.12
1210, 1211	0.18
1320, 1321	0.15

A#2

Frequency (Hz)	Masking Value
115, 116, 117	1
233, 234	0.53
349, 350	0.3
466, 467	0.51
699, 700	0.22

B2

Frequency (Hz)	Masking Value
122, 123, 124	1
246, 247	0.46
370, 371	0.35
493, 494	0.54
617, 618	0.16
740, 741	0.25
863, 864	0.17
1110, 1111	0.15
1234, 1235	0.12
1357, 1358	0.18
1480, 1481	0.14

C3

Frequency (Hz)	Masking Value
129, 130, 131	1
261, 262	0.94
392, 393	0.55
523, 524	0.53
654, 655	0.52
784, 785	0.16
915, 916	0.39
1046, 1047	0.14
1308, 1309	0.14
1569, 1570	0.21
1700, 1701	0.12
1962, 1963	0.11

C#3

Frequency (Hz)	Masking Value
138, 139, 140	1
277, 278	0.85
415, 416	0.46
554, 555	0.51
693, 694	0.37
970, 971	0.38
1663, 1664	0.14

D3

Frequency (Hz)	Masking Value
145, 146, 147	1
293, 294	0.95
440, 441	0.53
587, 588	0.53
734, 735	0.42
880, 881	0.16
1027, 1028	0.4
1174, 1175	0.15
1468, 1469	0.14
1761, 1762	0.22
1908, 1909	0.1
2202, 2203	0.1

D#3

Frequency (Hz)	Masking Value
154, 155, 156	1
311, 312	0.92
466, 467	0.54
622, 623	0.55
778, 779	0.42
1089, 1090	0.37
1866, 1867	0.22

E3

Frequency (Hz)	Masking Value
163, 164, 165	1
329, 330	0.84
494, 495	0.52
659, 660	0.52
824, 825	0.4
988, 989	0.15
1153, 1154	0.38
1318, 1319	0.15
1648, 1649	0.13
1977, 1978	0.19
2142, 2143	0.11

F3

Frequency (Hz)	Masking Value
173, 174, 175	1
349, 350	0.82
523, 524	0.53
698, 699	0.52
873, 874	0.39
1047, 1048	0.15
1222, 1223	0.38
1396, 1397	0.15
1746, 1747	0.14
2095, 2096	0.19
2269, 2270	0.11

F#3

Frequency (Hz)	Masking Value
184, 185, 186	1
370, 371	0.21
555, 556	0.32
740, 741	0.28
925, 936	0.36
1295, 1296	0.5

G3

Frequency (Hz)	Masking Value
195, 196, 197	1
392, 393	0.21
588, 589	0.34
784, 785	0.3
980, 981	0.38
1372, 1373	0.53
1764, 1765	0.16

G#3

Frequency (Hz)	Masking Value
206, 207, 208	1
622, 623, 624	0.32
830, 831	0.27
1038, 1039	0.37
1454, 1455	0.48

A3

Frequency (Hz)	Masking Value
219, 220, 221	1
440, 441	0.3
660, 661	0.4
880, 881	0.34
1100, 1101	0.43
1540, 1541	0.61
1980, 1981	0.19

A#3

Frequency (Hz)	Masking Value
232, 233, 234	1
466, 467	0.21
699, 700	0.32
932, 933, 934	0.28
1165, 1166	0.39
1632, 1633	0.52

B3

Frequency (Hz)	Masking Value
245, 246, 247	1
493, 494	0.28
740, 741	0.36
987, 988	0.34
1234, 1235	0.43
1728, 1729	0.63
2222, 2223	0.16

C4

Frequency (Hz)	Masking Value
260, 261, 262	1
523, 524	0.72
784, 785	0.41
1046, 1047	0.11
1308, 1309	0.15
1569, 1570	0.12
1831, 1832	0.25

C#4

Frequency (Hz)	Masking Value
276, 277, 278	1
554, 555	0.72
831, 832	0.42
1940, 1941	0.17

D4

Frequency (Hz)	Masking Value
292, 293, 294	1
587, 588	0.73
881, 882	0.41
1174, 1175	0.11
1468, 1469	0.16
1762, 1763	0.12
2055, 2056, 2057	0.26

D#4

Frequency (Hz)	Masking Value
310, 311, 312	1
622, 623	0.77
933, 934	0.41
2178, 2179	0.28

E4

Frequency (Hz)	Masking Value
328, 329, 330	1

F4

Frequency (Hz)	Masking Value
348, 349, 350	1

F#4

Frequency (Hz)	Masking Value
369, 370, 371	1

G4

Frequency (Hz)	Masking Value
391, 392, 393	1

G#4

Frequency (Hz)	Masking Value
414, 415, 416	1
830, 831	0.21

A4

Frequency (Hz)	Masking Value
439, 440, 441	1
880, 881	0.2
1320, 1321	0.14
1760, 1761	0.1

A#4

Frequency (Hz)	Masking Value
465, 466, 467	1
933, 934	0.26

B4

Frequency (Hz)	Masking Value
492, 493, 494	1
987, 988	0.21
1481, 1482	0.13

C5

Frequency (Hz)	Masking Value
522, 523, 524	1
1046, 1047, 1048	0.11

C#5

Frequency (Hz)	Masking Value
554, 555, 556	1

D5

Frequency (Hz)	Masking Value
586, 587, 588	1
1174, 1175	0.11

D#5

Frequency (Hz)	Masking Value
621, 622, 623	1

E5

Frequency (Hz)	Masking Value
658, 659, 660	1
1318, 1319	0.11

F5

Frequency (Hz)	Masking Value
697, 698, 699	1

F#5

Frequency (Hz)	Masking Value
739, 740, 741	1

G5

Frequency (Hz)	Masking Value
783, 784, 785	1

G#5

Frequency (Hz)	Masking Value
830, 831, 832	1

A5

Frequency (Hz)	Masking Value
879, 880, 881	1

A#5

Frequency (Hz)	Masking Value
931, 932, 933	1

B5

Frequency (Hz)	Masking Value
986, 987, 988	1

C6

Frequency (Hz)	Masking Value
1045, 1046, 1047, 1048	1

C#6

Frequency (Hz)	Masking Value
1108, 1109, 1110	1

D6

Frequency (Hz)	Masking Value
1173, 1174, 1175, 1176	1

D#6

Frequency (Hz)	Masking Value
1244, 1245, 1246	1

E6

Frequency (Hz)	Masking Value
1317, 1318, 1319	1

F6

Frequency (Hz)	Masking Value
1396, 1397, 1398	1

F#6

Frequency (Hz)	Masking Value
1479, 1480, 1481	1

G6

Frequency (Hz)	Masking Value
1567, 1568, 1569	1

G#6

Frequency (Hz)	Masking Value
1660, 1661, 1662	1

A6

Frequency (Hz)	Masking Value
1759, 1760, 1761	1

A#6

Frequency (Hz)	Masking Value
1864, 1865, 1866	1

B6

Frequency (Hz)	Masking Value
1974, 1975, 1976	1

C#7

Frequency (Hz)	Masking Value
2092, 2093, 2094	1